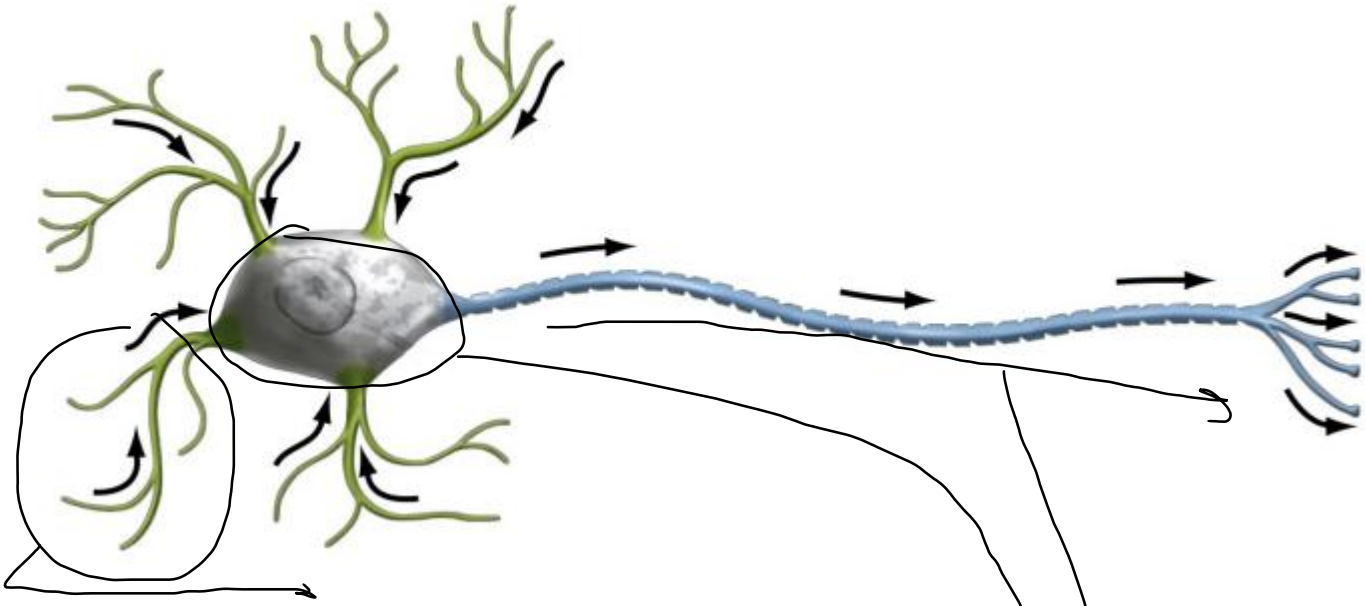
Redes Neuronales

La inspiración biológica en redes neuronales: fisiología neuronal básica, redes de neuronas biológicas y escalas de organización estructural del cerebro.

Neurona biológica



Dendritas: conforman un árbol dendrítico y permite que la neurona reciba señales. Cada señal puede ser excitatoria (+) que ayuda a que la neurona se active, o Inhibitoria (-) que evita que la neurona se active y de distintas intensidades.

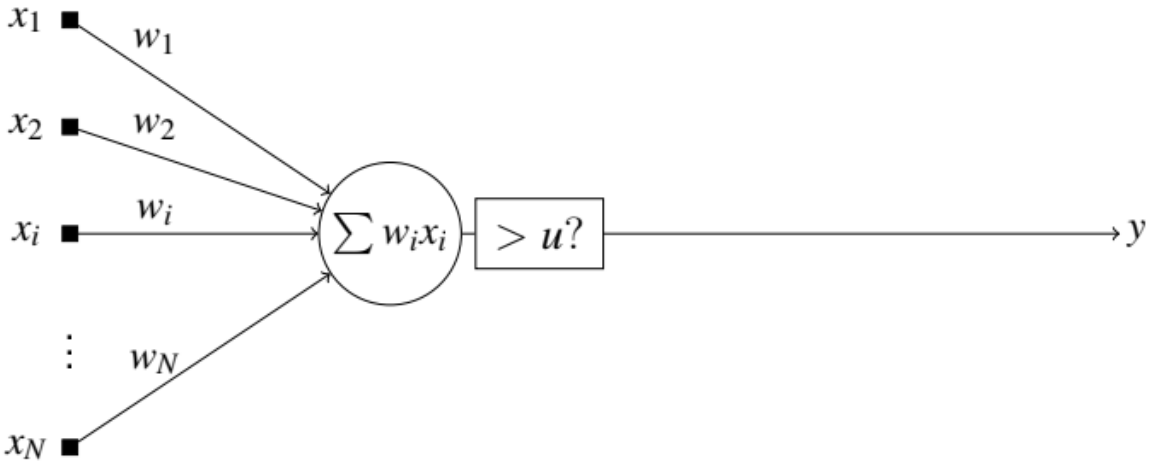
Núcleo (contenido en el soma – cuerpo neuronal): donde se procesa la información. La membrana del núcleo tiene una polaridad, y cuando la carga que se acumula en esta membrana supera un umbral, la neurona se despolariza y manda un único pulso por el axón. Si no supera el umbral, se encuentra en reposo, esto se conoce como comportamiento todo/nada.

Axón: por donde la neurona da su salida.

Sinapsis es el proceso de transmisión de los neurotransmisores.

- La corteza cerebral . . . 10^11 neuronas
- Redes de neuronas
    - **No linealidad:** la combinación de dos entradas no es una simple suma, y que un pequeño cambio en la entrada puede generar un cambio grande en la salida.
    - **Paralelismo:** hay muchas neuronas que, para resolver problemas complejos, se dividen en unidades de procesos.
    - **Capacidad de Aprendizaje**: aprenden a partir de datos o ejemplos
    - **Generalización**: capacidad de resolver problemas nunca antes vistos.
    - **Adaptabilidad:** capaces de adaptarse a nuevas situaciones
    - **Robustez:** capacidad de funcionar bien incluso cuando faltan datos o existe ruido.

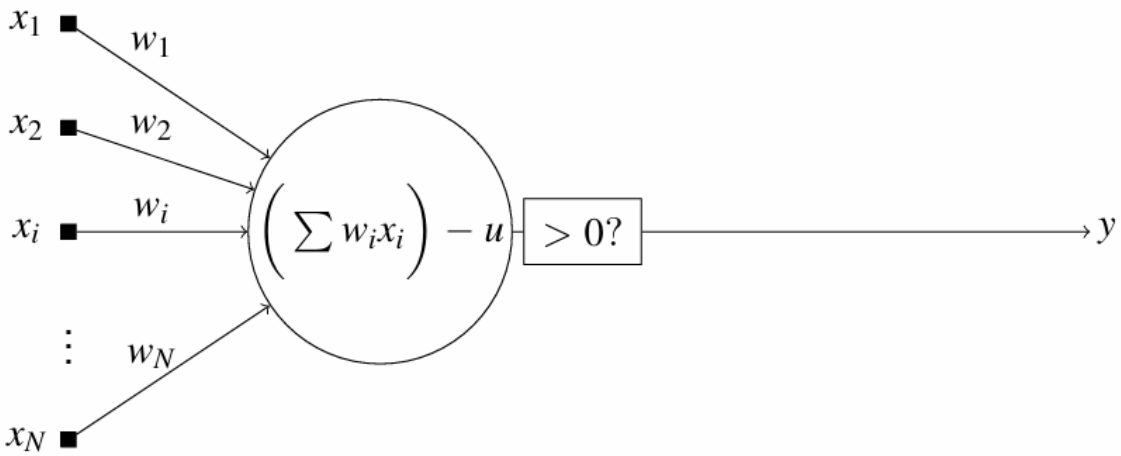
Modelos de neurona: la sinapsis, funciones de activación.



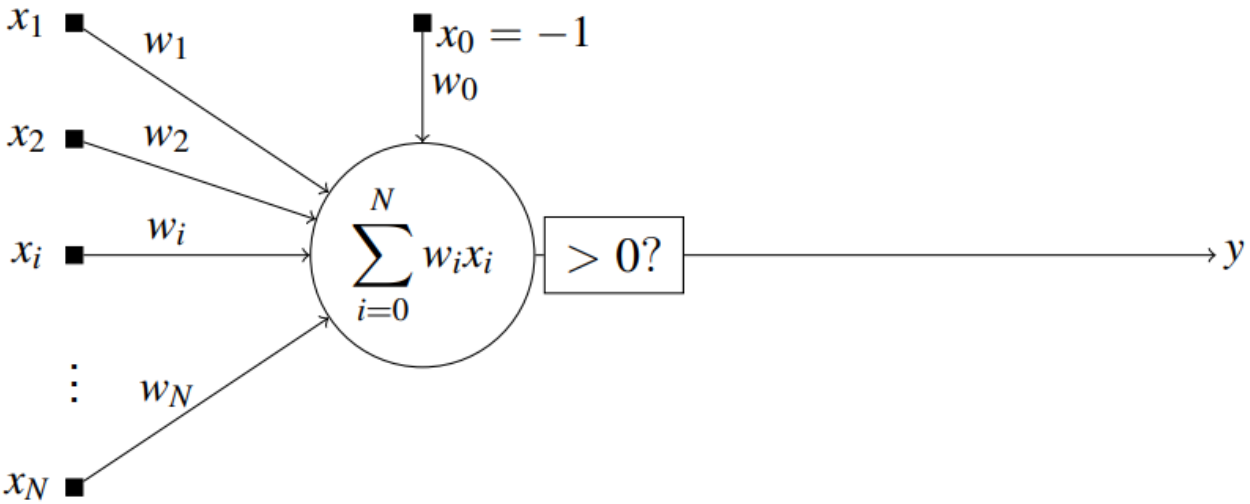
- Tiene x_N entradas y cada entrada x_i tiene un peso sináptico w_i, si la sinapsis es excitatoria será positivo y si es inhibitoria será negativo. De esta manera se ponderan la entradas de acuerdo a la importancia que tenga para la despolarización de la neurona.
- u es el umbral que debe superar la sumatoria para que la neurona de una salida y, la cual puede ser + y -1 o +1 y 0.
- Se supone que todos los estímulos llegan simultáneamente.

Se hace una suma ponderada y se verifica si supera el umbral. Si lo supera da la salida y.

Simplificación restando el umbral:



Simplificando más:



El umbral se simplifica siendo la multiplicación de x_0 y w_0. x_0 es siempre -1. Así consigo una sola sumatoria.

Perceptrón simple: hiperplanos para la separación de clases, entrenamiento y limitaciones.

Es el nombre que se le da al modelo simplificado de la neurona.

- Modelo matemático del perceptrón simple
  - Producto interno y umbral: $y = \phi(v - u) = \phi\left(\sum_{i=1}^{N} w_i x_i - u\right)$
  - Entrada extendida: $x_0 = -1, w_0 = u$

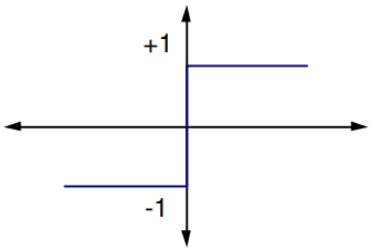
$$y = \phi\left(\sum_{i=0}^{N} w_i x_i\right) = \phi\left(\langle \mathbf{w}, \mathbf{x} \rangle\right)$$

v se llama activación lineal, u es el umbral. Nosotros usamos la entrada extendida, que se puede ver como un producto interno o escalar entre el vector de pesos sinápticos y el vector de entradas, esto da un escalar que indica el nivel de excitación de la neurona.

Funciones de activación $\phi(z)$

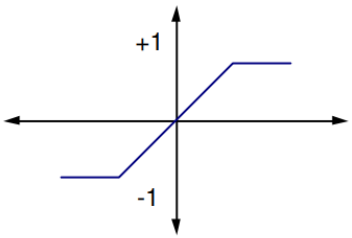
Sirven para verificar si se activa o no la neurona.

- Funciones de activación $\phi(z)$



$$\text{sgn}(z) = \begin{cases} -1 & \text{si } z < 0 \\ +1 & \text{si } z \geq 0 \end{cases}$$

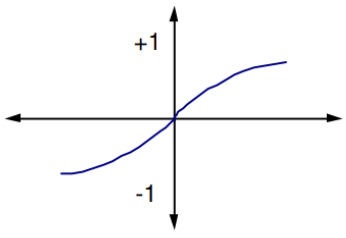
- Funciones de activación $\phi(z)$



$$\text{sln}(z) = \begin{cases} -1 & \text{si } z < -a \\ \alpha z & \text{si } -a < z < 0 \\ +1 & \text{si } z \geq a \end{cases}$$

Presenta una linealización de la zona de transición, se evita la discontinuidad, a cambia la pendiente de la función.

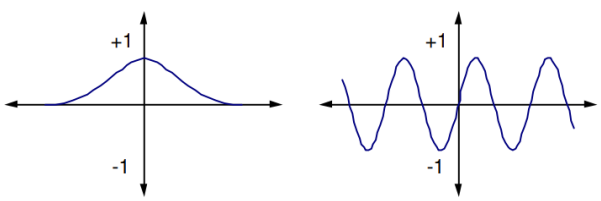
- Funciones de activación $\phi(z)$



$$\text{sig}(z) = \frac{1 - e^{-az}}{1 + e^{-az}}$$

Función sigmoidea. Es no lineal, no presenta discontinuidad. La constante a cambia la pendiente de la función, entre -1 y +1. Cuanto más grande el α, mas parecida al signo. Sigue pudiendo modelar el comportamiento todo o nada, pero ahora es derivable.

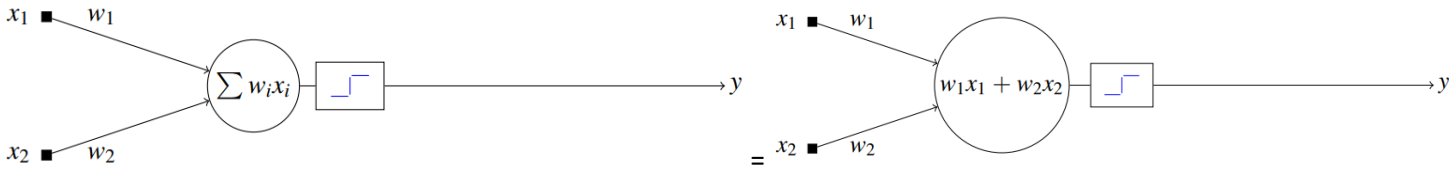
- Funciones de activación $\phi(z)$



Gaussiana y sinusoidal.

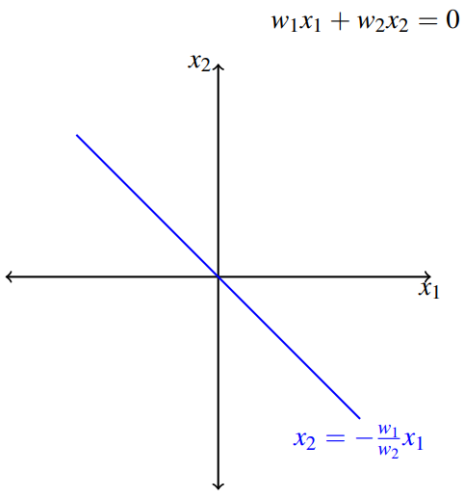
Hiperplanos para la separación de clases

Si por ejemplo tengo un perceptrón simple con dos entradas:

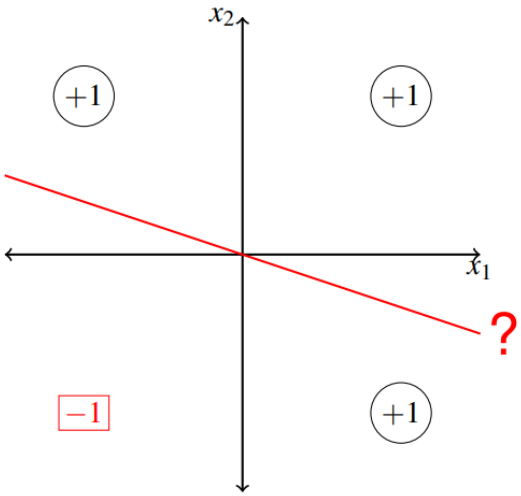


Utilizando la función de activación: $y = \text{sgn}(w_1x_1 + w_2x_2)$ Si la sumatoria da negativo, la salida será -1 y si la sumatoria da positiva, la salida será 1. Esta sumatoria permite identificar en que zona dará -1 y en que zona dará +1. Por encima de la recta dará +1, por debajo dará -1. Esto se sabe porque se supone que w_1 y w_2 tienen mismo valor. Por eso si x1 y x2 es 1 por ejemplo, toda esa zona tendrá como salida +1.

Y escribo una en función de la otra:

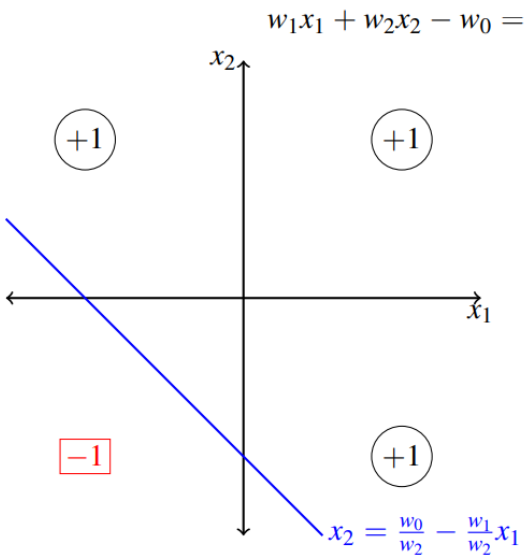


Caso del OR



No me permite plantear un OR:

Por lo que se incorpora lo que conocíamos como umbral w_0:



Limitación

No puede resolver el XOR

Entrenamiento: Algoritmos de aprendizaje

El objetivo es mostrarle ejemplos y que solo encuentre el valor de los pesos sinápticos.

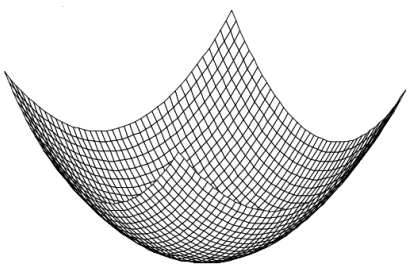
1. Inicialización al azar: $w(1) \in [-0.5, 0.5]$
2. Para cada ejemplo de entrenamiento $\boldsymbol{x}(n)|y_d(n)$:
   a. Se obtiene la salida: $y(n) = \phi\big(\boldsymbol{w}(n) \cdot \boldsymbol{x}(n)\big)$
   b. Se adaptan los pesos
3. Volver a 2 hasta satisfacer algún criterio de finalización.

Adaptación de los pesos por por método de gradiente

Mayor sustento matemático de como encontrar los pesos sinápticos, es prácticamente lo mismo que el el método corrección del error, pero con demostración matemática.

Concepto: Mover los pesos en la dirección en que se reduce el error, dirección que es opuesta a su gradiente con respecto a los pesos. Es decir, si el *gradiente apunta en la dirección en la que la superficie de error crece*, nos moveremos iterativamente en el sentido opuesto al gradiente.

Si tengo esta superficie de error, suponiendo que estoy graficando w1 y w2, para una combinación voy a estar en un punto en la superficie, el gradiente es un vector que desde ese punto apunta a donde crece el error.



Por eso modifico w en dirección opuesta al gradiente.

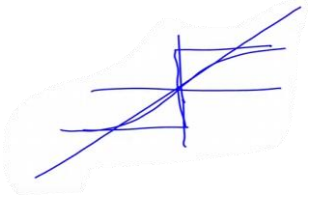
Ecuacion básica:

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_w \xi(\mathbf{w}(n))$$

μ es la velocidad con la que me voy a mover, cuando la superficie es plana, me puedo mover con mayor velocidad

Perceptrón simple: método del gradiente (caso lineal)

Caso lineal quiere decir que obviamos la función de activación, en vez de usar la signo o la sigmoidea es una recta.



Criterio del error instantáneo:

$$e^2(n) = [d(n) - y(n)]^2 = [d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]^2$$

y

d es la salida deseada

El gradiente del error con respecto a los pesos quiere decir la derivada del error respecto a los pesos: d(n) es una constante que esta sumando y x(n) es una constante que esta multiplicando, por lo que solo queda x

$$\nabla_w e^2(n) = 2\,[d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]\,(-\mathbf{x}(n))$$

error e(n)

$$\nabla_w e^2(n) = 2e(n)(-\mathbf{x}(n))$$

reemplazando en: $\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_w \xi(\mathbf{w}(n))$

$$\boxed{\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)\mathbf{x}(n)}$$

Si digo que $\eta = 2\mu$, obtengo lo mismo que con la demostración fácil.

Supongamos que: $w_0 = +1 \quad x_0 = -1 \quad d = -1$
$w_1 = +1 \quad x_1 = +1$
$w_2 = +1 \quad x_2 = +1$

$y = \text{sgn}(-1 + 1 + 1) \to 1$ no da -1 por lo que debo corregir los w

$\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)\mathbf{x}(n)$

$$\mathbf{w}(n+1) = \begin{bmatrix} +1 \\ +1 \\ +1 \end{bmatrix} + 2\mu(-1-1)\begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$$

suponiendo que $\mu = \frac{1}{2}$

$$\mathbf{w}(n+1) = \begin{bmatrix} +1 \\ +1 \\ +1 \end{bmatrix} + \begin{bmatrix} +2 \\ -2 \\ -2 \end{bmatrix} = \begin{bmatrix} +3 \\ -1 \\ -1 \end{bmatrix}$$

Qué sucede si volvemos a poner: $\mathbf{x}(n+1) = \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$?

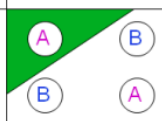
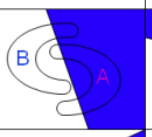
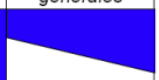
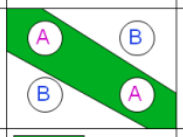
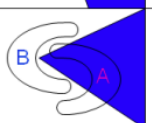
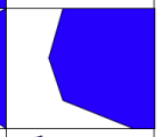
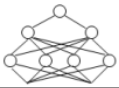
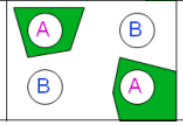
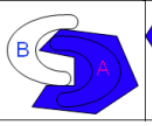
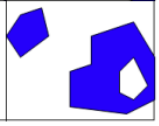
$y = \text{sgn}(-3 - 1 - 1) \to -1$

Ejemplo: ahora $e(n+1) = -1 - (-1) = 0$

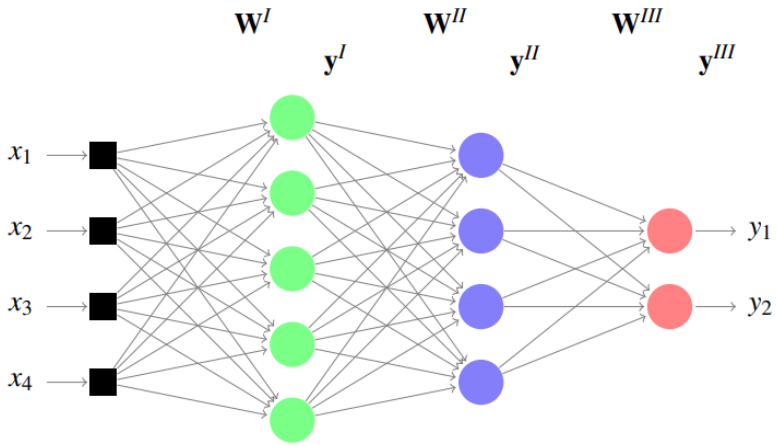
Perceptrón multicapa: formulación matemática del algoritmo de retropropagación, velocidad de aprendizaje y término de momento, inicialización y criterios de finalización, definición de la topología y los parámetros de entrenamiento.

El perceptrón multicapa puede resolver problemas más complejos que un perceptrón simple, como por ejemplo el problema del XOR.

Regiones de decisión

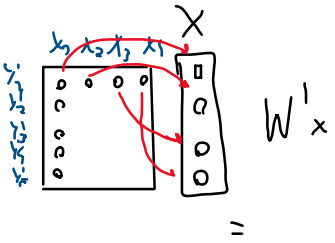
Estructura	Tipos de regiones de decisión	Problema XOR	Separación en clases	Formas regiones más generales
Una capa 	hemiplano limitado por hiperplano			
Dos capas 	Regiones convexas abiertas o cerradas			
Tres capas 	Arbitrarias (Complejidad limitada por N°. de Nodos)			

Notación



La matriz $\boldsymbol{W^I}$ tendrá todos los pesos entre las entradas y la primer capa, es decir va a tener 5*4=20 pesos
El vector de salidas de la primer capa $\boldsymbol{y^I}$ tendrá 5 salidas
La matriz $\boldsymbol{W^{II}}$ tendrá todos los pesos entre las salidas de $\boldsymbol{y^I}$ y la segunda capa, es decir va a tener 4*5=20 pesos
El vector de salidas de la segunda capa $\boldsymbol{y^{II}}$ tendrá 4 salidas
La matriz $\boldsymbol{W^{III}}$ tendrá todos los pesos entre las salidas de $\boldsymbol{y^{II}}$ y la tercer capa, es decir va a tener 2*4=8 pesos
El vector de salidas de la tercer capa $\boldsymbol{y^{III}}$ tendrá 2 salidas, las salidas finales

1 Calculo de las salidas en cada capa



- Capa 1:

$$\mathbf{y^I} = \phi(\mathbf{W^I}\,\mathbf{x}) = \phi(\mathbf{v^I})$$
$$\mathbf{v^I} = \sum_i^N w_{ji}^I x_i$$

- Capa 2:

$$\mathbf{y^{II}} = \phi(\mathbf{W^{II}}\,\mathbf{y^I}) = \phi(\mathbf{v^{II}})$$

- Capa 3:

$$\mathbf{y^{III}} = \phi(\mathbf{W^{III}}\,\mathbf{y^{II}}) = \phi(\mathbf{v^{III}})$$

La función de activación que se usa es $\phi(z) = \frac{2}{1+e^{-bz}} - 1$, que es simétrica ± 1.
Se usa esa función de activación ya que para el método del gradiente necesito poder derivar

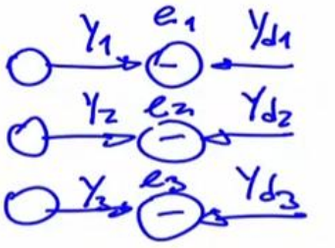
Filas de **W**: neuronas de capa siguiente, columnas, neuronas de capa anterior.

2 Propagación hacia atrás

Criterio de error

Para cada neurona habrá un error al instante o patrón n. Para el error total, la función de error que usaremos es la Suma del error cuadrático instantáneo:

$$\xi(n) = \frac{1}{2}\sum_{k=1}^{M} e_k^2(n)$$



Se genera un vector de errores comparando el valor deseado con el valor obtenido. El error es para la iteracion n

Se eleva al cuadrado ya que si el error de una neurona es -1 y la de otra es +1, el error seria 0 lo cual no es correcto.
El ½ esta por conveniencia ya que mas adelante se derivará bajando el 2.

Aplicación del gradiente (caso general)

Como en el caso de un perceptrón simple, se modificarán los pesos en el sentido opuesto al que hacen crecer al gradiente error. Nuevamente, μ es la velocidad de aprendizaje, que tan rápido se ajustan los pesos. Si modifico mucho los pesos, pierde o se olvida lo que aprendió en los ejemplos anteriores. Si la velocidad es baja va a tardar muchísimo.

Lo que debo sumar a los pesos será:

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

Con regla de la cadena,

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

La función de error depende de la salida de la neurona j,
a su vez esa salida depende de la salida lineal y la salida lineal depende directamente de los pesos.

$v_j(n) = \sum_{i=0}^{N} w_{ji}(n) y_i(n)$, entonces:

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n)$$

Entonces,

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} y_i(n)$$

OJO y_i es la entrada, es decir la salida de la capa anterior y y_j es la salida.

El gradiente de error local instantáneo es:

$$\delta_j(n) = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}$$

Local porque es para esa neurona e instantáneo porque es calculado en el instante n.

Para la salida con función de activación sigmoidea, $y_j(n) = \frac{2}{1+e^{-v_j(n)}} - 1$

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \frac{1}{2}\left(1 - y_j^2(n)\right)$$

Entonces el gradiente queda:

$$\delta_j(n) = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{1}{2}\left(1 - y_j^2(n)\right)$$

Entonces ahora lo defino de la siguiente manera:

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \delta_j(n) y_i(n)$$

$$\Delta w_{ji}(n) = -\mu \delta_j(n) y_i(n)$$

Retropropagación en capa III (salida)

$$\Delta w_{ji}^{III}(n) = -\mu \delta_j^{III}(n) y_i^{II}(n)$$

$$\delta_j^{III}(n) = \frac{\partial \xi(n)}{\partial y_j^{III}(n)} \frac{1}{2}\left(1 - y_j^{III\,2}(n)\right) = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2}\left(1 - y_j^{III\,2}(n)\right)$$

El criterio de error cuadrático medio depende del error cuadrático de la neurona de salida j, a su vez, el error cuadrático de esa neurona j depende de la salida que haya dado esa neurona.

$$\delta_j^{III}(n) = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2}\left(1 - y_j^{III\,2}(n)\right)$$

Como $\xi(n) = \frac{1}{2}\sum_{j=1}^{M} e_j^2(n)$, $\quad \frac{\partial \xi(n)}{\partial e_j(n)} = e_j(n)$

$$\delta_j^{III}(n) = e_j(n) \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2}\left(1 - y_j^{III\,2}(n)\right)$$

$$e_j(n) = d_j^{III}(n) - y_j^{III}(n), \qquad \frac{\partial e_j(n)}{\partial y_j^{III}(n)} = -1$$

$$\delta_j^{III}(n) = -e_j(n) \frac{1}{2}\left(1 - y_j^{III\,2}(n)\right) = -\frac{1}{2} e_j(n) \left(1 - y_j^{III\,2}(n)\right)$$

$$\Delta w_{ji}^{III}(n) = \mu \frac{1}{2} e_j(n) \left(1 - y_j^{III\,2}(n)\right) y_i^{II}(n)$$
$$\Delta w_{ji}^{III}(n) = \eta e_j(n) \left(1 - y_j^{III\,2}(n)\right) y_i^{II}(n)$$
$$\Delta w_{ji}^{III}(n) = \eta \left(d_j^{III}(n) - y_j^{III}(n)\right) \left(1 - y_j^{III\,2}(n)\right) y_i^{II}(n)$$

η es la velocidad de aprendizaje

Retropropagacion en capas ocultas: capa II y capa "p"

$$\Delta w_{ji}^{II}(n) = -\mu \delta_j^{II}(n) y_i^{I}(n)$$

$$\delta_j^{II}(n) = \frac{\partial \xi(n)}{\partial y_j^{II}(n)} \frac{1}{2}\left(1 - {y_j^{II}}^2(n)\right)$$

El error ξ es respecto a la capa de salida, pero lo queremos derivar respecto a la capa oculta

$$\frac{\partial \xi(n)}{\partial y_j^{II}(n)} = \sum_k \frac{\partial \xi(n)}{\partial e_k(n)} \frac{\partial e_k(n)}{\partial y_k^{III}(n)} \frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)} \frac{\partial v_k^{III}(n)}{\partial y_j^{II}(n)}$$

(hay suma sobre k porque ξ depende de todas las salidas de la capa III)

(el error de la capa de salida respecto a la salida de la capa de salida, la salida respecto a la salida lineal, la salida lineal respecto a la entrada, es decir la salida de la capa oculta). $\boldsymbol{k}$: neuronas de la capa III (salida). $\boldsymbol{j}$: neuronas de la capa II (oculta)

Como $v_k^{III}(n) = \sum_j w_{kj}^{III} y_j^{II}$

$$\frac{\partial v_k^{III}(n)}{\partial y_j^{II}(n)} = w_{kj}^{III}$$

Como ya calculamos antes, para la salida con función de activación sigmoidea, $y_k(n) = \frac{2}{1+e^{-v_j(n)}} - 1$

$$\frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)} = \frac{1}{2}\left(1 - {y_k^{III}}^2(n)\right)$$

Como $e_k(n) = d_k^{III}(n) - y_k^{III}(n)$,

$$\frac{\partial e_k(n)}{\partial y_k^{III}(n)} = -1$$

Como $\xi(n) = \frac{1}{2}\sum_k e_k^2(n)$,

$$\frac{\partial \xi(n)}{\partial e_k(n)} = e_k(n)$$

Entonces nos queda:

$$\frac{\partial \xi(n)}{\partial y_j^{II}(n)} = \sum_k e_k(n)(-1)\frac{1}{2}\left(1 - {y_k^{III}}^2(n)\right) w_{kj}^{III} = \sum_k -\frac{1}{2} e_k(n)\left(1 - {y_k^{III}}^2(n)\right) w_{kj}^{III}$$

Y

$$\delta_j^{II}(n) = \sum_k -\frac{1}{2} e_k(n)\left(1 - {y_k^{III}}^2(n)\right) w_{kj}^{III} \frac{1}{2}\left(1 - {y_j^{II}}^2(n)\right)$$

De la capa 3 tenemos el gradiente de error local en la capa 3:

$$\delta_k^{III}(n) = -\frac{1}{2} e_k(n)\left(1 - {y_k^{III}}^2(n)\right)$$

Entonces la formula del gradiente del error local instantáneo de la capa 2 nos queda:

$$\delta_j^{II}(n) = \sum_k \delta_k^{III}(n)\, w_{kj}^{III}\, \frac{1}{2}\left(1 - {y_j^{II}}^2(n)\right) = \left(\delta^{III} \cdot w_j^{III}\right)\frac{1}{2}\left(1 - {y_j^{II}}^2(n)\right)$$

Estamos haciendo pasar el gradiente de error local de la capa 3 de atrás para adelante, de la salida hacia la capa oculta, aquí se ve el concepto de retropropagacion. Se está pasando el gradiente del error de la capa 3 por los pesos de la capa 3 para obtener el gradiente de la capa 2, a diferencia de pasar las entradas por los pesos cuando hacíamos propagación hacia adelante.

Esto se puede generalizar para la capa "p":

$$\Delta w_{ji}^{(p)}(n) = -\mu \delta_j^{(p)}(n) y_i^{(p-1)}(n)$$

Con

$$\delta_j^{(p)}(n) = \left(\delta^{(p+1)} \cdot w_j^{(p+1)}\right)\frac{1}{2}\left(1 - {y_j^{(p)}}^2(n)\right)$$

p es la capa en la cual estamos parados actualmente, la capa para la cual queremos calcular el ajuste de pesos.
p-1 sera la entrada a esa capa
η velocidad de aprendizaje
$\left(\delta^{(p+1)} \cdot w_j^{(p+1)}\right)$ es el producto interno que es la retropropagacion del gradiente de error local de la capa siguiente p+1
$\left(1 - {y_j^{(p)}}^2(n)\right)$ es la derivada de la función de activación de la capa p
$y_i^{(p-1)}(n)$ es la entrada a la capa p, la salida de la capa p-1

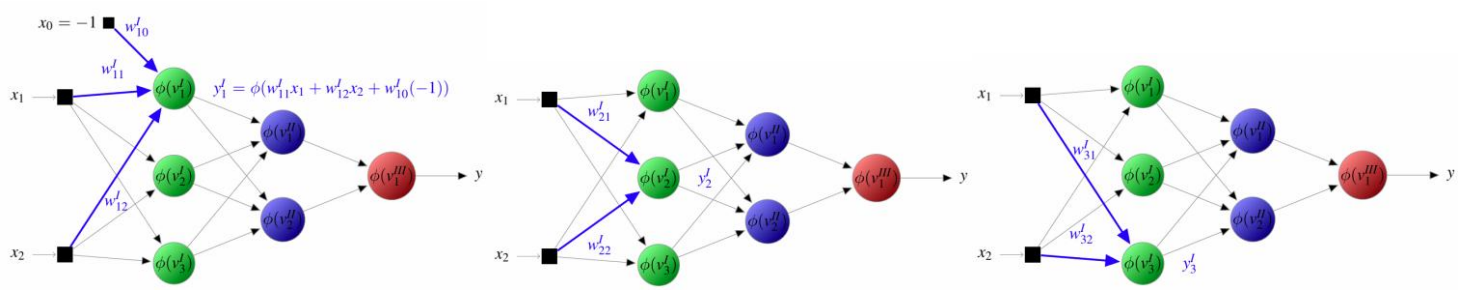
Resumen del algoritmo de retropropagacion

1. Inicialización aleatoria
2. Propagación hacia adelante
3. Propagación hacia atras
4. Adaptación de los pesos
5. Iteración: vuelve a 2 hasta convergencia o finalización

Todos estos pasos se van a repetir para cada patron.

Propagación hacia adelante

Capa 1:



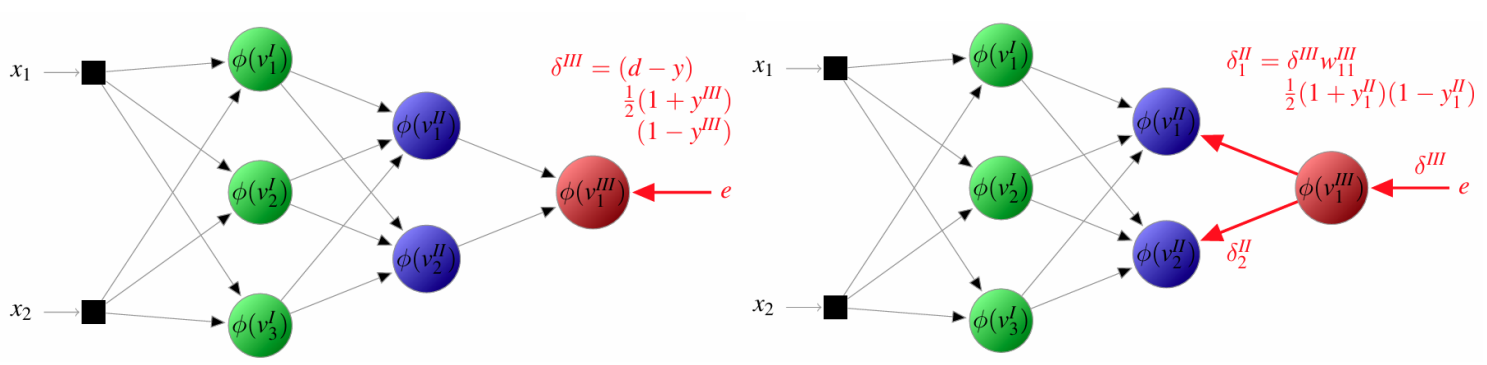
Capa 2 y capa 3:

Recordar que también se suma el sesgo que no está graficado

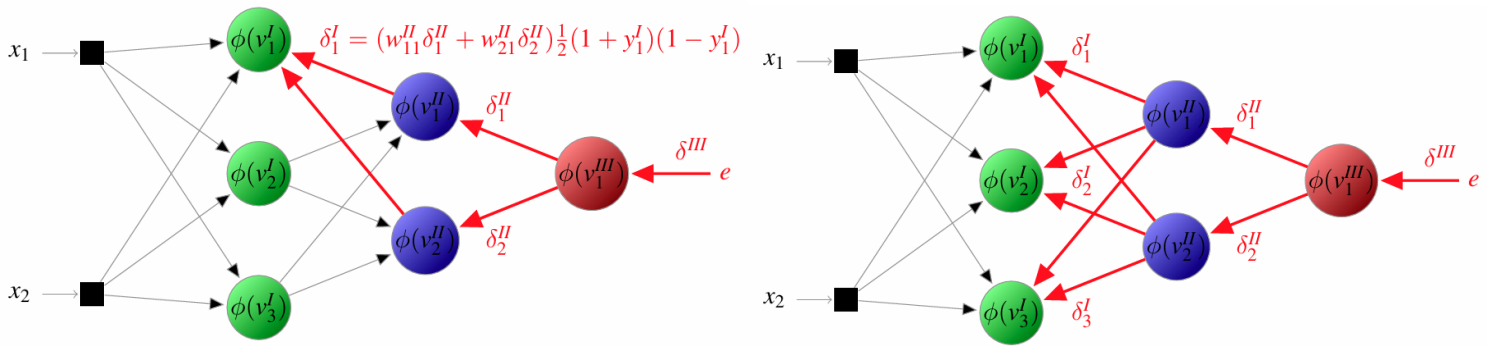


Propagación hacia atrás

Capa 3 y capa 2:



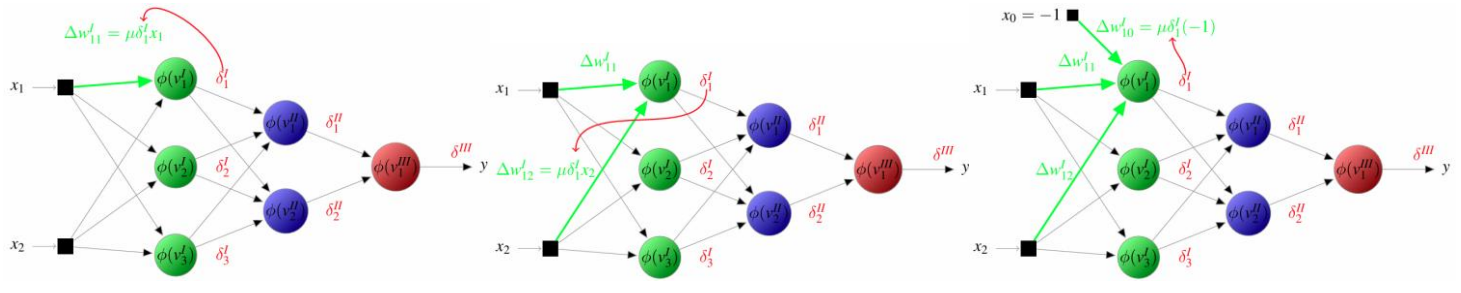
Capa 1:



Ajuste de pesos

El delta que calculo, luego se debe sumar al peso actual

Capa 1: Neurona 1:



Capa 1: Neuronas 2 y 3:

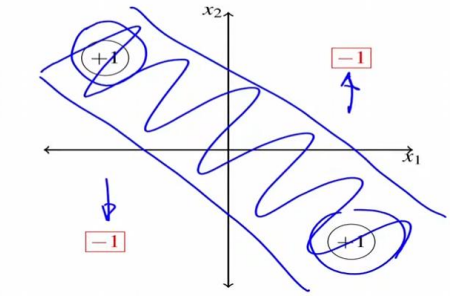


Capas 2 y 3:



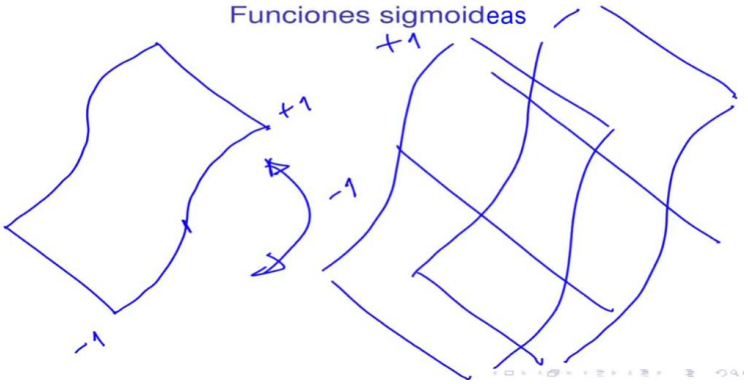
Redes neuronales con funciones de base radial: arquitectura, fronteras de decisión, algoritmos de entrenamiento.

Con la función signo o sigmoidea, se logra la siguiente separación mediante fronteras de decisión:



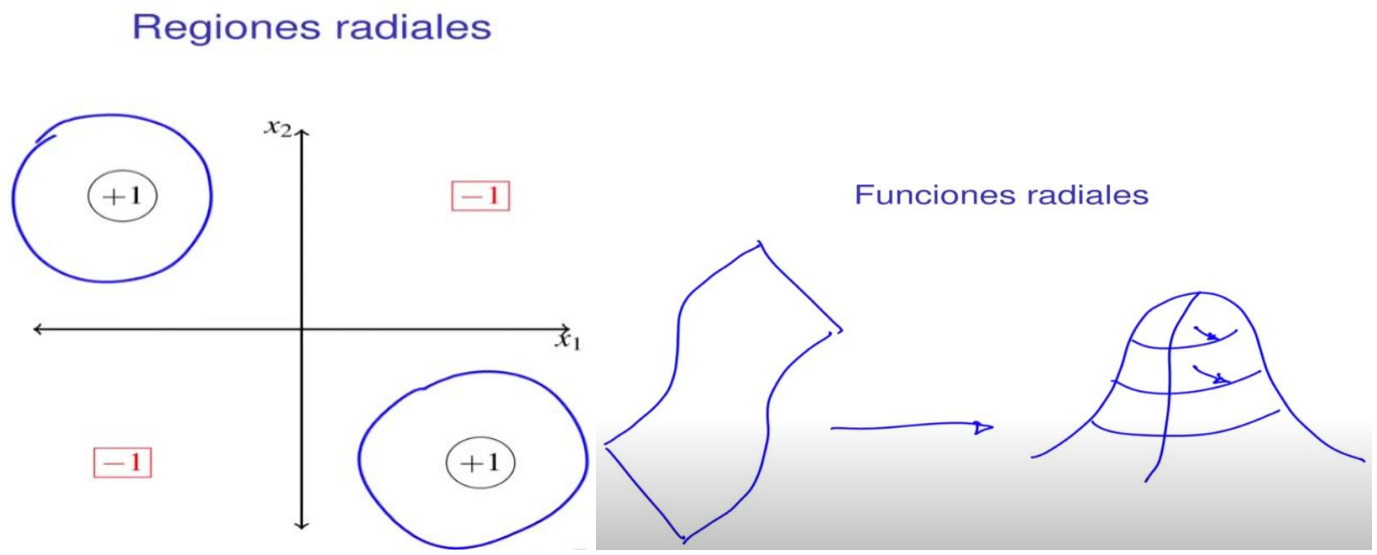
Estas fronteras de decisión son donde la función de activación pasa de -1 a +1.

En 3d las sigmoideas se ven asi:



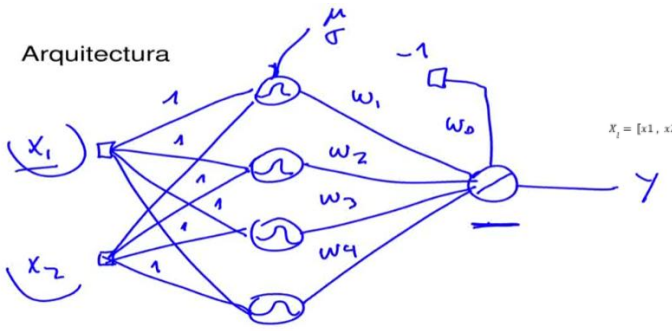
En cada recta de la región separada con dos perceptrones habría una función sigmoidea (imagen de la derecha).

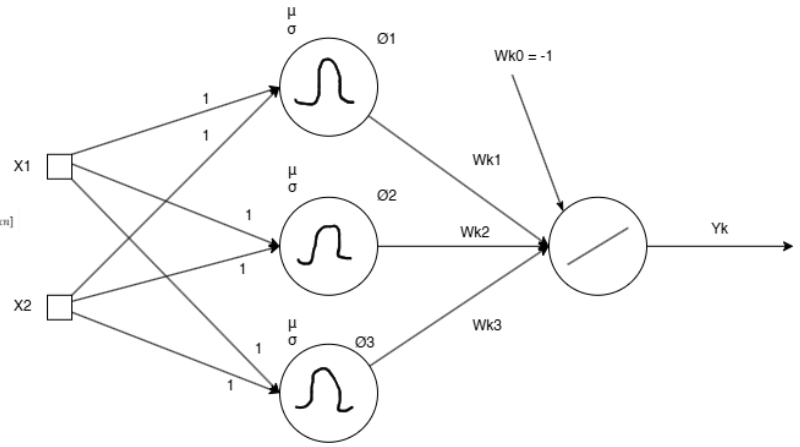
Si en vez de usar las funciones sigmoidea o signo usamos una **radial** que encierre la zona que nos interesa con un círculo podríamos clasificar todo lo que este dentro del círculo como una clase y todo lo que este afuera como otra.



Entonces ahora se arman redes neuronales, pero en vez de tener función de activación sigmoidea, tiene radial.

RBF-NN: Arquitectura





En este ejemplo, las gausseanas están en R^2 porque reciben x1 y x2.

Las neuronas intermedias son gausseanas y la de salida es lineal, esto porque simplemente es para aproximación de funciones, los pesos para las capa de entrada son 1. Las neuronas de base radial no requieren bias.

La relación entre las entradas y las gausseanas se ajustan con la media y la desviación estándar. Cada valor es un vector de 2 valores porque hay dos entradas.

RBF-NN: Modelo matemático

Modelo matemático

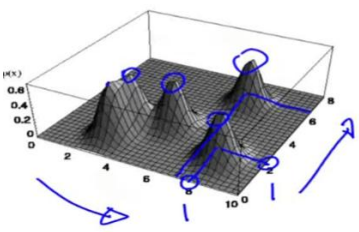
donde:

$$y_k(\mathbf{x}_\ell) = \sum_{j=1}^{M} w_{kj}\phi_j(\mathbf{x}_\ell)$$

$$\phi_j(\mathbf{x}_\ell) = e^{-\frac{\|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2}{2\sigma_j^2}}$$

μ_j es el vector de medias de la neurona j, Equivale al centroide.

σ_j es la desviacion estandar de la neurona j. Equivale al “ancho” de la gaussiana. Aca esta simplificado porque es un solo valor, con esto es suficiente porque pesos de la capa de salida hacen que le dé más o menos importancia a esa gaussiana, por lo que me evita calcular las desviaciones para que sea más o menos ancha. Si quiero cambiar la forma también, debo usar gaussianas N-dimensionales.(no lo veo)



$\mu_1 = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$

$\mu_2 = \begin{bmatrix} 7 \\ 7 \end{bmatrix}$

La salida y_k va a ser de un perceptron simple con salida lineal.

La función depende de la distancia entre la entrada y el centro.

Cada neurona j de base radial va a tener su propio vector de medias.

RBF-NN: Entrenamiento

Se adaptan las RBF y w_{kj} por separado

- **Método 1:**
  - **Paso 1:** Adaptación no supervisada de las RBF
    Primero se entrena sin decirle cuales son las salidas correctas
    - Utilizando el método k-medias
    - Utilizando mapas autoorganizativos
    - Otros
  - **Paso 2:** Adaptación supervisada de los w_{kj} (LMS)
    Comparando con las salidas correctas, se adaptan SOLO los pesos.
- **Método 2:**
  En este método además se hace una adaptación supervisada tanto de los pesos sinápticos como de las medias y desvíos.
  - **Paso 1:** Inicialización por el método 1
  - **Paso 2:** Adaptación supervisada de las RBF $\left(\frac{\partial \xi}{\partial \mu_{ji}}, \frac{\partial \xi}{\partial \sigma_j}\right)$

NO SUPERVISADO: quiere decir que no se utiliza la salida deseada para el entrenamiento.

1 Adaptación de las RBF

Utilizando k-medias

Método NO-supervisado! (uno de los más simples)

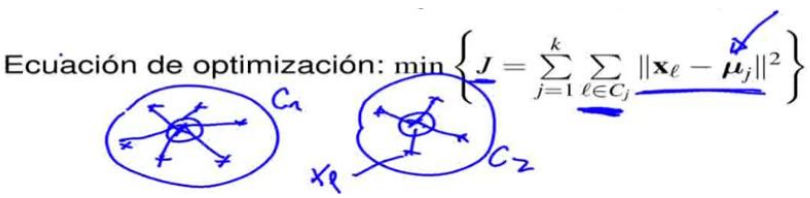
Objetivos:

- Encontrar k conjuntos C_j de forma que:
  - Cada conjunto C_j sea lo más diferente posible de los demás
  - Los patrones $\mathbf{x}_\ell$ dentro de cada C_j sean lo más parecidos posible entre ellos
- Encontrar el centroide $\boldsymbol{\mu}_j$ de cada conjunto C_j

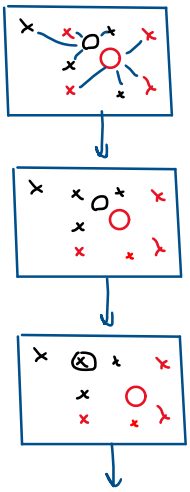
Ecuación de optimización: $\min \left\{ J = \sum_{j=1}^{k} \sum_{\ell \in C_j} \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 \right\}$

Los centroides van a ser luego, las medias de esas gaussianas. Son a su vez el promedio de todos los patrones de cada conjunto.

Se minimiza las distancias entre los centroides y los patrones:



Acumulación de las distancias entre todos los centroides y sus patrones. Minimizamos esto para ubicar el centroide bien en el medio.



Procedimiento opción 1: k-medias por lotes

1. Inicialización: se forman los k conjuntos $C_j(0)$ con patrones $\mathbf{x}_\ell$ elegidos al aleatoriamente.
2. Se calculan los centroides:

$$\boldsymbol{\mu}_j(n) = \frac{1}{|C_j(n)|} \sum_{\ell \in C_j(n)} \mathbf{x}_\ell$$

3. Se reasignan los $\mathbf{x}_\ell$ al C_j más cercano:

$$\ell \in C_j(n) \Leftrightarrow \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 < \|\mathbf{x}_\ell - \boldsymbol{\mu}_i\|^2 \quad \forall i \neq j$$

4. Volver a 2 hasta que no se realicen reasignaciones.

3- El patrón se asigna a Cj si y solo si la distancia entre el dato y el centroide j es menor que la distancia entre ese mismo dato y otro centroide i

k es la cantidad de neuronas que tenemos en la capa de base radial.

El paso 2 es un promedio simplemente, toma los patrones del conjunto j, los suma y los divide por la cantidad de elementos que tiene el conjunto j. X es un vector.

Este método hace el proceso por lotes, es decir, hacer con todos los patrones, ajustar los centroides, hacer con todos los patrones, ajustar los centroides.

Procedimiento opción 2: k-medias online

Se realiza patrón a patrón:

Se calcula el valor del centroide en la iteración siguiente en base a el valor actual, el patrón que acaba de entrar y una velocidad de aprendizaje.

1. Inicialización: se eligen k patrones aleatoriamente y se usan como centroides iniciales $\boldsymbol{\mu}_j(0) = \mathbf{x}'_\ell$.
2. Selección:

$$j* = \arg\min_j \left\{ \|\mathbf{x}_\ell - \boldsymbol{\mu}_j(n)\| \right\}$$

3. Adaptación:

$$\boldsymbol{\mu}_{j*}(n+1) = \boldsymbol{\mu}_{j*}(n) + \eta(\mathbf{x}_\ell - \boldsymbol{\mu}_{j*}(n))$$

4. Volver a 2 hasta no encontrar mejoras significativas en J.

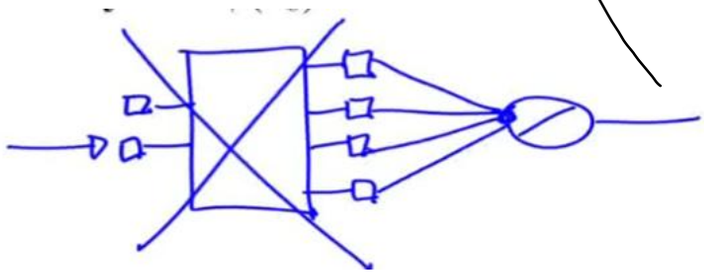
2: entra un patrón y se elige el centroide ganador con la menor distancia al patrón que acaba de entrar.

3: se adapta el peso de ese centroide ganador hacia el patrón. $x_l - u_{j*}(n)$ es un vector que apunta desde el centroide al patrón.

2 Adaptación de los pesos

- Al entrenar los pesos, las RBF quedan fijas
- Al estar las RBF fijas se pueden obtener las salidas intermedias para cada patrón de entrada: $\phi(\mathbf{x}_\ell)$
- Con esas salidas intermedias se puede entrenar cada perceptrón simple:

$$\mathbf{y} = \mathbf{W}\phi(\mathbf{x}_\ell)$$



esta simplificación nos permite olvidarnos de las primeras entradas y tomar estas segundas como las entradas.

Método de entrenamiento LMS: Gradiente descendiente sobre el error cuadrático instantáneo

$$w_{kj}(n+1) = w_{kj}(n) - \eta \frac{\partial \xi(n)}{\partial w_{kj}(n)}$$

$$\frac{\partial \xi(n)}{\partial w_{kj}(n)} = \frac{\partial \xi(n)}{\partial e_k(n)} \frac{\partial e_k(n)}{\partial y_k(n)} \frac{\partial y_k(n)}{\partial w_{kj}(n)}$$

$y_k(n) = \sum_j w_{kj}(n)\phi_j(n)$, entonces

$$\frac{\partial y_k(n)}{\partial w_{kj}(n)} = \phi_j(n)$$

$e_k(n) = d_k(n) - y_k(n)$, entonces

$$\frac{\partial e_k(n)}{\partial y_k(n)} = -1$$

$\xi(n) = \frac{1}{2}\sum_k e_k^2(n)$, entonces

$$\frac{\partial \xi(n)}{\partial e_k(n)} = e_k(n)$$

$$w_{kj}(n+1) = w_{kj}(n) + \eta e_k(n)\phi_j(n)$$

Comparación RBF-NN vs. MLP

RBF-NN	MLP
1 capa oculta	p capas ocultas
distancia a prototipos gaussianos	hiperplanos sigmoideos
representaciones locales sumadas	representaciones distribuidas combinadas
convergencia más simple (linealidad)	
entrenamiento más rápido	
arquitectura más simple	
combinación de diferentes paradigmas de aprendizaje	

1 capa oculta porque puede partir el espacio en muchas secciones radiales.

Representaciones locales sumadas quiere decir que agrupa de a zonas.

Redes neuronales dinámicas: redes de Hopfield, retropropagación a través del tiempo, redes neuronales con retardos en el tiempo.

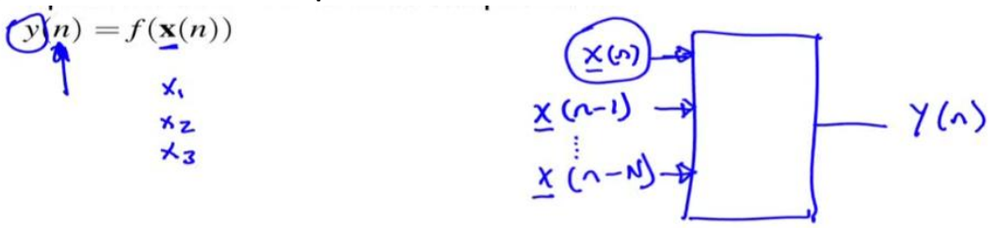
Son redes que tienen un comportamiento que evoluciona con el tiempo.

Caso general:

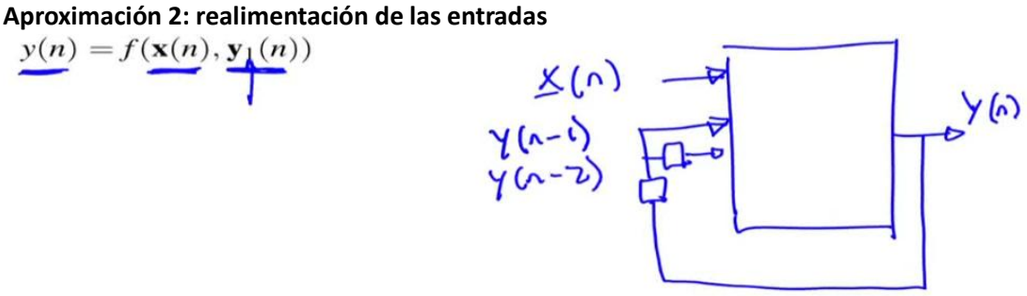
$$y(n) = f(\mathbf{x}(n), \mathbf{z}_1(n), \mathbf{y}_1(n))$$

Tiene realimentación de las entradas en instantes anteriores, de los estados internos en instantes anteriores y de las salidas en instantes anteriores:

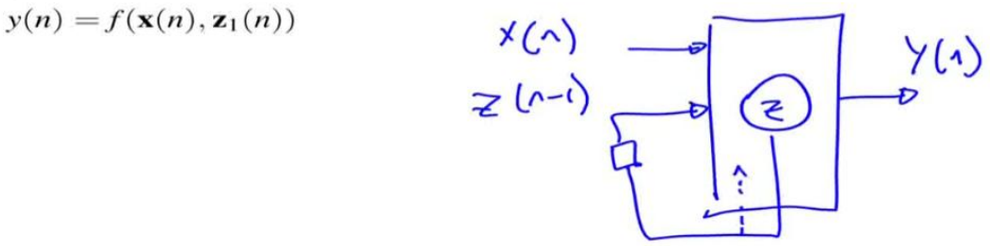
- **Aproximación 1: entradas desplazadas**



  n es el tiempo discreto (iteración)
  Con esta aproximación suponemos que en una red neuronal hay entradas que no solo contemplan el momento actual x(n) sino lo que valieron esas mismas entradas en instantes anteriores x(n-N). Estas entradas anteriores también influyen en la salida. las x son vectores
  La red sigue siendo **estática**, ya que no hay una dinámica interna, solo le paso entradas en instantes anteriores.
- **Aproximación 2: realimentación de las entradas**



  Además de tener las entradas, tengo las salidas en instantes anteriores como entradas.
- **Aproximacion 3: realimentación de estados internos**



  Se realimenta un estado interno de un instante interior. Hasta aca sigue siendo TDNN, estatica.
  Si se realimenta a alguna capa interna de la red neuronal (con retardo), es decir sin pasar como entrada al principio (flecha discontinua), En este caso la red neuronal en si misma si se comporta de forma **dinámica**:
  La salida actual va a depender no solo de la entrada actual sino de los estados anteriores de la red es decir de las salidas internas anteriores, que indirectamente dependen de las entradas anteriores de la red. La red empezó a considerar cierta memoria, responde a la historia de las entradas.

Clasificación

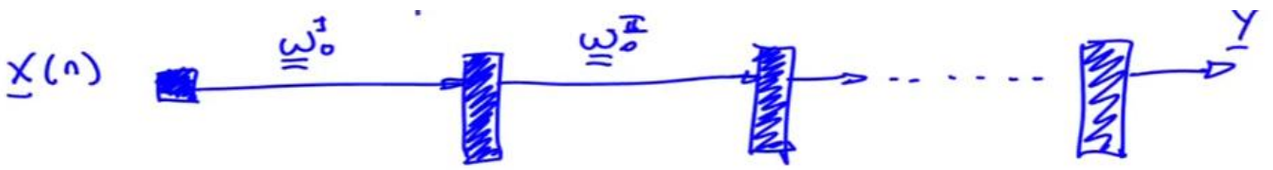
Redes neuronales dinámicas (DNN)

- **Redes con retardos en el tiempo (TDNN).**
  Generalización de las redes que planteamos anteriormente (los bloques). Tienen un comportamiento dinámico, pero se pueden seguir entrenando con los métodos que ya conocemos. Siguen siendo feed forward.
- **Redes recurrentes (RNN)**
  Tienen realimentación hacia atrás. La sinapsis no va siempre hacia adelante, sino que hay realimentación de una neurona a sí misma, a otra de la misma capa o a una neurona de la capa anterior.
  - **Redes totalmente recurrentes**
    - Redes de Hopfield (memorias asociativas) ← solo vemos esta
    - Redes de Boltzman (supervisadas)
    - Teoría de la resonancia adaptativa (ART)
  - **Redes parcialmente recurrentes (PRNN)**
    - Método de retropropagacion a través del tiempo (BPTT) ← solo vemos esto
    - Redes de Elman ← solo arquitectura
    - Redes de Jordan ← solo arquitectura

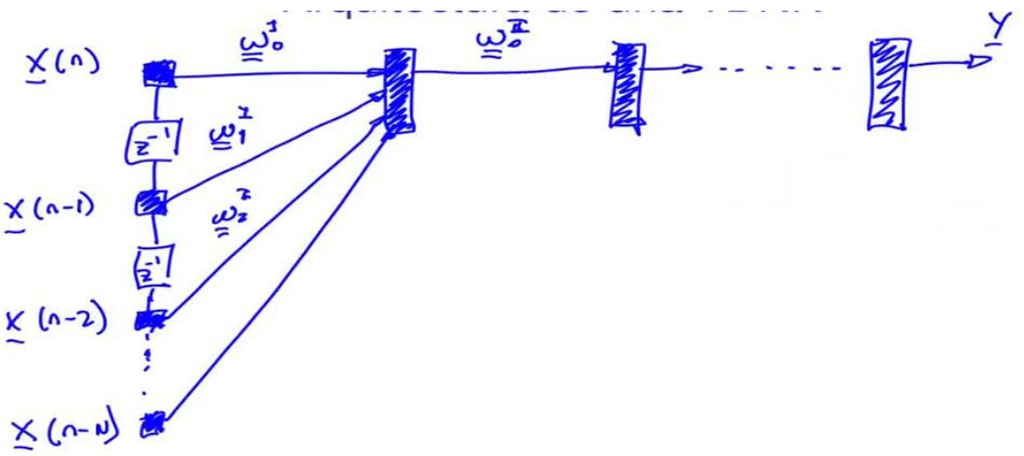
Redes neuronales con retardos en el tiempo (TDNN)

Arquitectura de una TDNN

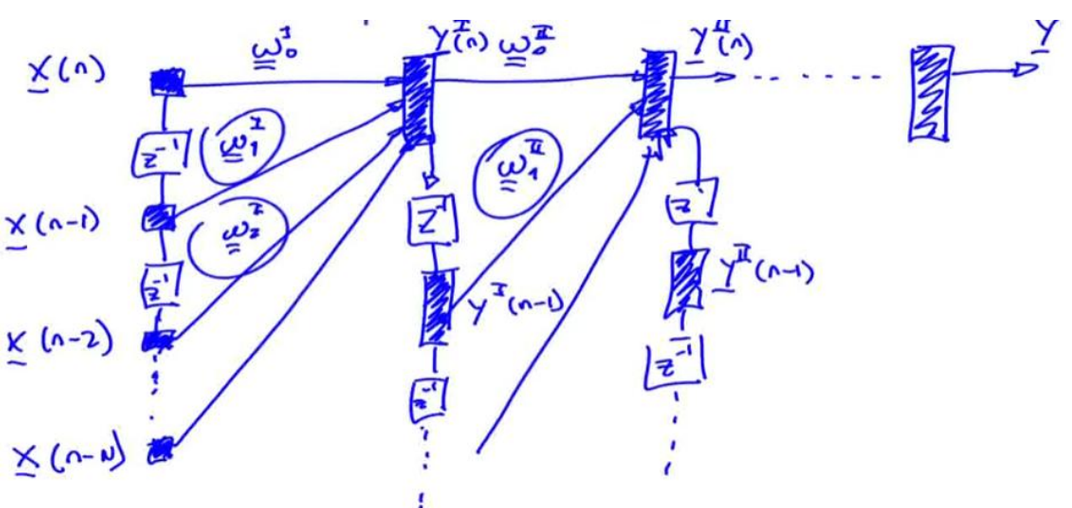
Arranca como un perceptrón multicapa, el _0 de los pesos quiere decir con retardo 0.



Se agrega un retardo temporal Z^{-1} y las mismas entradas, pero en un instante anterior (n-1). Esto se agrega como entrada en la capa 1 con una matriz de pesos, pero con retardo 1. Esto se puede expandir hacia la cantidad de retardos N que queramos.



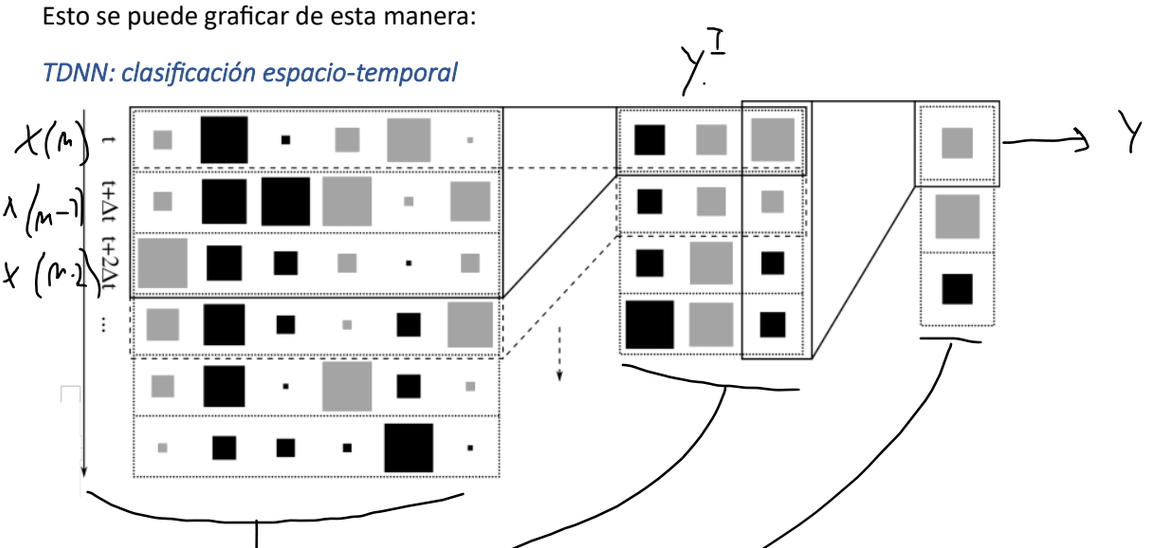
De este modo agregamos la historia de las entradas, pero también podemos hacer lo mismo con las salidas de cada una de las capas. Obteniendo asi las salidas de cada capa instantes de tiempos anteriores.



Puede ser entrenada con retropropagacion como el perceptrón multicapa, solo que cambiando el formato de los pesos para tener en cuenta que paso en el tiempo.

Esto se puede graficar de esta manera:

TDNN: clasificación espacio-temporal



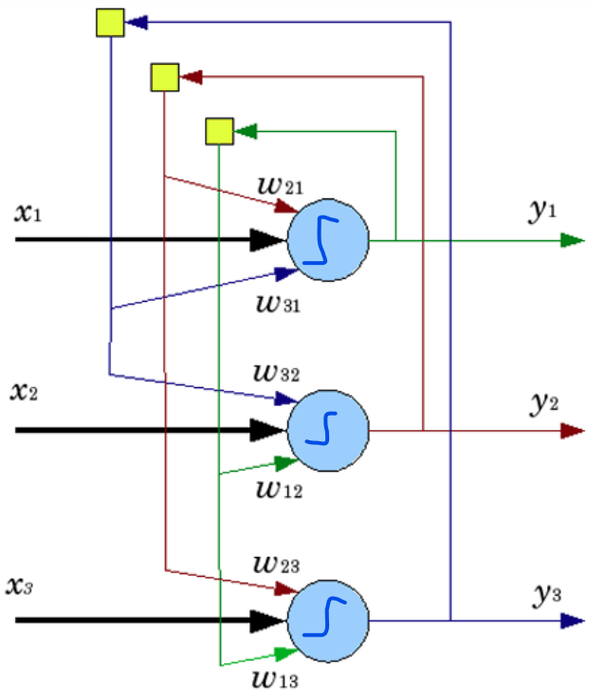
Corto plazo
Largo plazo
Plazo mas largo

Redes recurrentes (RNN)

Totalmente recurrentes

Redes de Hopfield

Arquitectura



En estas redes siempre hay la misma cantidad de entradas, neuronas y salidas y hay una sola capa.

Se realimenta la salida de una neurona con un retardo temporal (el cuadradito) al resto de las neuronas. No hay realimentaciones a la misma neurona.

Los pesos sinápticos están vinculados a las salidas de las neuronas y las realimentan como entradas.

Modelo matemático

$$y_j(n) = sgn(x) = sgn\left(\sum_{i=1}^{N} w_{ji} y_i(n-1) - \theta_j\right) \cdots \begin{cases} x > 0 & +1 \\ x = 0 & y_j(n-1) \\ x < 0 & -1 \end{cases}$$

$$w_{ji} = w_{ij} \quad \forall\, i \neq j$$
$$w_{ii} = 0 \quad \forall i$$

Sumatoria de los pesos por la salida del instante anterior.

Sesgo.

Estas redes son binarias, es decir que dan +1 o -1 en la salida y solo pueden recibir +1 y -1 en las entradas.

x llamamos a la sumatoria en este caso. Si la sumatoria es 0, la salida permanece igual que en el instante anterior.

Los pesos sinápticos son simétricos: el peso de la salida de la neurona 1 que vincula con la neurona 2 es lo mismo que el peso que los vincula al revés. Y no hay pesos que vinculen a una neurona consigo misma (no hay realimentación en una neurona).

Generalidades

- **Cada neurona tiene un disparo probabilístico**
  Es decir que hay una variable aleatoria que se elige al azar a cuál le toca disparar en cada momento. Sirve para que no entre en bucles infinitos. Se elige al azar una neurona -> se actualiza la salida -> Se elige al azar… etc
- **Conexiones simétricas**
- **El entrenamiento es no-supervisado**
- **Puede utilizarse como memoria asociativa**
  Es decir que se puede utilizar no como memoria por dirección, donde se le pasa la dirección y te devuelve el contenido, sino que se utiliza de manera que le doy algo que quiero encontrar y me devuelve lo más parecido que tenga en memoria. Es decir, la puedo acceder por el contenido y no por una dirección. Por ejemplo le doy la foto de una persona y me da la cara en limpio.

Almacenamiento y recuperación (Entrenamiento y prueba)

Entrenamiento (almacenamiento)

Dado un conjunto de patrones (memorias fundamentales o datos limpios):

$$\mathrm{X}^* = \{\mathbf{x}_\mathrm{k}^* \in \mathbb{R}^\mathrm{N}\}$$

Tengo P patrones que quiero guardar en la red, cada patrón $\mathbf{x}_\mathrm{k}^*$ es un es un vector de N componentes (una memoria de N neuronas). Estos patrones son las memorias fundamentales que la red podrá recordar.

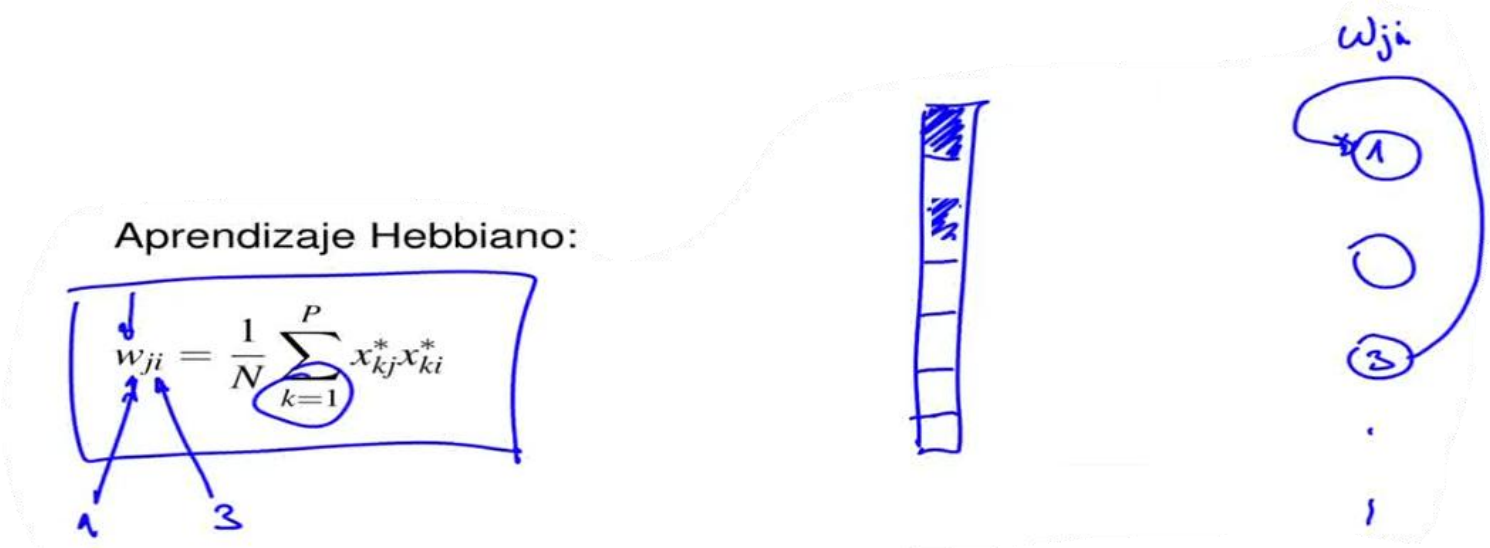
Para calcular los pesos sinápticos de la red se utiliza Aprendizaje Hebbiano:

$$w_{ji} = \frac{1}{N}\sum_{k=1}^{P} x_{kj}^* x_{ki}^*$$

Hace una especie de promedio de los patrones. El peso que vincula el la entrada j con la i. Promedia la i y la j para todos los patrones k.

Calcula la relación o sincronización entre dos neuronas, si no tienen sincronización, el peso será 0. Cuando las neuronas están sincronizadas el peso sináptico aumenta.

Por ejemplo, dada la entrada 1 y la entrada 3 de muchos patrones:



En la imagen se ve el patron k. P es la cantidad de patrones, se calcula con la formula Pmax

Si en la mayoría de patrones, ambos valen +1, el peso es muy grande y positivo, si ambos valen -1 también el peso será grande.
Si uno vale +1 y el otro -1 en la mayoría de los casos, va a dar -1. Esto quiere decir que esas dos neuronas están sincronizadas a la inversa, por lo que el peso va a ser negativo y grande.

Entonces se puede observar que si el peso es grande y positivo hay sincronización, si es negativo y grande hay sincronización inversa y si no hay sincronización el peso será 0. A veces las dos están en +1, a veces en -1, a veces cruzadas, etc.

- +1 +1 o -1 -1 -> w grande y positivo
- -1 +1 o +1 -1 siempre -> w grande y negativo
- Si varia de -1 +1 a +1 -1 a -1 -1 a +1 +1 no hay sincronización -> w = 0

Observaciónes

- **El proceso de entrenamiento NO es interativo.** Le mostramos una vez todos los patrones, se realiza el calculo y termino.
- w_{ji} es mayor cuando las neuronas i y j se tienen que activar juntas (regla de Hebb).
- La capacidad de almacenamiento está limitada a: $P_{max} = \frac{N}{2\ln(N)}$ con un 1% de error. Esta es la cantidad máxima de memorias fundamentales que podemos almacenar.

Prueba (recuperación)
Dado un patron $\boldsymbol{x}$ (incompleto, ruidoso…) se fuerza:

$$\boldsymbol{y}(0) = \boldsymbol{x}$$

Esto es inicializar la salida en el tiempo 0 con la entrada. La primer salida es el patrón que queremos buscar.

Iteración:

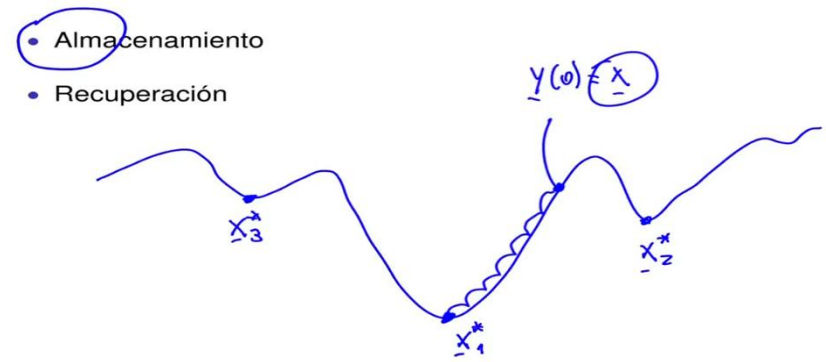
1. Se elige al azar una neurona:
$$j^* = rnd(N)$$
2. Para esa neurona calculamos su salida:
$$y_{j^*} = sgn\left(\sum_{i=1}^{N} w_{ji} y_i(n-1)\right)$$
3. Volver hasta 1 hasta no observar cambios en las y_j

Cuando se cumple que no hay cambios, se convergió a una memoria o patrón fundamental. Es decir, el valor de las salidas es una de las memorias fundamentales almacenadas.

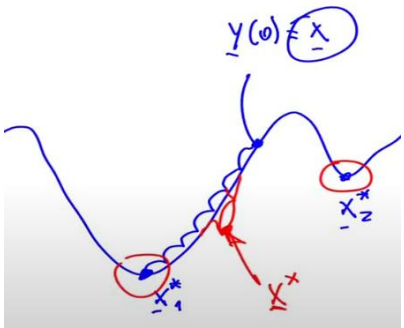
Observaciónes

- **El proceso de recuperación ES iterativo (dinámico)**
  La dinámica es que se calcula y se itera hasta estabilizarse y recién ahí dan una salida. No es como un perceptrón que entra la entrada y sale la salida y se acabó. En cambio, estas redes son dinámicas porque para dar una salida necesitan iterar. ESTAS REDES SON JUSTO LO INVERSO DE LO QUE VENIAMOS VIENDO. ENTRENAMIENTO ESTATICO SALIDA ITERATIVA
- En general no se utilizan los θ_j
- La salida final es $\boldsymbol{y}(M)$ cuando no hay cambios al "recorrer" todas las salidas
- **Se pueden obtener estados espureos y oscilaciones**
  Esto quiere decir que no converge. Esto puede suceder cuando queremos almacenar más memorias fundamentales que las posibles o cuando hay dos memorias fundamentales parecidas.

El almacenamiento y recuperación se puede graficar mediante los campos energéticos de Hopfield:



En almacenamiento se generan estos "valles". En recuperación se itera hasta encontrar la memoria fundamental.
Si no estuvo bien entrenado o superamos la cantidad de memorias fundamentales máximas se genera un valle espureo, en la recuperación se cae en un valle de una memoria que no es fundamental:

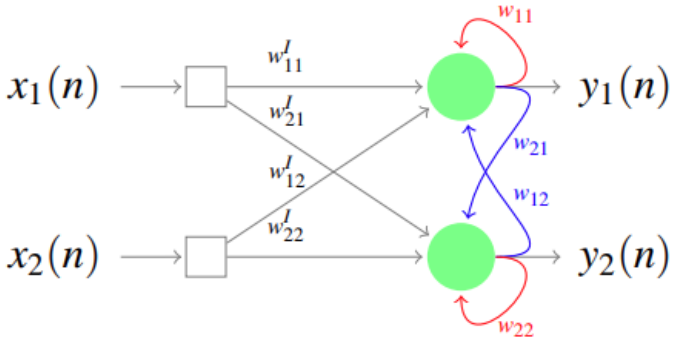


Parcialmente recurrentes (PRNN)

Retropropagación a través del tiempo (BPTT)

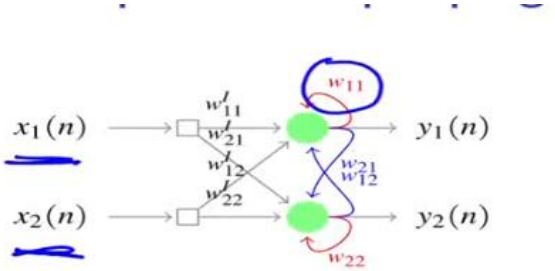
Es la forma de convertir redes recurrentes en parcialmente recurrentes y de esa manera expandirla solo para que tenga conexiones hacia adelante.

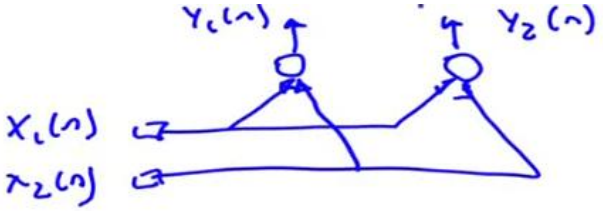
Arquitectura con recurrencia total



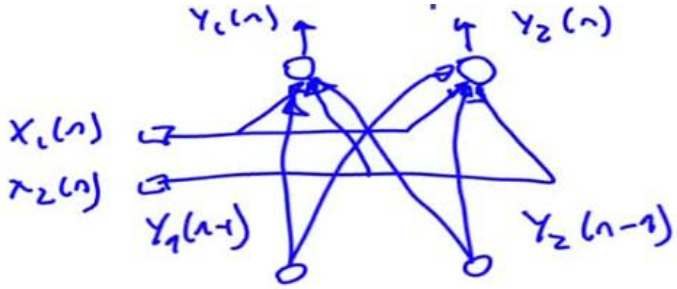
Hay pesos entre las neuronas todas con todas y con sí misma. Hay recurrencias totales.

Para entrenar estas redes se expande la red neuronal a lo largo del tiempo:

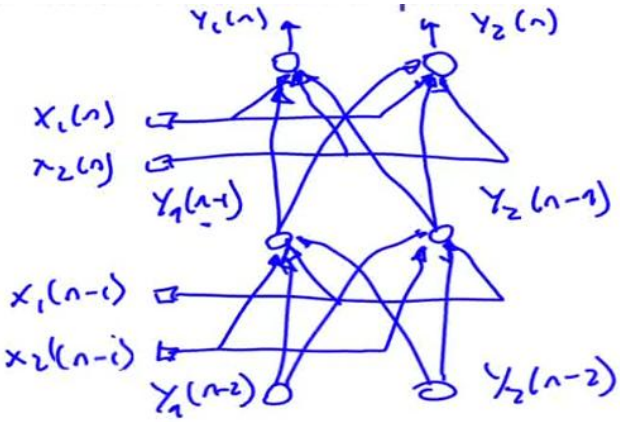




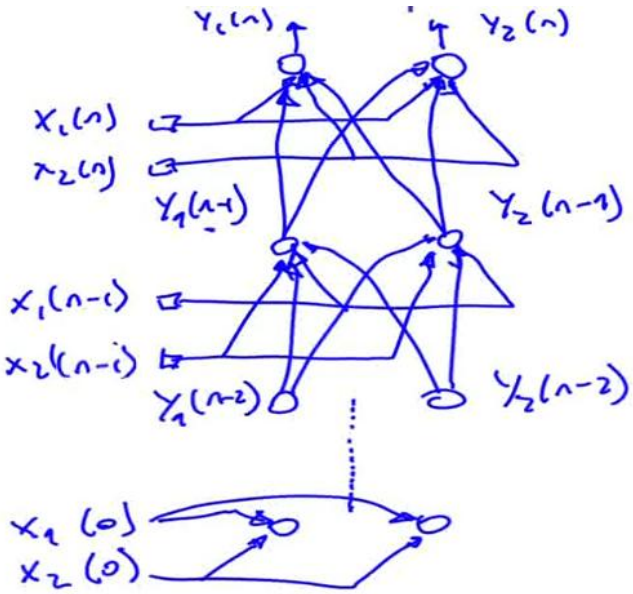
Las neuronas tienen las entradas conectadas a cada una, pero también tenemos como entrada las salidas de ellas mismas en un instante anterior, por lo para calcular las salidas en el instante n se expande:



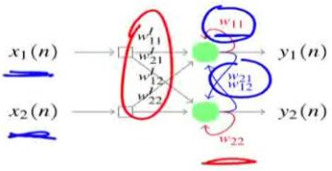
Ahora para calcular la salida en el instante n-1 agrego las entradas, pero para calcular esas salidas también necesito las salidas de las neuronas en dos instantes anteriores:

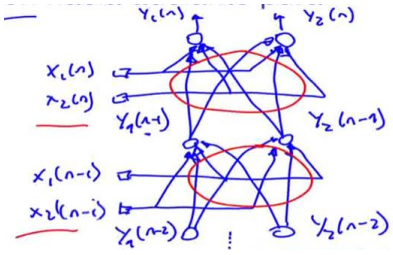


Esto se hace hasta el instante inicial donde se inicializan las salidas con los valores de x:



OJO:



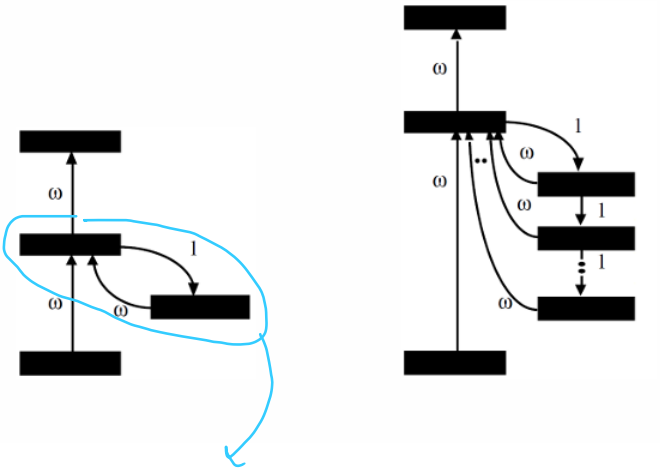


Esta expansión hizo que no existan retropropagaciones y que la propagación sea puramente hacia adelante, por lo que se puede entrenar como entrenamos cualquier perceptrón multicapa. OJO hay que tener en cuenta que los pesos de la primer capa serian los mismos que de la segunda (círculo rojo en grafico derecha), por lo que se tendría que hacer algún promedio luego de la propagación hacia atrás del entrenamiento. Estos pesos son los 8 del grafiquito, los w' y los w.

Se puede hacer retropropagacion truncada diciendo solo quiero ir hasta dos instantes anteriores.

Elman y Jordan

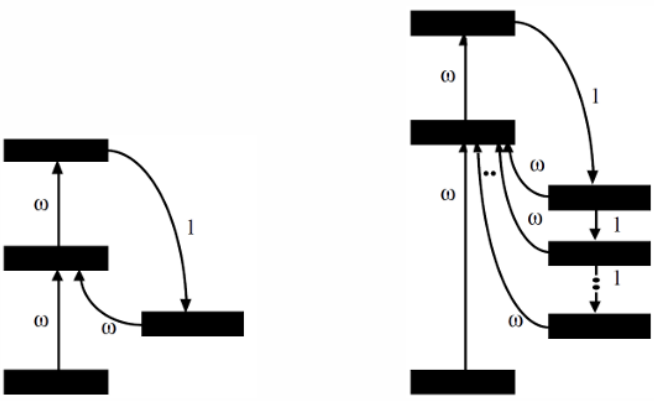
Elman



Hay un retardo de la capa intermedia hacia si misma. La salida de la capa oculta se guarda y en el instante de tiempo siguiente se vuelve a ingresar en esa capa.

A diferencia de las TDNN, estas no son feed forward, sino que hay una realimentación.

Jordan



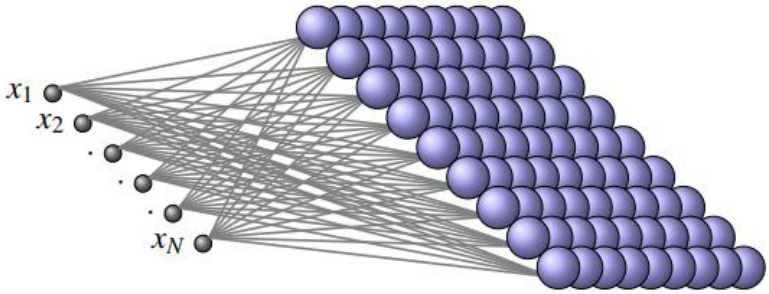
A diferencia de Elman, la realimentación se toma de la salida de la red.

Mapas auto-organizativos: arquitecturas, algoritmo de entrenamiento, mapas topológicos, cuantización vectorial con aprendizaje, comparación con otros métodos de agrupamiento.

Autoorganización: es el proceso en el cual, por medio de interacciones locales, se obtiene ordenamiento global (Turing, 1952).

- Ejemplo 1: evolución de la ubicación de los alumnos asisten a un curso...
- Ejemplo 2: las celulas del cerebro se auto-organizan en grupos de acuerdo a la información que reciben...

SOM: Arquitectura

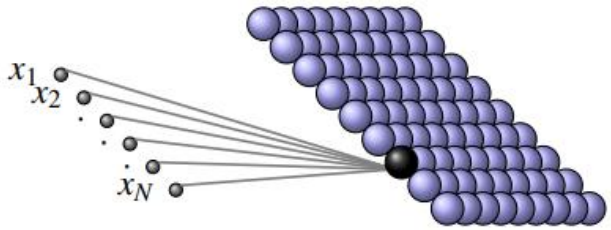


Las neuronas están organizadas en un plano, una entrada se conecta con todas las neuronas mediante un peso. Tenemos vector de entradas, matriz de pesos y capa de neuronas.

Esto se puede ver desde cada neurona:

- Arquitectura SOM 2D

$$\mathbf{x} \rightarrow \mathbf{w}_j \rightarrow s_j$$



Cada entrada tiene un peso asociado con la neurona s_j.

Es una arquitectura competitiva, es decir solo se activa una neurona.

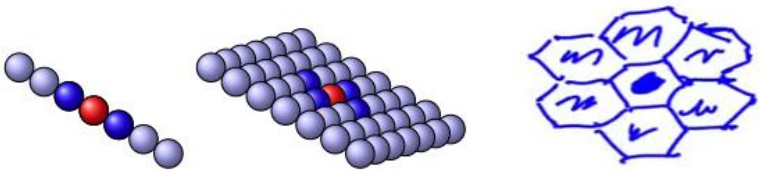
- Salidas: se activa sólo la ganadora j^*

$$j^*(n) = G(\mathbf{x}(n)) = \arg\min_j \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$$

Se compara la entrada x con todo el vector de pesos y aquel peso que se parece más a la entrada es el de la neurona ganadora.

Entornos de influencia o vecindades

- Formas básicas:



Una neurona activada puede estimular a neuronas cercanas. Las neuronas se pueden organizar en una cuadricula, una línea, o en una arquitectura hexagonal.

Estos entornos pueden ser (alcance Λ_G):

- De tamaño fijo
- De tamaño variable

La excitación de las neuronas del entorno con respecto a la ganadora puede ser:

- Excitatorias puras
- Inhibitorias puras
- Funciones generales de excitación/inhibición

Funciones de excitación/inhibición lateral

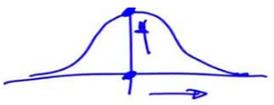
$h_{G,i}$ es la función de influencia en el entorno, y puede ser:

- **Uniforme (entorno simple):**
  por ejemplo para entorno $\Lambda_G = 2$

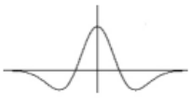
$$\Lambda_G = 2 \Rightarrow \begin{cases} h_{G,i} = \beta(n) & si\ |G-i| \leq 2 \\ h_{G,i} = 0 & si\ |G-i| > 2 \end{cases}$$

  Esto quiere decir que si la distancia entre la neurona ganadora y cualquier neurona es menor o igual a 2, se excita con la constante beta, caso contrario la función de exitacion/inhibición da 0, por lo que no está afectada.
- **Gaussiana:**

$$h_{G,i} = \beta(n) e^{-\frac{|G-i|^2}{2\sigma^2(n)}}$$



  Es decir, cuanto más lejos menos se excita.
- **Sombrero mejicano:** Campos receptivos retina, inhibicion lateral



Entrenamiento de un SOM

Características principales:

- Entrenamiento NO-supervisado
- Aprendizaje competitivo

Proceso:

1) **Inicialización:**
   Hay dos opciones:
   - Pequeños valores aleatorios con $w_{ji} \in [-0.5, 0.5]$ (peso de neurona j con entrada i)
   - Eligiendo aleatoriamente $\boldsymbol{w}_j(0) = \boldsymbol{x}_\ell$ $(\ell \in [1, \dots, L])$.
     en esta segunda se elige una entrada al azar y se le asigna los valores de ese patron a los pesos de esa neurona.
2) **Selección del ganador:**
   Busco que neurona se parece mas al patrón x y la guardo como ganadora.

$$G(\boldsymbol{x}(n)) = \arg min_{\forall j}\{\|\boldsymbol{x}(n) - \boldsymbol{w}_j(n)\|\}$$

3) **Adaptación de los pesos:**
   Si una neurona en la vecindad de la ganadora, al vector de pesos se le suma el vector de error multiplicado por una velocidad de aprendizaje, acercando así los pesos al patrón de entrada.

$$\boldsymbol{w}_j(n+1) = \begin{cases} \boldsymbol{w}_j(n) + \eta(n)\left(\boldsymbol{x}(n) - \boldsymbol{w}_j(n)\right) & si\ j \in \Lambda_G(n) \\ \boldsymbol{w}_j(n) & si\ j \notin \Lambda_G(n) \end{cases}$$

$$\mathbf{w}_j(n+1) = \mathbf{w}_j(n) + \eta(n) h_{G,j}(n)(\mathbf{x}(n) - \mathbf{w}_j(n))$$

   No aparece $\boldsymbol{h}_{G,i}$ porque asumimos vecindad uniforme.
   O se actualiza o no se actualiza

   Formula con $\boldsymbol{h}_{G,i}$
4) **Volver a 2 hasta no observar cambios significativos**

Consideraciones practicas

- El mapa o entorno $\Lambda_G(n)$ es generalmente cuadrado.
- $h_{G,i}(n)$ es uniforme en i, decreciente con n
- $0 < \eta(n) < 1$ es decreciente con n. (velocidad de entrenamiento no fija)

- *Etapas del entrenamiento*

Si arrancamos con una vecindad chica y tasa de aprendizaje chica cada neurona solo va a escuchar a sus vecinas inmediatas y el mapa se puede quedar en un orden global incorrecto.

1) **Ordenamiento global (o topologico)**
   - $\Lambda_G(n)$ grande (≈ medio mapa)
   - $\eta(n)$ grande (entre 0.9 y 0.7)
   - Duracion 500 a 1000 epocas

1 - Para armar bien el ordenamiento global. Esto obliga a que grandes bloques de neuronas se muevan juntas permitiendo "desenredar" el mapa a grandes rasgos para empezar a mirar los detalles

2) **Transicion**
   - $\Lambda_G(n)$ se reduce linealmente hasta 1
   - $\eta(n)$ se reduce lineal o exponencialmente hasta 0.1
   - Duración ≈ 1000 épocas

2 - Reducimos la vecindad hasta 1 para dejar de mover bloques gigantees y enfocarnos en acomodar grupos mas chicos

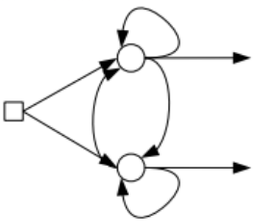
3) **Ajuste fino (o convergencia)**
   - $\Lambda_G(n) = 0$ (solo se actualiza la ganadora
   - $\eta(n) = cte$ (entre 0.1 y 0.01)
   - Duración: hasta convergencia (≈ 3000 épocas)

3 - solo actualizamos la ganadora, para que cada neurona quede lo mas parecida al patron

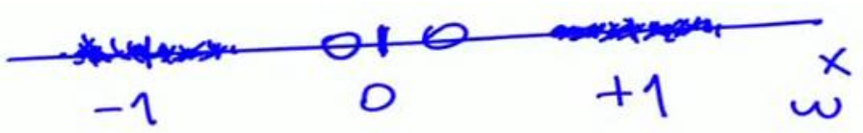
Formación de mapas topológicos

El primer R1 es la cantidad de componentes de entrada, que a su vez es la dimensión de los vectores de pesos de cada neurona. El segundo R1 quiere decir que las neuronas se encuentran ubicadas linealmente (una al lado de la otra). O R2 en un plano

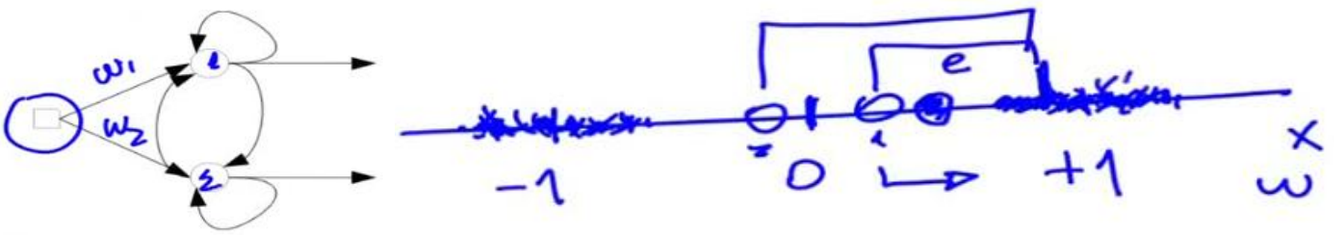
- Ejemplo 1: $\mathbb{R}^1 \rightarrow \mathbb{R}^1$, 2 neuronas



Tengo muchos patrones de un solo número como entrada, supongamos que están cerca de +1 y de -1. Los circulitos representan los pesos que elegimos al azar al comienzo:

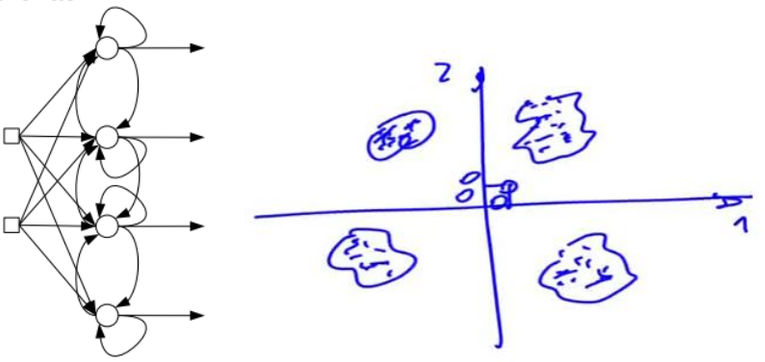


Si entra un patron cercano a +1 gana la neurona 1 porque el error es menor, ajusto los pesos un poco para acercarlo hacia el +1:



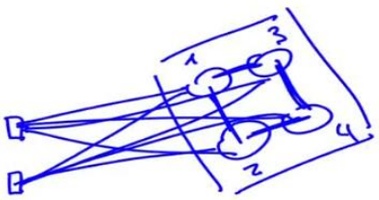
Cuando toque un patrón que haga ganar a la neurona 2, esta se ajustara hacia la izquierda. La neurona 1 terminara en el centro de la distribución de la derecha y la neurona 2 en el centro de la distribución de la izquierda.

- Ejemplo 2: $\mathbb{R}^2 \rightarrow \mathbb{R}^1$, 4 neuronas

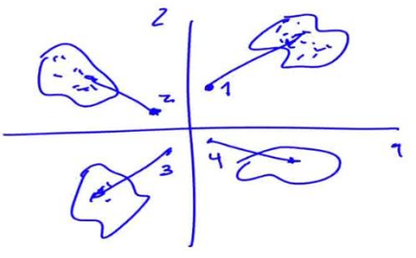


- Ejemplo 3: $\mathbb{R}^2 \rightarrow \mathbb{R}^2$, 4 neuronas

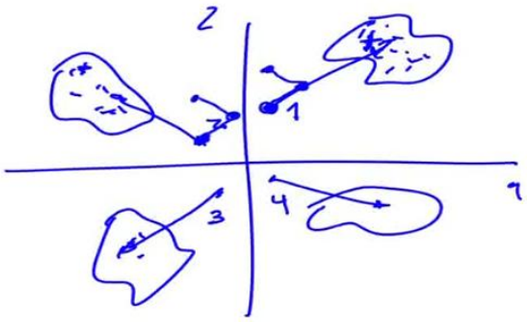
En este caso, las neuronas no están ubicadas linealmente, sino que están ubicadas en un plano. Además, recordemos que las neuronas pueden estar conectadas entre si dependiendo del tamaño de la vecindad:



Como vimos antes, si no hubiera vecindad, los pesos se acercan a los centroides.

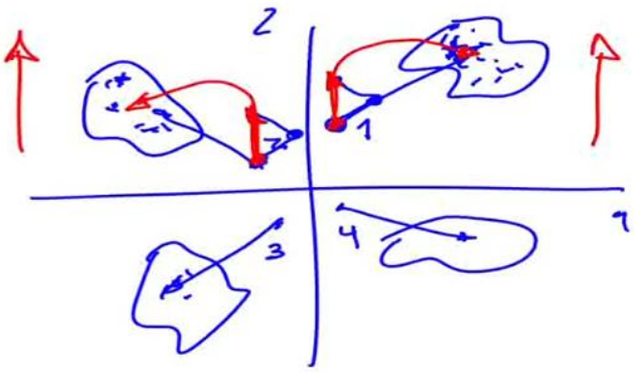


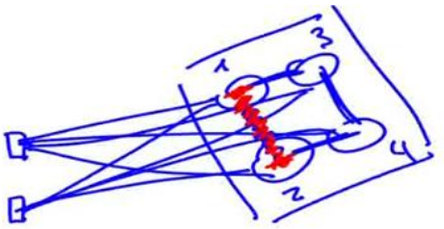
Cuando hay vecindad, si la neurona 1 es vecina de la neurona 2, ambas se mueven juntas al principio:



Esto hace que al comienzo, las neuronas de un entorno se muevan a una misma región del espacio de soluciones sin importar donde hayan comenzado (hacia arriba en imagen) y luego hacia el grupo que les corresponda.
De este modo, las conexiones topológicas entre las neuronas determinan que neuronas que estén cercanas, es decir en una vecindad, también van a representar regiones cercanas en el espacio de solución:

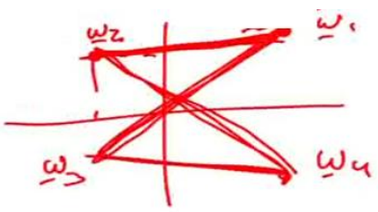
- Ejemplo 3: $\mathbb{R}^2 \rightarrow \mathbb{R}^2$, 4 neuronas





De este modo se va generando el ordenamiento topológico, es decir cómo se logra que neuronas que están cercas en el mapa 2d representan patrones que están cerca en el espacio N dimensional de entrada.

Una forma de graficar esto y que nos permita ver la evolución en el tiempo en R2 es graficar los pesos:



Estos pesos arrancaran cerca del 0,0 y se ira expandiendo en el plano.

- Ejemplo 4: $\mathbb{R}^N \rightarrow \mathbb{R}^2$, M neuronas

El interés principal de los SOM, es que como dije anteriormente, nos permite representar un espacio en dimensiones que no puedo visualizar proyectado en un mapa bidimensional de neuronas donde las neuronas que están cercas representan patrones que también están cerca en el espacio N dimensional.

Un ejemplo de la aplicación de esto es teniendo en cuenta muchas características de los países, agruparlos por similitud.

Como utilizar un SOM para clasificar patrones

El SOM solo permite agrupar de manera no supervisada, pero con los siguientes pasos puede servir para clasificar patrones:

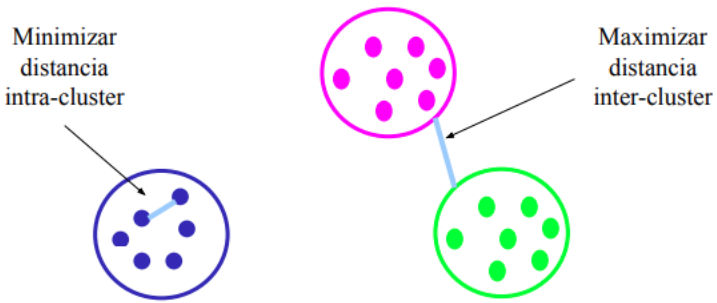
1) **Entrenamiento no-supervisado**
2) **Etiquetado de neuronas.**
   Para esto ahora si tenemos de que clase es cada patrón, entonces veo cuantas veces se activó la neurona con cada clase y con la clase que más se activó la etiqueto.
3) **Clasificación por mínima distancia**
   Ahora cuando entre un patrón de clase desconocida, cuando se active esa neurona, nos dirá que clase es. Para ver cuál es la ganadora veo cual es la mínima distancia entre el patrón de clase desconocida y los pesos.

Comparación con otros métodos de agrupamiento: k-medias

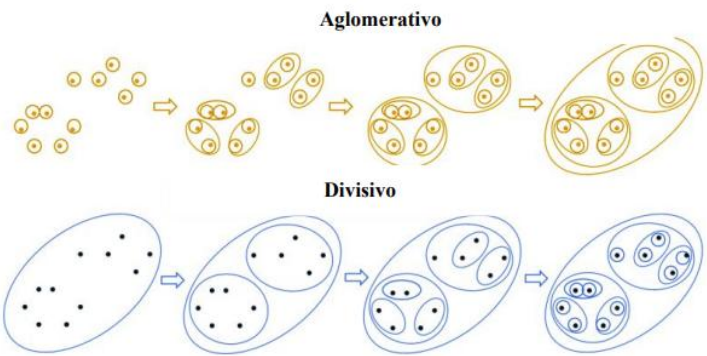
Aprendizaje no supervisado = clustering.

Clustering es agrupar objetos similares basados en alguna nocion de similaridad. El resultado es una partición de los datos. Un **Cluster** es un conjunto de objetos que son similares entre ellos mientras que difieren de los objetos en otros clusters.

Los objetivos de clustering son:



Agrupamiento jerárquico



Aglomerativo: este es un enfoque "de abajo hacia arriba": cada observación comienza en su propio grupo y los pares de grupos se fusionan a medida que uno asciende en la jerarquía.

Divisivo: este es un enfoque "de arriba hacia abajo": todas las observaciones comienzan en un grupo y las divisiones se realizan de forma recursiva a medida que uno desciende en la jerarquía.

K-medias

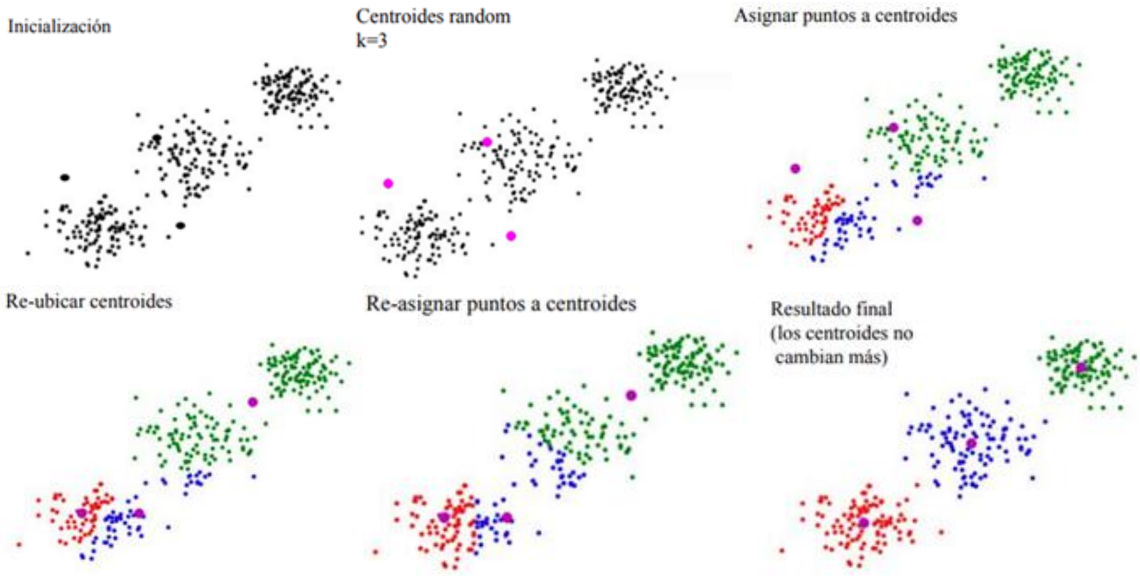
- Algoritmo muy popular y muy simple.
- Particiona los datos en k clusters.
- Utiliza centroides (prototipos de los clusters).
- K-medias es un algoritmo que agrupa objetos en k grupos basándose en sus características.
- Minimizando la suma de distancias entre cada objeto y el centroide de su grupo o cluster.
- Se suele usar la distancia euclídea.
- Las principales ventajas del método k-medias son que es un método sencillo y rápido.
- Pero… es necesario decidir el valor de k a-priori
- El resultado final depende de la inicialización de los centroides.
- En principio no converge al mínimo global sino a un mínimo local.

Algoritmo

El algoritmo consta de tres pasos:

1) **Inicialización**: una vez elegido el número de grupos k, se establecen k centroides en el espacio de los datos aleatoriamente.
2) **Asignación objetos a los centroides:** cada objeto de los datos es asignado a su centroide más cercano.
3) **Actualización de centroides**: se actualiza la posición del centroide de cada grupo tomando como nuevo centroide la posición del promedio de los objetos pertenecientes a dicho grupo.

Se repiten los pasos 2 y 3 hasta que los centroides no se mueven.

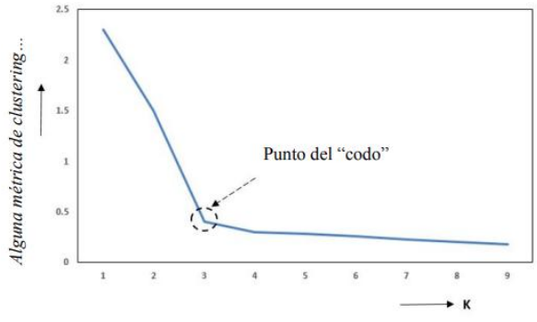


K optimo

El número de clusters (k)

Las métricas de clustering pueden ser usadas para estimar el numero adecuado de clusters para un dataset.

- Eso involucra calcular los resultados de alguna métrica de clustering para un rango de valores de k y plotear la curva resultante.
- El mejor número de clusters puede determinarse como un “codo” en la curva de performance obtenida.



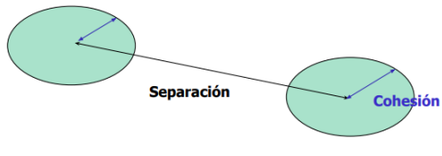
Métricas de clustering

Permiten evaluar los resultados obtenidos con varios algoritmos y varias configuraciones. Además, permite saber si la estructura encontrada por un algoritmo de clustering en los datos es válida.

Medidas internas

Reciben como entradas los datos y las soluciones de clustering obtenidas, se evalúan según la información intrínseca de los datos

- **Cohesión:** evalúa que tan compactos u homogéneos son los clusters obtenidos por una solución.
- **Separación:** mide el grado de separación entre los clusters.
- **Medida combinada o Davies Bouldin:** combina la cohesion con la separación.



Medidas externas

Comparan soluciones entre sí, o contra una referencia (“gold standard”)

- **FM:** determina la similitud entre dos particiones de clusterings.
- **Rand Index (RI):** también mide la similaridad entre dos clusters.

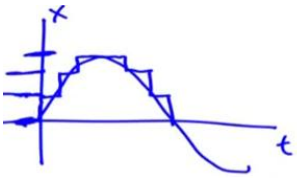
Cuantización vectorial con aprendizaje (LVQ)

Está inspirado en los mapas autoorganizativos. La idea principal es tratar de cuantizar de manera adaptada a las necesidades del espacio. Cuantizar significa separar en cuantos.

LVQ es un clasificador supervisado.

Conceptos básicos:

- **Cuantizador escalar:** señales cuantizadas en escalones



- **Cuantizador vectorial:** podemos cuantizar espacios no necesariamente de una dimensión. Además, puede cuantizar algunas zonas del espacio con mayor detalle y otras con menos.
  - **Centroides o prototipos**
    Los prototipos son el centro de un grupo de patrones. Cuando hablaba de neuronas, una neurona tenía un vector de pesos que era un centroide o un prototipo.
  - **Diccionario o code-book**
    Un conjunto de prototipos hace a un diccionario de prototipos.
  - **El proceso de cuantización:** de vectores a números enteros

La idea es que yo pueda recibir unos vectores en RN y guardar en un archivo comprimido el índice del prototipo que es mas parecido.

Descompongo así por ejemplo una imagen en muchos prototipos o zonas y guardo el índice del prototipo al que más se le parece. Por ejemplo, el prototipo guardado puede ser una curva que es una especie de promedio de varias curvas.

Luego cuando quiero descomprimir voy a poner en esa zona de la imagen el prototipo. Obvio voy a tener un error.

Ideas de como entrenarlo:

- **Algoritmo k-medias etiquetado para clasificación.**
  Los prototipos son los centroides etiquetados de k-medias.
- **SOM etiquetado como clasificador**
  Los prototipos o cuantos son las neuronas etiquetadas del SOM.
- **Algoritmo supervisado LVQ**
  A diferencia de k medias y SOM que realizan el etiquetado y la supervisión aparecen luego del entrenamiento, LVQ nos permite clasificar durante el entrenamiento, es supervisado desde el entrenamiento.

Algoritmo LVQ1

1) **Inicialización aleatoria**
   Asignamos a los cuantos/prototipos valores pequeños al azar o igual a algún patrón del entrenamiento.

$$\boldsymbol{m}_i(0) \in [-0.5, 0.5]; o\ \boldsymbol{m}_i(0) = rnd(\boldsymbol{x});\ \ \forall i$$

mi vector de cuantos

2) **Selección**
   Dado un patrón, busca cual es el centroide o prototipo más cercano a ese patrón:

$$c(n) = \arg min_i\{\|\boldsymbol{x}(n) - \boldsymbol{m}_i(n)\|\}$$

c es el indice del centroide ganador

3) **Adaptación:**

$$\boldsymbol{m}_c(n+1) = \boldsymbol{m}_c(n) + s(c,d,n)\alpha[\boldsymbol{x}(n) - \boldsymbol{m}_c(n)]$$

$$m_c(n+1) = m_c(n) + \alpha[x - m_c(n)]$$

$$m_c(n+1) = m_c(n) - \alpha[x - m_c(n)]$$

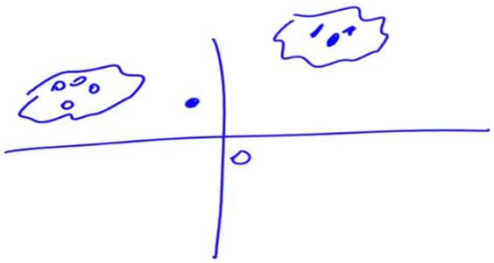
c: indice
d: clase deseada
n: iteracion

$$s(c,d,n) = \begin{cases} +1 & si\ \mathcal{C}(\boldsymbol{m}_c(n)) = d(n) \\ -1 & si\ \mathcal{C}(\boldsymbol{m}_c(n)) \neq d(n) \end{cases}$$

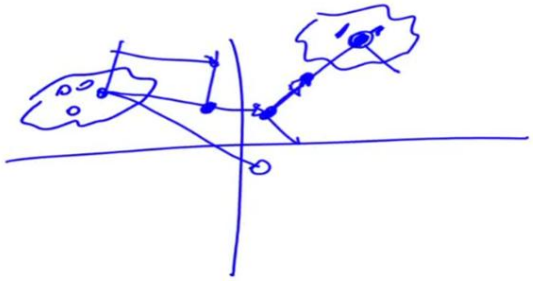
$s(c,d,n)$ es un factor que dice si acierto a la clase o no. Es decir, si la clase del prototipo ganador es igual a la clase del patrón $d(n)$. Si lo acierta acerca el prototipo al patrón que acaba de entrar, si no, lo aleja. α es la velocidad de aprendizaje.

4) **Volver a 2 hasta satisfacer error de clasificación**.

Acá no hay vecindad como en el SOM.



Si llega un patrón de los de la izquierda, el de menor distancia es el circulo pintado, pero como no es de la misma clase se mide el vector de error y se aleja.



Entonces cuando es incorrecta la aleja y cuando es correcta la acerca.

Luego para clasificar entradas desconocidas se medirá la distancia mínima entre el patrón y los prototipos. La clase del prototipo ganador será la clase del patrón.

Observaciones

- No hay arquitectura neuronal
- Se puede ver al cuantizador como SOM lineal, sin entorno y supervisado
- Velocidad de aprendizaje
    - ¿Existe un α_c óptimo para cada centroide?
    - ¿Se debe considerar un $\alpha_c(n)$ optimo para cada instante de tiempo?

Si desarrollamos:

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n)\,[\mathbf{x}(n) - \mathbf{m}_c(n)]$$

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n)\mathbf{x}(n) - s(n)\alpha(n)\mathbf{m}_c(n)$$

$$\mathbf{m}_c(n+1) = [1 - s(n)\alpha(n)]\,\mathbf{m}_c(n) + s(n)\alpha(n)\mathbf{x}(n)$$

$$\mathbf{m}_c(n+1) =$$

$$= [1 - s(n)\alpha(n)] \{\mathbf{m}_c(n-1) + s(n-1)\alpha(n-1)\,[\mathbf{x}(n-1) - \mathbf{m}_c(n-1)]\} + s(n)\alpha(n)\mathbf{x}(n)$$

$\mathbf{x}(n-1)$ es afectado dos veces por α

Siendo $\alpha < 1$ la influencia de los primeros patrones de entrenamiento siempre será menor que la de los últimos. Esto se da porque que el ultimo patrón va a estar multiplicado por un solo α el anterior por 2, el anterior por 3 y así sucesivamente.

Lo que genera esto es que se olvide rápido: las cosas que primero le mostramos van a tener muy poca influencia en la posición del centroide.

Si queremos que α afecte por igual a todos los patrones debemos hacerlo decrecer con el tiempo.

Se debe cumplir que:

$$\alpha_c(n) = [1 - s(n)\alpha_c(n)]\alpha_c(n-1)$$

Es decir, que el α en dos iteraciones consecutivas valga lo mismo. A la izquierda tengo el $\alpha(n)$ y todo lo de la derecha es el $\alpha(n-1)$.

Obtengo así que el α en la iteración n debe valer:

$$\alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n)\alpha_c(n-1)}, \quad no\ sobrepasar\ \alpha > 1$$

Puedo arrancar con $\alpha = 0.9$ y el siguiente calcularlo con esa formulita. Haciendo esto logro que todos los patrones a lo largo del entrenamiento tienen la misma influencia, el mismo peso en el valor final de los centroides.

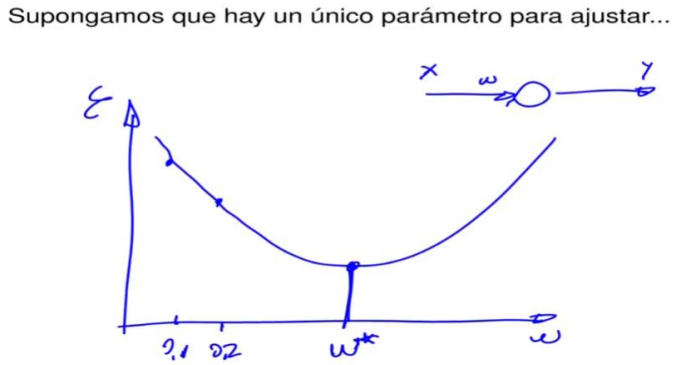
Aprendizaje automático

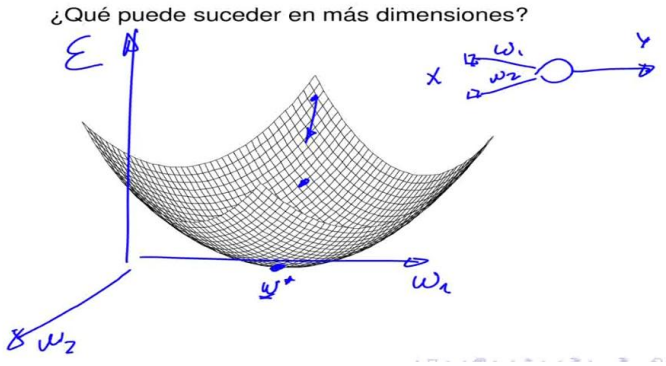
Espacio de soluciones, mínimos locales y globales, capacidad de generalización y técnicas de validación cruzada.

Capacidad de generalización

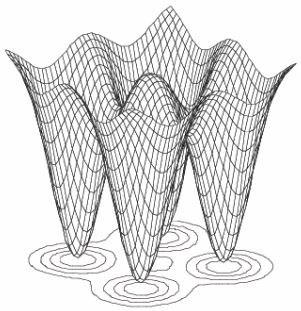
Es la capacidad que tienen los sistemas para funcionar bien en aquellos casos que no vieron durante el entrenamiento.

Superficies de error

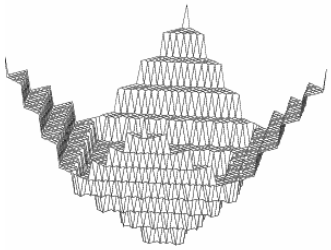




Con muchos mínimos:



Un método para no caer en un mínimo local puede ser inicializar con los pesos al azar muchas veces, para que arranque en distintos puntos de la superficie y me quedo con el mejor mínimo.



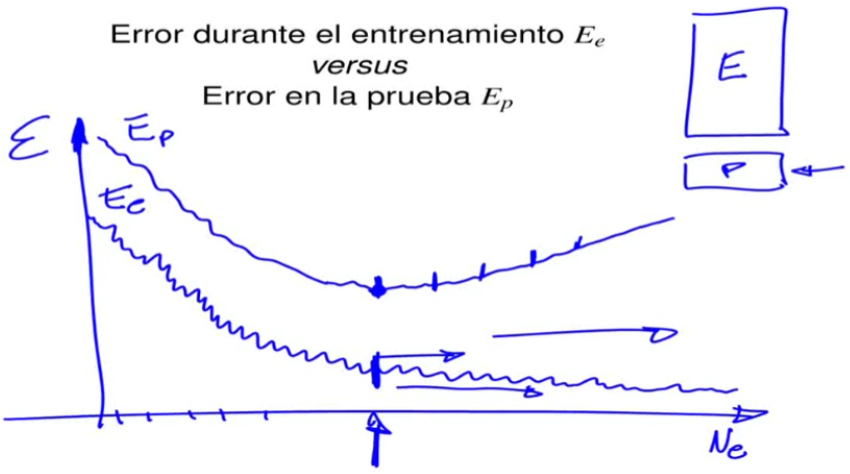
En este caso el método del gradiente se puede quedar en una meseta.

Estos errores se miden durante el entrenamiento, pero necesitamos que el modelo funcione bien en casos más generales.

Sobre entrenamiento

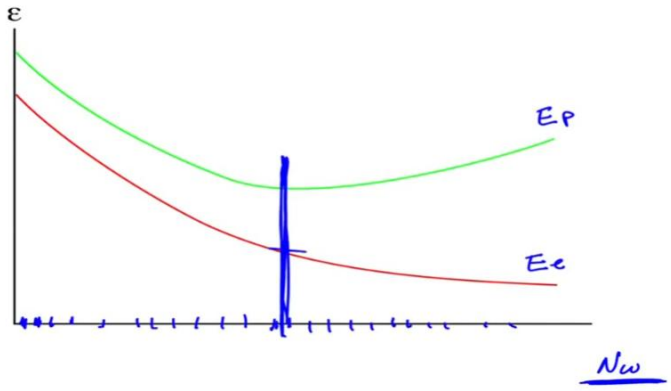
Es cuando un modelo se ajusta mucho a los datos de entrenamiento y pierde capacidad de ajustarse a datos que nunca vio.

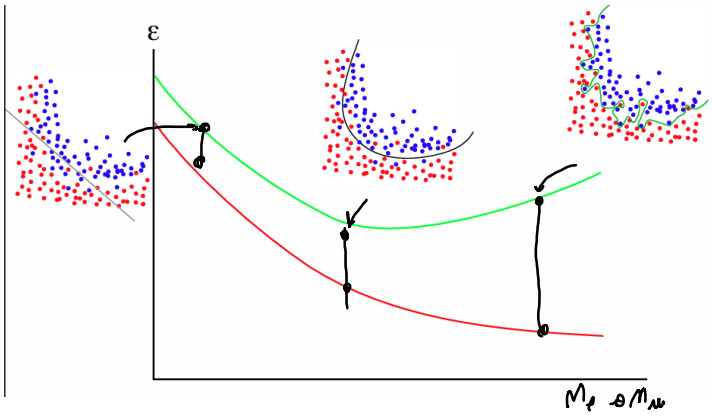
Durante el entrenamiento siempre se va a mejorar el error a medida que pasan las epocas, pero en la prueba, puede ser que llegue un punto en el que el error empiece a subir, ese punto es donde el algoritmo aprendió detalles de los datos de entrenamiento como el ruido propio de la medición, etc., que no es propio del problema que queremos resolver. Esta sobre ajustando o perdiendo capacidad de generalización:



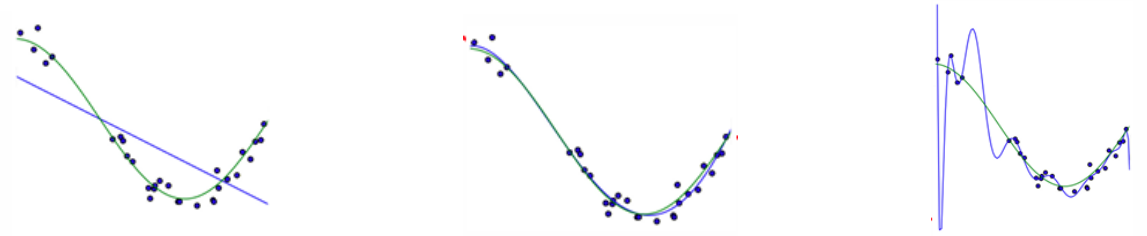
Early stopping es el método de cortar el entrenamiento en ese punto.

Lo mismo puede suceder agregando complejidad al sistema, esto puede ser agregando la cantidad de pesos, es decir la cantidad de neuronas:





Esto puede pasar también con problemas de regresión:



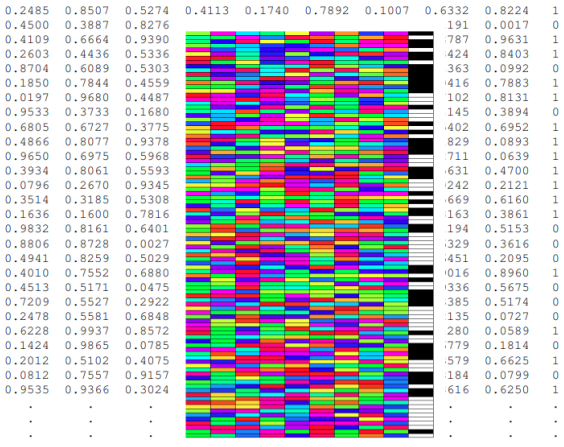
Como evitar? Early stopping.

Recordar que nunca debo usar la parte de prueba para el entrenamiento, por lo que la particion de entrenamiento se parte en entrenamiento y prueba o mejor llamada validacion.

Entonces para medir la capacidad de generalizacion se hace:

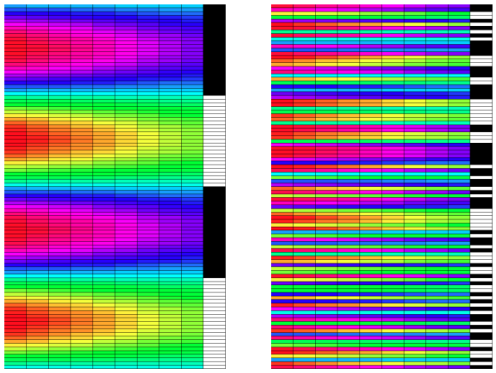
Estimación de la capacidad de generalización

Supongamos que tengo los siguientes datos:

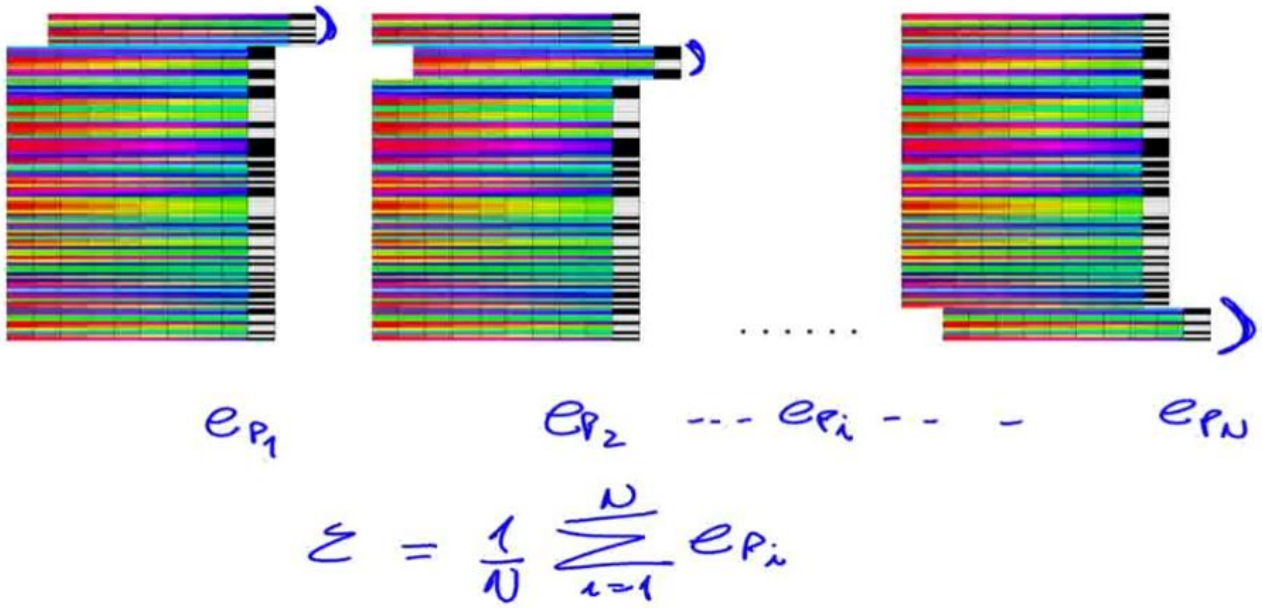


Validación cruzada

El dataset generalmente viene ordenado por clases, por lo que la desordeno para no tomar de validación todos de la misma clase:



Se realizan particiones de prueba y se entrena con dichos subsets:



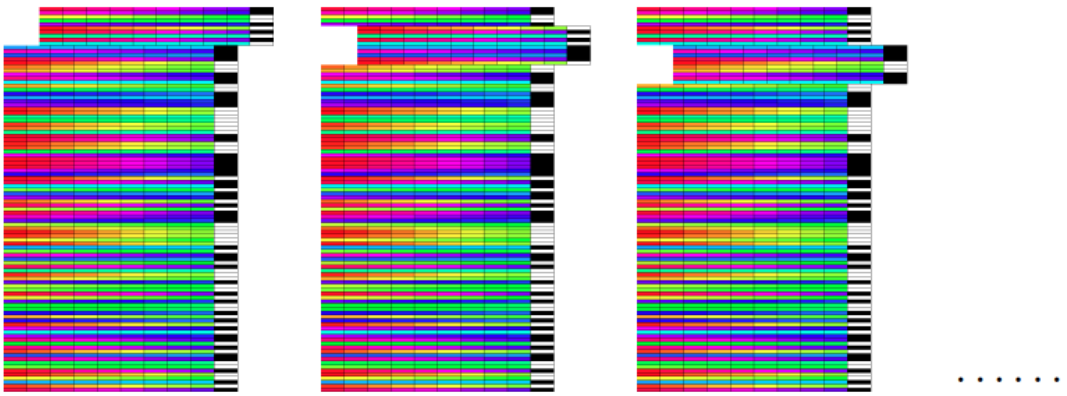
Además de la media del error puedo medir la varianza para ver si las particiones son buenas o si los distintos errores son muy distintos.

Este método se llama leave-k-out, dice cuántas se dejan afuera.

k-fold es cuantas particiones hay, en este caso N particiones.

Otro método es el leave-1-out

Otra manera es no realizar particiones excluyentes y realizar solapamientos:



Otro método es elegir patrones con reposición, es decir que un mismo patrón puede ser elegido varias veces.

Medidas de desempeño

Hablamos de medir el error, esta parte define como medimos ese error.

Para clasificador binario

Matriz de confusión

Matriz de confusión

		Predicciones $\oplus$	Predicciones $\ominus$	
Clases correctas	$\oplus$	$t_\oplus$	$f_\ominus$	$N_\oplus = t_\oplus + f_\ominus$
	$\ominus$	$f_\oplus$	$t_\ominus$	$N_\ominus = f_\oplus + t_\ominus$
				$N = N_\oplus + N_\ominus$

Verdaderos positivos: eran positivos y acerto. Falsos negativos: eran positivos y dijo que eran negativos.

En estos lugares se ponen la cantidad de casos de dicho estilo.

N+ es la cantidad de casos positivos que había, N- la cantidad de casos negativos, N la cantidad de casos.

En base a esta matriz se definen los índices de medidas:

Sensibilidad y especifidad

$$s^+ = \frac{t_\oplus}{t_\oplus + f_\ominus} = \frac{t_\oplus}{N_\oplus} \longrightarrow \begin{cases} \textbf{sensitivity} \\ \text{true positive rate} \\ \text{hit rate} \\ \text{recall} \\ \text{power} \end{cases}$$

$$s^- = \frac{t_\ominus}{t_\ominus + f_\oplus} = \frac{t_\ominus}{N_\ominus} \longrightarrow \begin{cases} \textbf{specificity} \\ \text{true negative rate} \end{cases}$$

Precisión

$$p = \frac{t_\oplus}{t_\oplus + f_\oplus} \longrightarrow \begin{cases} \textbf{precision} \\ \text{positive predictive value} \end{cases}$$

$$n = \frac{t_\ominus}{t_\ominus + f_\ominus} \longrightarrow \{ \text{ negative predictive value}$$

Exactitud y otras medidas compuestas

$$a = \frac{t_\oplus + t_\ominus}{N} \longrightarrow \{ \textbf{ accuracy}$$

$$F_1 = \frac{2t_\oplus}{2t_\oplus + f_\oplus + f_\ominus} = 2\frac{s^+ p}{s^+ + p} \longrightarrow \begin{cases} F_1 \textbf{ score} \\ \text{F-score/measure} \\ \text{harmonic mean} \end{cases}$$

$$G = \sqrt{s^+ p} \longrightarrow \begin{cases} \textbf{G-measure} \\ \text{geometric mean} \end{cases}$$

En general son basadas en lo positivo porque es lo que me interesa, clasificar algo como positivo.

Errores relativos

Dos clasificadores tienen

$$a_1 = 50\,\% \quad a_2 = 51\,\%$$

mientras que otros tienen

$$a_3 = 98\,\% \quad a_4 = 99\,\%$$

Supongamos que $e = 1 - a_i$
el error relativo a la referencia e_r es

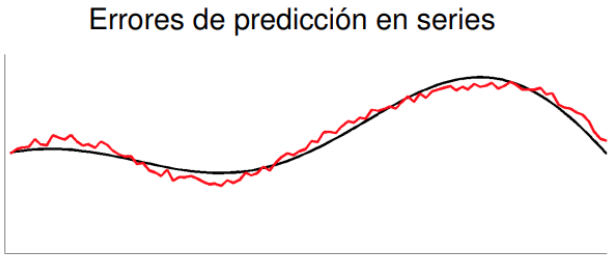
$$\delta_e = \frac{e_r - e}{e_r}$$

e_r seria el clasificador de referencia, por ej a1 o a3.

Parece que en los dos casos el clasificador 2 y 4 mejoran la misma cantidad respecto al 1 y 3, pero no. Para eso sirve el error relativo.

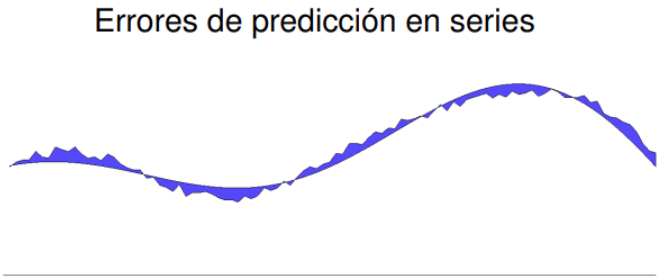
Errores de predicción en series

Se podría pensar lo siguiente:



$$e_i = y_i - \hat{y}_i$$

Esto solo me sirve en instante i. Porque si hago una sumatoria para todos los valores los errores negativos se compensarían con los positivos, entonces lo que se hace es calcular el área sin importar si esta por arriba o por abajo:



$$\tilde{\varepsilon}_A = \frac{1}{N}\sum_{i=1}^{N} |e_i|$$

$$\tilde{\varepsilon}_S = \frac{1}{N}\sum_{i=1}^{N} e_i^2 \qquad \tilde{\varepsilon}_R = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}$$

MAE/MAD	MSE	RMSE
Error medio absoluto.	Error cuadrático medio.	Raíz cuadrada del error cuadrático medio.

RMSE vuelve el cuadrado aplicado a la escala real.

Métodos básicos de aprendizaje supervisado y no supervisado

Supervisado: SVM, Arboles de decisión, Random Forest, redes neuronales, K-NN, boosting

No supervisado: K-Medias, Agrupamiento jerárquico, SOMs

Aprendizaje automático "no neuronal"

Clasificadores más elementales.

Clasificadores Bayesianos Ingenuos (Naive Bayes)

Modelo de clasificación probabilística basado en el teorema de bayes:

$$P(w|\boldsymbol{x}) = \frac{P(\boldsymbol{x}|w)P(w)}{P(\boldsymbol{x})}$$

Supone que las dimensiones o características son condicionalmente independientes (esta es la parte ingenua).

$$P(x_i|w, x_j) = P(x_i|w) \qquad \forall i \neq j$$

Luego de desarrollar se llega a que, considerando la misma probabilidad $P(w)$ para cada clase, y que $P(\boldsymbol{x})$ no depende de w, la clase mas probable se puede obtener mediante:

$$\widehat{w}(x) = \arg\max_{w} \prod_{i=1}^{n} P(x_i|w)$$

Y para un NB gaussiano con (μ, σ^2) se usa:

$$P(x_i|w) = 1/\sigma_w\sqrt{2\pi}\, e^{-\frac{(x_i-\mu_w)^2}{2\sigma_w^2}}$$

Ventajas y desventajas

Ventajas:

- Puede usarse directamente en mezclas de datos continuos y categóricos.
- En varios conjuntos reales funciona mejor que los arboles de decisión y otros métodos simples.
- El tiempo de entrenamiento crece linealmente con la cantidad de dimensiones.
- Rapido y simple de implementar.

Desventajas:

- La hipótesis de independencia no se cumple en muchos datos reales.
- Suele presentar problemas de estimación durante el entrenamiento.

k vecinos más cercanos

El algoritmo mas simple de aprendizaje automático: porque en realidad no aprende.

- Un dato es clasificado por mayoría de votos entre sus vecinos.
- No tiene parámetros que aprender (es no paramétrico).
- Vuelve a buscar los mas cercanos con cada nuevo caso a clasificar (basado en instancias).

Clasificador kNN

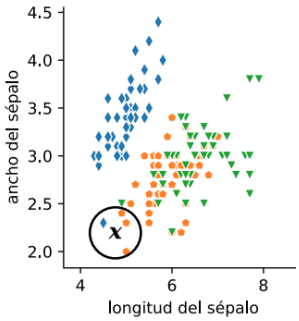
Supongamos que tenemos:

- N_k puntos de la clase C_k
- N puntos en total $(\sum_k N_k = N)$

Para un nuevo $\mathbf{x}$, podemos imaginar una esfera centrada en $\mathbf{x}$

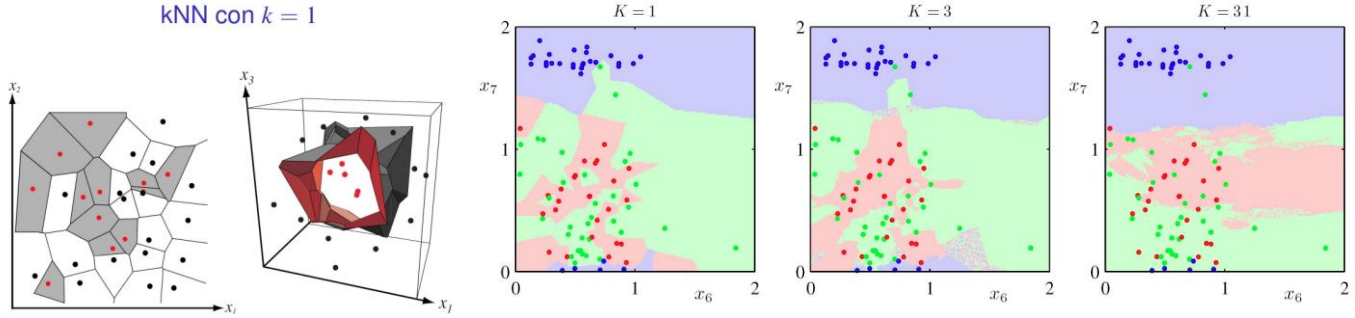
Supongamos que esta esfera:

- tiene un volumen V
- contiene K puntos en total
- de los cuales K_k son de la clase C_k



Esta esfera se va expandiendo hasta que incluya a k puntos, el punto x será clasificado con la clase que más aparece.

kNN: fronteras de decisión



Ventajas y desventajas

Ventajas:

- Muy simple: no hay una regla de aprendizaje, de hecho… no hay aprendizaje!
- No hay parámetros entrenables.
- Es bastante efectivo si el espacio esta bien cubierto por los datos de entrenamiento.
- Puede hacer bastante robusto al ruido en los datos (con el k apropiado).

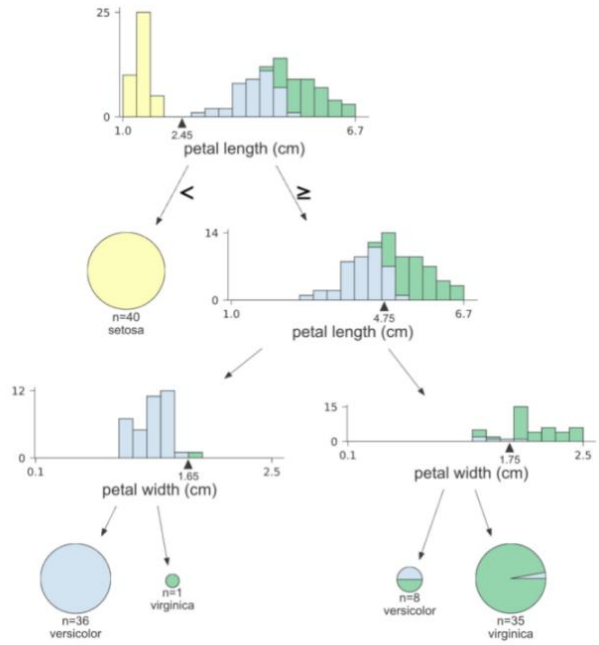
Desventajas:

- Hay que determinar el valor optimo de k.
- No es tan claro que distancia usar en cada caso.
- Para cada dato de prueba requiere calcular la distancia a todos los datos de entrenamiento.
- Sensible a outliers y clases desbalanceadas

Arboles de decisión

Son clasificadores que utilizan reglas IF-THEN para separar los datos en función de sus atributos.

Por ejemplo:



1. Separa si el largo del petalo es menor o mayor a 2.45

2. Menor a 2.45= setosa mayor= versicolor+virginica

2. separa nuevamente por el largo en 4.75

3. luego separa por el ancho.

¿Como elegimos por qué dimensión (nodo) dividir primero? Con las siguientes métricas:

Impureza de clasificación:

$$\mathcal{I}_n^C = 1 - \max_i \mathsf{P}(\mathbf{x} \in w_i|n)$$

Impureza de Gini:

$$\begin{aligned}\mathcal{I}_n^G &= \sum_{i \neq j} \mathsf{P}(\mathbf{x} \in w_i|n)\mathsf{P}(\mathbf{x} \in w_j|n) \\ &= 1 - \sum_i \mathsf{P}^2(\mathbf{x} \in w_i|n)\end{aligned}$$

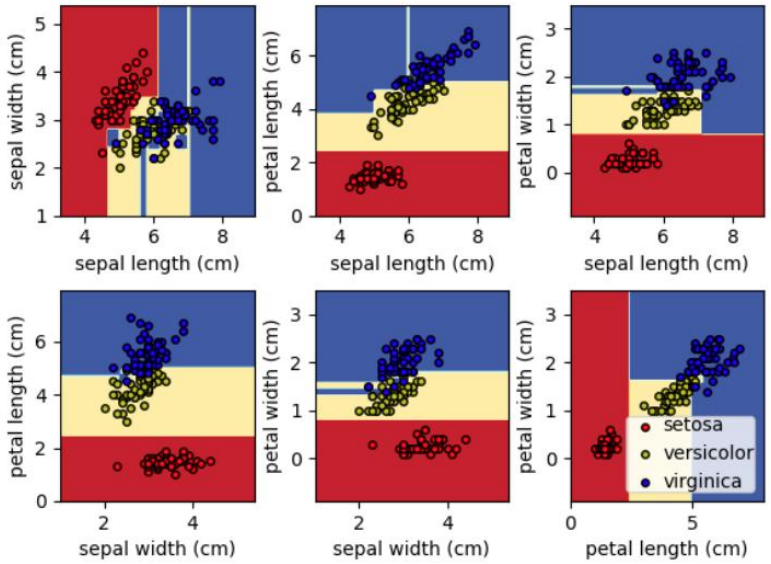
Impureza de Entropía:

$$\mathcal{I}_n^H = -\sum_i \mathsf{P}(\mathbf{x} \in w_i|n) \log \mathsf{P}(\mathbf{x} \in w_i|n)$$

Ganancia de informacion: G = I_p - \sum_h (Nh/N I_h)

Arboles de decisión: fronteras de decisión

Si graficamos las fronteras por cada par de atributos podemos ver las regiones que asigna a cada clase:



Ventajas y desventajas

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.
- Tienen bajo costo computacional tanto para en entrenamiento como para su uso en producción.
- No requieren grandes cantidades de datos para entrenar.
- Bastante robustos a los outliers.

Desventajas

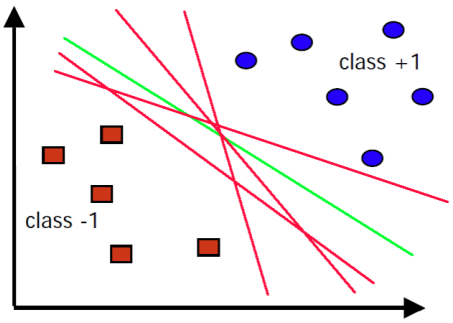
- No son robustos a ruido (pequeños cambios) en los datos.
- Fronteras de decisión ortogonales a las dimensiones de entrada.
- No escalan bien con la complejidad de los problemas a resolver.
- Se sobreajustan con facilidad

Maquinas con vectores de soporte (SVM)

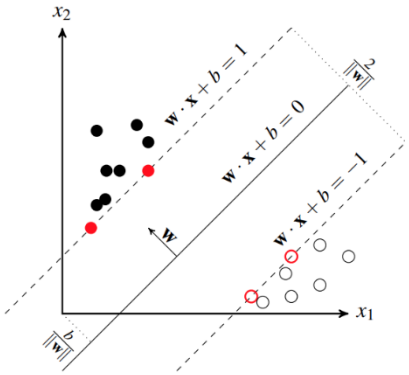
SVM es un algoritmo de aprendizaje supervisado utilizado principalmente para clasificación.

Su objetivo es encontrar un **hiperplano óptimo** que separe dos clases de datos con el **mayor margen posible**.

Margen: cuál sería el mejor clasificador?



SVM: maximizando el margen

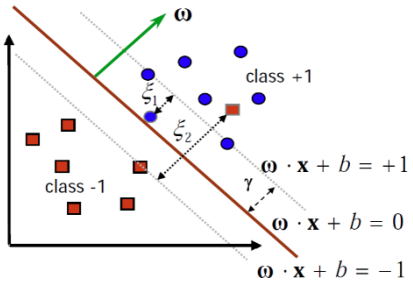


Vectores de soporte

Los puntos en rojo son los **vectores de soporte**, es decir, los puntos más cercanos al hiperplano separador que definen su posición.

SVM: márgenes blandos

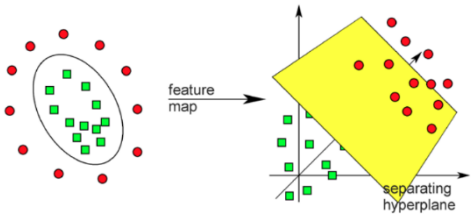
Cuando las clases no son separables linealmente se utilizan márgenes blandos para permitir algunos errores.



El truco del kernel

El **truco del kernel** (kernel trick) es una técnica que permite a las **Máquinas de Vectores de Soporte (SVMs)** manejar problemas donde los datos **no son linealmente separables** en su espacio original. En lugar de trabajar en el espacio original de los datos, el truco del kernel proyecta los datos a un **espacio de mayor dimensión** donde sí pueden ser separados linealmente

Supón que tienes datos en **2D** que no pueden separarse con una línea recta. Aplicando un **kernel**, los datos se transforman en **3D**, donde ahora sí se pueden separar con un plano.



$$\phi : \mathbb{R}^2 \to \mathbb{R}^3$$

$$\phi : (x_1, x_2) \mapsto (z_1, z_2, z3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

Algunos kernels:

- Gaussiano: $e^{-\left(\frac{\|\mathbf{x}-\mathbf{z}\|}{\sigma}\right)^2}$
- Polinomial: $(\mathbf{x}\cdot\mathbf{z}+c)^d$
- Sigmoideo: $\tanh\left(c(\mathbf{x}\cdot\mathbf{z})+\theta\right)$

Ventajas y desventajas

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- Funciona bastante bien en altas dimensiones de entrada.
- Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
- Suele proveer buena capacidad de generalización.

Desventajas:

- El desempeño es dependiente del kernel que se elija (y sus hiperparametros).
- Tiene problemas con grandes conjuntos de datos.
- No maneja bien el solapamiento de clases.
- No provee buena interpretabilidad de las decisiones.
- No es directa la extensión a problemas multiclase.

Ensambles de clasificadores

Consiste en combinar varios modelos simples en lugar de construir un modelo complejo.

Bagging

Consiste en entrenar B modelos separados haciendo Bootstrap sobre el conjunto de datos, cada uno para predecir $y_b(\boldsymbol{x})$.

La predicción del ensamble estará dada por simple votación de las salidas de cada modelo $y_b(\boldsymbol{x})$ (o promedio en caso de regresión).

Se puede medir el error del ensamble E_E en relación al promedio de los errores individuales $\tilde{E}_b$ con: $\tilde{E}_E = \frac{1}{B}\tilde{E}_b$ (random forests).

Hacer Bootstrap sobre el conjunto de datos quiere decir generar muestras de entrenamiento tomando datos al azar con reposición (con reposición quiere decir que, al elegir un patrón, no se elimina por lo que puede ser elegido nuevamente generando que el set de prueba tenga patrones repetidos). Es distinto a k-fold ya que estas muestras se toman del set de prueba que pudo haber sido definido con k-fold por ejemplo.

Boosting

Ideas clave:

- A diferencia del bagging, en boosting modificamos la distribución de los datos de entrenamiento antes de hacer un nuevo muestreo con bootstrap. En resumen, a diferencia de bagging donde todos votan, lo que propone es prestar atención a los casos más difíciles. No toma datos al azar, sino que los vamos a ir modificando a medida de lo que nos vayan dando los clasificadores que usamos.
- Buscamos que la nueva muestra sea más informativa para el nuevo modelo a entrenar.

Boosting por muestreo 3

Usa 3 clasificadores.

Dado un conjunto de entrenamiento S de n patrones, para un clasificador **binario**:

1. Se entrena el clasificador C_1 usando la muestra de bootstrap S_1 de $n' < n$ patrones (sin reposición).
2. Se entrena el clasificador C_2 en un conjunto de entrenamiento S_2 en el que **la mitad** de sus patrones hayan sido clasificados correctamente por C_1.
3. Se entrena el clasificador C_3 con los patrones en los que la predicción de C_1 y C_2 difieren.

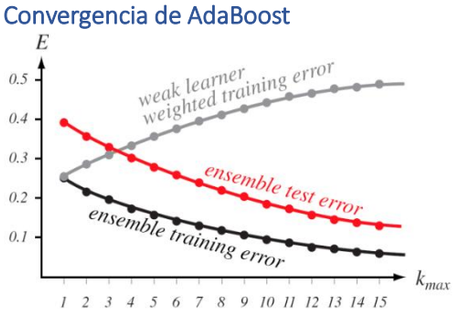
Para un dato de prueba: se usa la predicción de C_1 y C_2 sí coinciden, y si no se usa la predicción de C_3 (equivalente a una votación).

Adaboost

Boosting adaptativo. La idea clave: prestar más atención a los patrones difíciles.

- Cada patrón de entrenamiento tiene asociado un peso que determina la probabilidad con la que va a ser elegido en el boosting.
- Si el patrón se clasifica bien entonces la probabilidad de ser elegido nuevamente tiene que bajar, y a la inversa, aumentar si es más clasificado.
- En un test se usa un voto pesado de acuerdo al desempeño previo de cada clasificador del ensamble. (cuanto menos error haya tenido, mas vale su voto)

Convergencia de AdaBoost



A medida que agrego más clasificadores débiles en el boosting, cada clasificador débil nuevo comete más error porque el algoritmo va aumentando los pesos de los patrones difíciles, así que cada clasificador ve un problema más complicado que el anterior.

Sin embargo, el ensamble completo mejora cada vez más: el error en entrenamiento baja continuamente y el error en test también disminuye, porque cada clasificador corrige los errores del anterior.

Ventajas y desventajas de los ensambles

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalizacion).
- Se adaptan al desbalance de clases.
- Pueden combinar ventajas de multiples clasificadores y fuentes/tipos de datos.
- Hacen un mejor aprovechamiento de los datos cuando son escasos.
- Tratan naturalmente los problemas multiclase.
- Estables y robustos.

Desventajas:

- El entrenamiento puede ser costoso.
- No son tan fáciles de interpretar.
- Los resultados pueden variar mucho dependiendo de la elección de los modelos débiles, su diversidad y la de los datos.

Inteligencia colectiva

Introducción: Autómatas de estados finitos y autómatas celulares. Agentes inteligentes. Inspiración biológica de los métodos de inteligencia colectiva. Modelos de vida artificial: comportamiento emergente, autoorganización.

Autómatas de estados finitos

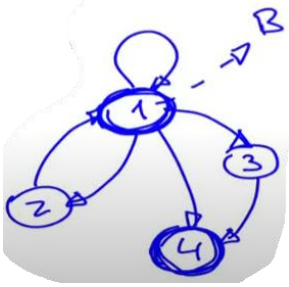
- **Definición:**

$$A =< X, Y, E, D >$$

  Es una estructura que tiene:
  X = entradas, Y = Salidas, E = conjunto de estados, D = Reglas que permiten al autómata ir cambiando de estados
- **Extensión con funciones de salida:**

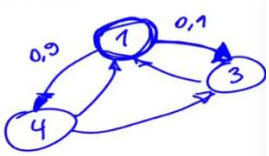
$$y = \lambda(x, E)$$

  A veces la salida puede ser una función del estado que esta y la entrada. Otras veces no hay salida y otras veces es el número de estado en el que esta.
- **Un autómata se representa mediante un grafo**: se pueden visualizar los estados, las reglas de transición y si hay salidas. En algunos casos también se pueden considerar entradas.



En este caso, B seria la salida del estado 1.

- **Las reglas de transición pueden ser**:
  - **Determinísticas:**
    Si es determinística se basa en alguna regla, función, etc. Por ej, si la entrada es mayor que 5 pasa al estado 2.
  - **Probabilísticas**



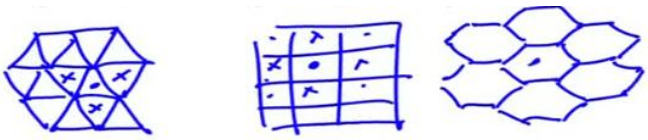
Hay una probabilidad que asocia las transiciones entre estados.

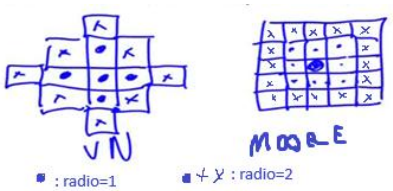
Autómatas celulares

- **Definición:** Son otra estructura compuesta por

$$R =< A, T, C >$$

  A = autómatas, T = topología, C = como se conectan entre si
  Vendría a ser un conjunto de autómatas que se vinculan entre si.
- **Topologías (T):**
  - Triangular
  - Rectangular
  - Hexagonal
  - Etc


- **Acoplamiento (C):** dentro de una topología, como se conectan entre si
  - **Tipos y tamaños de vecindad:**
    - Von Neumann: solo los que comparten un lado
    - Moore: incl. diagonales
  - **Tipos de conexiones:**
    - Isotrópicas: iguales en todas las direcciones
    - Anisotrópicas: conectadas con un sentido preferencial



Ejemplos de autómatas celulares: Juego de la vida de Conway: un juego que para un organismo, dependiendo de los organismos del entorno se define si vive o muere. con esto se puede simular el crecimiento de plantas, bacterias, poblaciones, tejidos biológicos: cardiaco, nervioso. Fluidos.

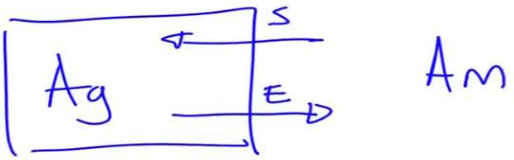
Agentes

Un agente es un sistema que:

- Esta situado en un ambiente
- Es capaz de realizar acciones automáticas
- Tiene objetivos de diseño

Un agente es todo aquello que:

- Percibe su ambiente mediante **sensores**
- Responde o actúa sobre el ambiente mediante **efectores**



Todo agente debe poseer **autonomía**, es decir, capacidad de:

- Aprender de la experiencia
- Modificar comportamiento en tiempo de ejecución

Un **agente inteligente** debe:

- Ser proactivo: realizar acciones sin requerimiento especifico.
- Ser reactivo: reacciona
- Poseer habilidad social: interactuar con otros agentes

Un **agente racional** es aquel que

- Realiza acciones correctas

Un **agente racional ideal** debe ser capaz de:

- Percibir
- Conocer
- Decidir
- Actuar

Sistemas multi-agente

Son cooperativos:

- Por interacción
- Por contratos
- Por negociación

Un agente posee racionalidad social:

- Si puede realizar acciones que generan un beneficio **a todos**
- Si ese beneficio es más grande que las perdidas

$$\text{utilidad esperada} = f(\text{utilidad individual}) + f(\text{utilidad social})$$

Inteligencia colectiva: conceptos básicos

Denominaciones y relaciones:

- Computación evolutiva: basados en la teoría de Darwin
- Inteligencia de colonias
- Inteligencia de enjambres
- Inteligencia colaborativa
- Inteligencia social

Características generales:

- Auto-organización (relación con SOM...). Es la capacidad de armar una topología o un mapa que tiene que ver con la estructura de los datos en un espacio n dimensional. No la definimos nosotros sino que se va ordenando automáticamente.
- Estigmergía: “colaboración a través del medio físico”. Ej rasgo de hormonas en colonia de hormigas.
- Comportamiento emergente: inteligencia distribuida, robustez. Es un comportamiento que no es definido de antemano, sino que emerge a partir de la colaboración de individuos sencillos
- Fuerte interacción local. Fuerte interacción con los que están cerca.
- Organización social altamente estructurada. Se empiezan a formar roles o jerarquías.
- Colaboración versus competencia
- Componentes estocásticas. Son los valores que se determinan al azar.
- Bio-inspiración

Ejemplos de bio-inspiracion:

- Bandadas de pájaros
- Colonias de hormigas
- Paneles/enjambres de abejas
- Cardúmenes de peces
- Rebaños de ovejas o cabras
- Manadas de predadores

Elementos individuales:

- "Boids": partículas, objetos, elementos...
- Autómatas: redes de autómatas celulares
- Agentes: en estructuras de multi-agentes
- Neuronas...? como con el som.

Algoritmos(con tilde los que se ven en la materia):

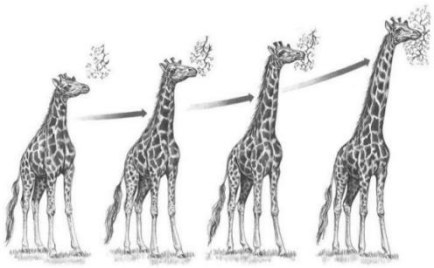
- ✓ **Algoritmos evolutivos**
- ✓ **Colonias de hormigas**
- ✓ **Enjambre de partículas**

Principales aplicaciones:

- Optimización: aproximación de funciones, entrenamiento, estimación, identificación, planificación,...
- Búsqueda, ruteo
- Agrupamiento no supervisado (clustering), clasificación

Métodos evolutivos: inspiración biológica, estructura, representación del problema, función de aptitud, mecanismos de selección, operadores elementales de variación y reproducción.

Creacionismo: las especies de la naturaleza fueron creadas como tales
Evolucionismo: hubo cambios a lo largo de los años



Lamarck: creía que los caracteres adquiridos en vida iban a ser heredados a la siguiente generación.
Por ejemplo, una generación nueva iba a tener el cuello más largo que la anterior.

Ahora sabemos que los caracteres adquiridos en vida no se heredan, sino que ya están codificados en el material genético.

Teoría de la evolución de Darwin: Variación y selección natural

Variación y selección natural: si hay variabilidad en la longitud del cuello de las jirafas, las de cuello corto tendrán menos probabilidades de sobrevivir y procrear. Así...





...en la próxima generación habrá menos jirafas de cuello corto.

Inspirándonos en esto, surge la evolución como un algoritmo:

La evolución como un algoritmo

Inspirado de

- ✓ El creacionismo y las ideas de Lamarck
- ✓ Darwin versus Lamarck
- Poblaciones versus individuos
- Mejores versus "adaptados": busca la mejor solución a las condiciones actuales, no quiere decir que sea el mejor global.
- Aleatoriedad en la selección natural
- Diversidad y operadores de variación en la población

La evolución como un algoritmo

```
Inicializar(Población)
MejorAptitud ← Evaluar(Población)
mientras MejorAptitud < AptitudRequerida
    Progenitores ← SelecciónNatural(Población)
    Población ← ReproducciónVariación(Progenitores)
    MejorAptitud ← Evaluar(Población)
fin
```

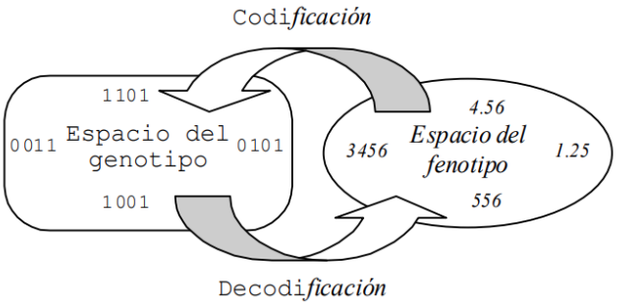
la población son potenciales soluciones al azar.

Elementos de un algoritmo evolutivo

- Representación de los individuos
- Función de aptitud (o fitness)
- Mecanismo de selección
- Operadores de variación
- Reproducción y reemplazo generacional

Variantes de la computación evolutiva

Basándonos en cómo se **representan los individuos**, el algoritmo/programación/estrategia puede ser:



- **Genético:** Representación BINARIA o "genotípica"
  - Muchos genes con pocos alelos (2: 1 y 0): convergencia asegurada por el teorema de esquemas.
  - Epítasis: un gen incorrecto invalida a todo el cromosoma (un cambio en un bit puede generar un valor muy distinto o invalido en el fenotipo)
  - Representación lejana al dominio del problema (estoy transformando del dominio del problema al genotipo)
  - Gran cantidad de soluciones invalidas en la población (no se diseña bien la representación de los individuos la población va a tener muchos individuos inválidos)
- **Evolutivo (o estrategias de evolución):** representación REAL o "fenotípica"
  - Pocos genes con muchos alelos: representación fenotípica.
  - Convergencia muy dependiente de los operadores
  - Necesidad de redefinición de operadores no "biológicos" (redefinir el operador de cruza, mutación porque mutar ya no es cambiar un 0 por un 1 si es en un número real)
- **Otras representaciones** como cromosomas de longitud variable, arboles, grafos

Genético

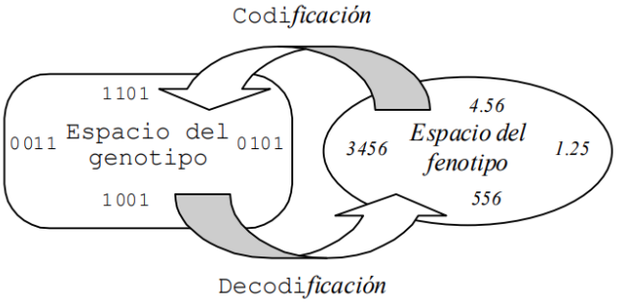
Representación de los individuos

Es como codificamos las soluciones.

Fenotipo: la forma final que toma ese genotipo, por ej, el color de una hoja.

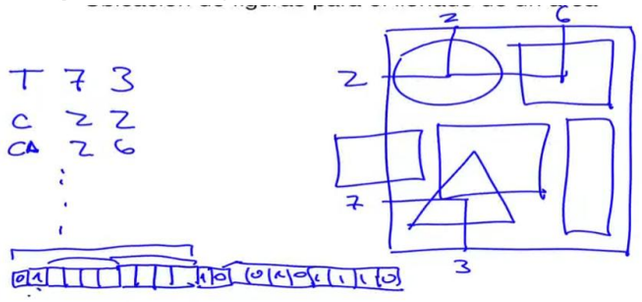
Un individuo debe representar una solución al problema. Un individuo es un **cromosoma**

Cada bloque de genes (bits 0 o 1) representa una característica.

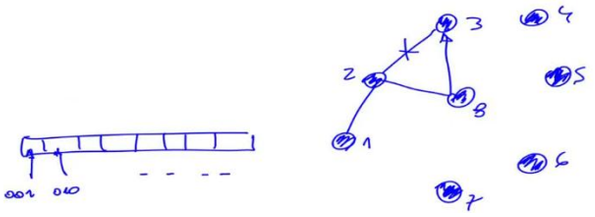


Ejemplos

- Ubicación de figuras para el llenado de un área: primeros dos bits la figura, los primeros 4 bits las coordenadas en x y los siguientes 4 en y.



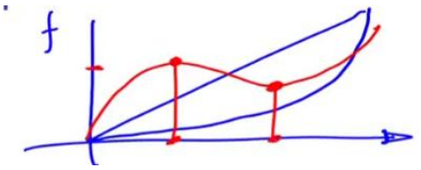
- Entrenamiento de una red neuronal: los primeros bits pueden ser la cantidad de neuronas de la primer capa, de la oculta, de la de salida, los pesos, …
- Problema del agente viajero: cada 3 bits es una ciudad



Función de aptitud

Como medir que tan bueno es un individuo para resolver el problema. Nos sirve para elegir a los padres de la siguiente generación.

- Características generales:
  - **Monotonicidad:** creciente

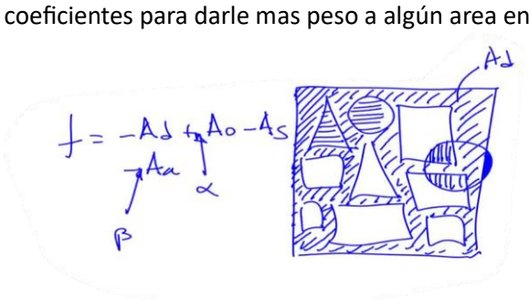


Rojo es erróneo, a mayor bondad del individuo, debe mayor fitness.

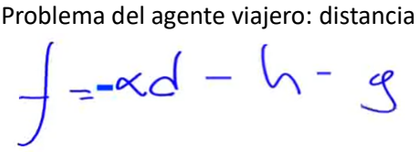
  - **Precisión:** debe diferenciar individuos que sean apenas parecidos
  - **Suavidad regulable:** debo poder variar la suavidad (ej en la sigmoidea variando el alpha)
  - **Penalización de complejidad:** ante una solución simple y una compleja debe elegir la simple.
- Algunos ejemplos típicos:
  - Promedios de error: cuadrados medios, desviación media absoluta, error relativo medio,...
  - Estadísticas: estimación de la varianza, validación cruzada, verosimilitud, predicción de error,...
  - Medidas de información: criterio de Akaike, criterio de información Bayesiano, descriptor de mínima longitud, información mutua, minimización del riesgo empírico
  - Otras: correlaciones, distancias,...

Ejemplos

- Ubicación de figuras para el llenado de un área: área de tela desperdiciada Ad, busco minimizar esa área, también el área ocupada Ao, también el area solapada As y tambien el area por afuera de la tela Aa, también se pueden agregar coeficientes para darle mas peso a algún area en particular



- Entrenamiento de una red neuronal: tasa de acierto (clasificaciones correctas), o error, etc
- Problema del agente viajero: distancia total recorrida, costo de hoteleria, etc

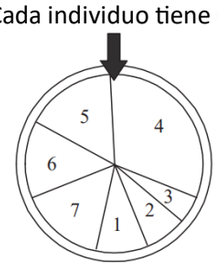


Estas dos cuestiones dependen mucho del problema, pero los siguientes operadores se usan para cualquiera:

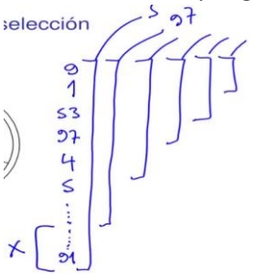
Estrategias de selección

(de padres para la próxima generación)

- **Rueda de ruleta**
  Cada individuo tiene un área proporcional a su fitness.



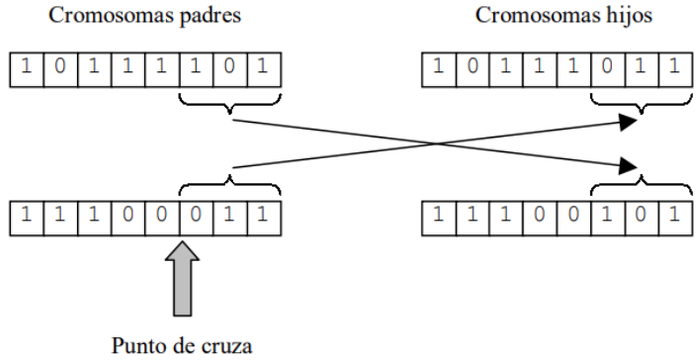
  Problemas:
  - **Mar de mediocres:** si tengo 1 individuo con mucho fitness y muchos individuos con bajo fitness, es más probable que salga elegido uno de estos últimos.
  - **Mar de virtuosos:**
    Si tengo fitness muy parecidos, va a elegir prácticamente al azar.
    Por lo tanto falla cuando la función de distribución (un histograma) es picuda.
- **Ventanas**: En ventana, se ordenan de mayor a menor en fitness, y primero elige una ventana grande al azar, luego achica la ventana y elige al azar. De cada ventana elige un progenitor.



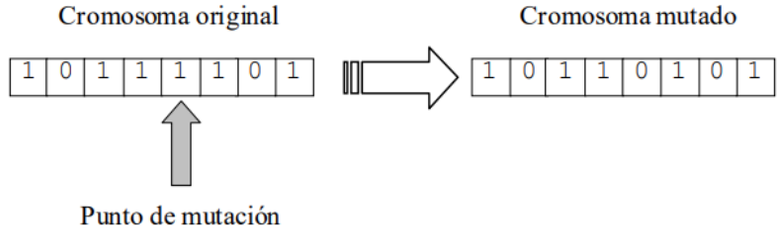
- **Competencias**: Se eligen k individuos, y el que tiene mayor fitness gana

Operadores de variación

- **Cruzas simples**
  Se elige un punto al azar y se intercambian los genes.



  El rol de esto es combinar las mejores partes de los padres, es decir es afinar la búsqueda del mínimo error en una superficie de error.
- **Mutaciones**
  Se toma un punto al azar de los hijos si se desea y se muta ese gen.



  La tasa de mutación puede ser a nivel de individuos, a nivel de cromosomas o a nivel de genes.
  En la búsqueda sirve para explorar regiones lejanas en el espacio.

Operador de reemplazo durante la reproducción

- **Reemplazo total:** reemplazo la generación por todos los hijos
- **Reemplazo con brecha generacional:** hay generaciones en las que pueden convivir padres con hijos. Por ejemplo se eligen 10 padres y con esos 10 se generan los 90 restantes para llegar a el tamaño de la población de 100.
- **Elitismo:** se toma al mejor individuo de la población y se lo pasa a la siguiente generación. La consecuencia es que la función de fitness aumentara de a escalones.

Restricciones en el dominio de la aplicación

Es cuando queremos incorporar una restricción al problema, por ejemplo, para que no haya figuras fuera del espacio que queríamos en el ejemplo 1.

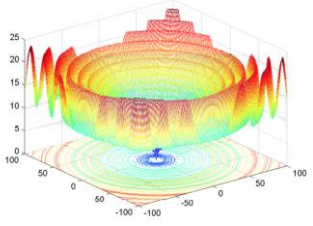
Las opciones para establecer restricciones al problema durante la evolucion son:

- **Redefinición de la representación de forma que siempre se generen fenotipos válidos.**
  Es decir, buscar la forma de que, en la misma codificación, sin importar como actue la cruza o la mutacion, el individuo resultante va a ser siempre valido
  Ej. para representar hasta el número 100, con 7 bits llego de 0 a 127, para solucionar puedo hacer una escala, 0=0, 127=100
- **Rechazo o eliminación de individuos invalido**
  Al hacer una mutación o cruza, si veo que es invalido lo elimino y la hago nuevamente. Esta solución no es tan deseable.
- **Reparación del material genético**
  Corrijo ciertas partes del material genético para que vuelva a ser valido.
  Ej. si una figura queda por fuera del área deseada, la muevo dentro del área. Tampoco es tan deseada.
- **Modificación de los operadores de variación**
  Modificación para que no generen individuos inválidos.
  Ej. limitar donde se puede realizar una cruza o mutación. Un poco más deseada que las anteriores, pero menos que la redefinición de la representación.
- **Esquemas de penalización en la función de aptitud**
  Por ejemplo, al penalizar cuando la figura esta por fuera del área. Se hace que caiga la función de fitness.

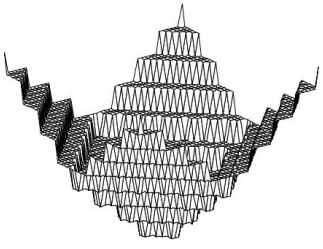
Análisis de las características principales

- **Búsqueda en un espacio codificado de parámetros:**
  la búsqueda se hace en el genotipo, es decir en el espacio de los cromosomas, ventajas: existe un teorema que demuestra que está asegurada la convergencia
- **Búsqueda en múltiples puntos del espacio de soluciones**, se inician en ubicaciones al azar en el espacio de soluciones y con mutaciones y cruzas se van a ir ubicando en distintas regiones hasta encontrar la solución. El gradiente no encontraría a donde ir. El gradiente tampoco serviría en regiones planas.

• Ejemplo 1:



• Ejemplo 2:



- Pocos requisitos sobre la función objetivo: como en el gradiente necesitamos la derivada de la función, acá no necesito saber una formula analítica de la función objetivo.
- Algoritmo de naturaleza estocástica: combina algo de determinismo con algo de naturaleza estocástica. Hay reglas, pero con componentes aleatorios.
- La estructura de los operadores los hace muy efectivos al realizar búsquedas globales
- Múltiples objetivos: por ejemplo, con el área, aprovechar al máximo, no solapar, etc
- ¿Algunas desventajas...? Son lentos, requieren mucho costo computacional. Evaluar la función de fitness es lo costoso. Para esto se pueden paralelizar en distintos clusters.

Comparación con otros métodos

Métodos tradicionales	**Algoritmos evolutivos**
Trabajan con los propios parámetros a optimizar	Emplea una codificación de los parámetros. (en binario)
Utilizan información de las derivadas de la función objetivo u otro conocimiento adicional	Utilizan la información de la función objetivo en forma directa
Reglas de transición deterministas	Reglas de transición combinando componentes determinísticos y probabilísticos
Exploran el espacio de soluciones a partir de un punto	Exploran el espacio de soluciones en múltiples puntos a la vez

Evolutivo / Estrategias de evolución

Nosotros anteriormente nos centramos en los genéticos nomas (cromosomas binarios). Pero ahora:

Si los parámetros tales como:

- Probabilidad de mutaciones
- Probabilidad de cruzas
- Tamaño de la población
- Brecha generacional
- Elitismo

Fueran adaptables, surgen los algoritmos llamados **estrategias de evolución**.

Para esto se utilizará:

- **Representación fenotípica**
  Es decir que se pone el valor directamente en los cromosomas, sin convertirlos a binarios. Por ej, se ponen los pesos tal cual de una red neuronal.
  Dentro de esta representación tendremos:
  - **Variables objetivo:** las que venimos evolucionando, pesos, coordenadas, etc.
  - **Variables de control:** controlan la evolución ejemplo probabilidad de mutación.
- **Fitness:** igual que en algoritmos genéticos.
- **Operadores:**
  - Mutación (solo si no empeora)
  - Cruza (optativa)
- **Selección:** aleatoria, combinada con mutación.
  Se muta un padre elegido al azar y si ese padre mejoro, se lo deja para la próxima generación, sino se elimina.
- **Reproducción:** determinística, hay dos esquemas
  - **Mecanismo $(\boldsymbol{\mu}+\boldsymbol{\lambda})$:** μ padres producen $\lambda \geq 1$ hijos.
    Próxima generación: $\{\mu \cup \lambda\}$ eliminando los peores de λ
  - **Mecanismo $(\boldsymbol{\mu}, \boldsymbol{\lambda})$:** μ padres producen $\lambda > \mu$ hijos.
    Próxima generación: subconjunto con los mejores de λ

Programación genética

Es otro algoritmo. Es simplemente generar programas de forma automática.

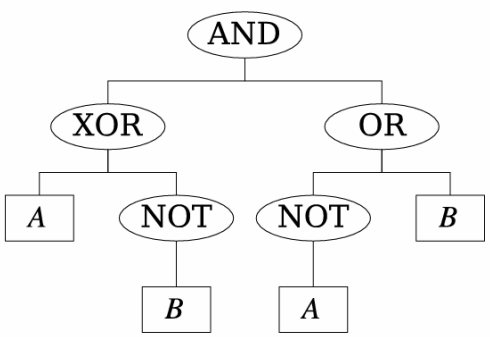
Ejemplo

De los elementos básicos de un programa:

- Variables y constantes
- Operadores aritméticos y lógicos
- Funciones matemáticas
- Condicionales
- Bucles
- Recursiones

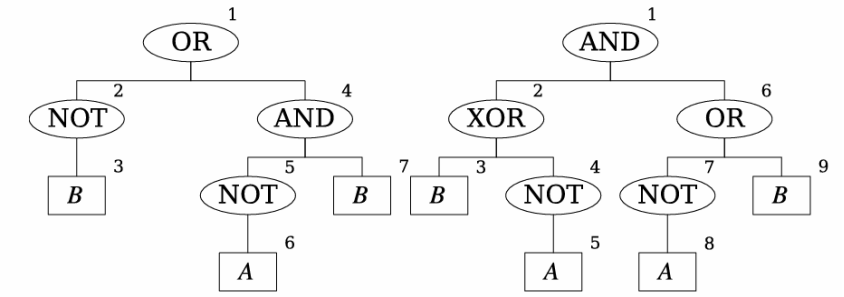
Podemos para hacer un ejemplo sencillo de operaciones lógicas y representarlo en un árbol:

$$((A)\text{XOR}(\text{NOT}(B)))\text{AND}((\text{NOT}(A))\text{OR}(B))$$

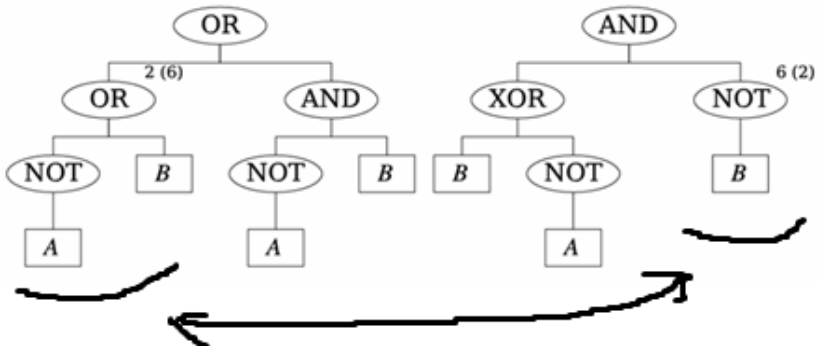


Cruzas en PG

1. Para los dos padres numero los nodos y elijo los puntos a cruzar, ej 2 y 6.



2. Hago la cruza intercambiando las ramas



Mutaciones en PG

1. Numeración de nodos
2. Selección de la rama a mutar
3. Generación de un árbol al azar
4. Mutación en base al reemplazo

Inteligencia de enjambres

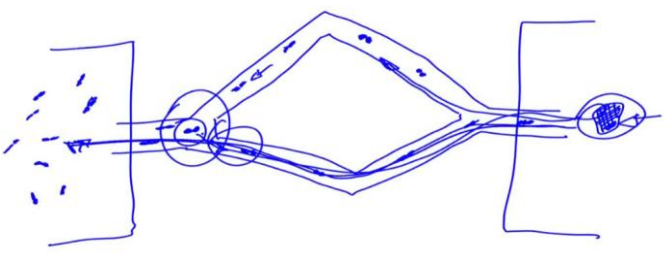
Colonias de hormigas: representación del problema, feromonas, búsqueda de alimento, modelo estocástico, experimento de los dos puentes.

La inspiración biológica

- **Feromonas:** las búsqueda del camino más corto a la comida
  - 1. Comportamiento inicial aleatorio
  - 2. Cuando encuentran una fuente de comida se organizan y comienzan a seguir el mismo camino
    - 2.1 Mecanismo de reclutamiento: mayormente por feromonas, liberadas al regresar (algunas las liberan en proporción a la cantidad de alimento encontrado)
    - 2.2 Si otras encuentran feromonas siguen el rastro (con más probabilidad, tratando de alejarse del hormiguero si no llevan comida)
    - 2.3 El camino se refuerza al ser seguido por más y más hormigas
- Notar:
  - Comunicación indirecta
  - Modificación del entorno físico: estigmergía (uso del medio físico como medio de comunicación)

Experimento del puente binario

La hormiga de abajo llego más rápido, por lo que el camino de feromonas se termina antes, entonces una nueva hormiga va a decidir ir por abajo. El camino de abajo va a estar más marcado que el de arriba.



Así se encontró el camino optimo.

Algoritmos

Definiciones:

- $G = (V, E)$: grafo con vértices V y matriz de conexiones E
- σ_{ij}: feromonas en la conexión entre i y j
- $k = 1,2, \dots, K$: hormigas
- N_i: nodos disponibles a partir del nodo i
- $\boldsymbol{p}^k(t)$: camino de la hormiga k

Notar:

- $t \to t+1$: se incrementa una vez que todas las hormigas encuentran el alimento y vuelven al origen.
- N_i^k: en algunos casos el entorno se limita a los nodos que no haya visitado previamente la hormiga k.

Algoritmo 1: Colonia de hormigas simple (sACO)

1. $t = 0$, inicializar feromonas con valores pequeños al azar $\left(\sigma_{ij}(t) \rightsquigarrow U(0, \sigma_0)\right)$ (U es distribución uniforme)
2. Ubicar K hormigas en el nodo origen.
3. **Repetir**
   - 3.1. **Para** cada hormiga $k = 1,2, \dots, K$
     - 3.1.1. $\boldsymbol{p}^k(t) = \emptyset$
     - 3.1.2. **Repetir**
       - Seleccionar el próximo nodo según la probabilidad

         $$p_{ij}^k(t) = \begin{cases} \dfrac{\sigma_{ij}^\alpha(t)}{\sum_{\forall u \in N_i} \sigma_{iu}^\alpha(t)} & si\ j \in N_i \\ 0 & en\ otro\ caso. \end{cases}$$

         el α nos permite potenciar el efecto de la cantidad de feromonas, esto sirve para que las diferencias pequeñas de feromonas tengan una influencia más grande en las diferencias de probabilidad.
       - Agregar un paso (i, j) al camino $\boldsymbol{p}^k(t)$

       Hasta alcanzar el destino
     - 3.1.3. Eliminar los ciclos en $\boldsymbol{p}^k(t)$
     - 3.1.4. Calcular la longitud del camino encontrado $f\left(\boldsymbol{p}^k(t)\right)$
   - 3.2. **Para** cada conexión (i, j)
     - Reducir por evaporación la cantidad de feromonas:

       $$\sigma_{ij}(t) = (1-\rho)\sigma_{ij}(t)$$

       $(1-\rho)$ *es la tasa de evaporación de feromonas*
     - Depositar feromonas proporcionalmente a la bondad de la solución

       $$\sigma_{ij}(t+1) = \sigma_{ij}(t) + \sum_{\forall k/(i,j) \in \boldsymbol{p}^k(t)} \frac{1}{f\left(\boldsymbol{p}^k(t)\right)}$$

       Como $f\left(\boldsymbol{p}^k(t)\right)$ es la longitud del camino, el deposito de feromonas es inversamente proporcional.
   - 3.3. $t \leftarrow t+1$

   Hasta que todas las hormigas sigan el mismo camino
4. **Devolver** el camino más corto $\left(\downarrow f\left(\boldsymbol{p}^k(t)\right)\right)$

Algoritmo 2: Sistema de hormigas (AS)

Mejora del algoritmo 1

1. $t = 0$, inicializar feromonas con valores pequeños al azar $\left(\sigma_{ij}(t) \leftsquigarrow U(0, \sigma_0)\right)$ (U es distribución uniforme)
2. Ubicar K hormigas en el nodo origen.
3. **Repetir**
   - 3.1. **Para** cada hormiga $k = 1,2, \dots, K$
     - 3.1.1. $\boldsymbol{p}^k(t) = \emptyset$
     - 3.1.2. **Repetir**
       - Seleccionar el próximo nodo según la probabilidad

$$p_{ij}^k(t) = \begin{cases} \dfrac{\sigma_{ij}^{\alpha}(t)\eta_{ij}^{\beta}(t)}{\sum_{\forall u \in N_i^k} \sigma_{iu}^{\alpha}(t)\eta_{iu}^{\beta}(t)} & si\ j \in N_i^k \\ 0 & en\ otro\ caso. \end{cases}$$

         Esto quiere decir probabilidad de moverse del nodo i al nodo j, se divide por la sumatoria de cada posible destino desde el nodo i.

         *η es el **deseo**: es un conocimiento adicional de la hormiga, que define que tanto le conviene ir hacia un nodo.*

         El deseo de moverse es inverso al costo entre nodos: $\eta_{ij} = 1/d_{ij}$ donde d_{ij} es la distancia entre dos nodos u otro costo.

         Lista de nodos vecinos con tabú: N_i^k, de todos los posibles nodos se eliminan aquellos por los que la hormiga ya paso. Esto evitan que se generen ciclos.

         Con α y β defino un mayor o menor peso relativo.
       - Agregar un paso (i, j) al camino $\boldsymbol{p}^k(t)$

       Hasta alcanzar el destino
     - 3.1.3. Calcular la longitud del camino encontrado $f\left(\boldsymbol{p}^k(t)\right)$
   - 3.2. **Para** cada conexión (i, j)
     - Reducir por evaporación la cantidad de feromonas:

$$\sigma_{ij}(t) = (1 - \rho)\sigma_{ij}(t)$$

       $(1 - \rho)$ *es la tasa de evaporación de feromonas*
     - Depositar feromonas proporcionalmente a la bondad de la solución

$$\Delta\sigma_{ij}^k(t) = \begin{cases} Q/f\left(\boldsymbol{p}^k(t)\right) & global \\ Q & uniforme \\ Q/d_{ij} & local \end{cases}$$

$$\sigma_{ij}(t+1) = \sigma_{ij}(t) + \sum_{\forall k/(i,j) \in \boldsymbol{p}^k(t)} \Delta\sigma_{ij}^k(t)$$

   - 3.3. $t \leftarrow t + 1$

   Hasta que todas las hormigas sigan el mismo camino
4. **Devolver** el mejor camino

Q es una constante.

Si es **global**, deposita una cantidad de feromonas dividido la longitud entera del camino, si Q=1 es igual que el algoritmo 1, tiene en cuenta que tan largo o corto fue el camino entero. Favorece los mejores caminos porque la cantidad de feromonas depende de la longitud del recorrido.

Si es **uniforme**, cuenta cuantas hormigas pasa por el camino. No discrimina entre caminos buenos y malos, lo que mantiene la exploración alta.

Si es **local** tiene en cuenta cuanto le costó a una hormiga ir desde el nodo i al j. Favorece trayectorias cortas entre nodos individuales, pero no garantiza que el camino completo sea óptimo.

Enjambre de partículas: representación del problema, restricciones, tamaño de partícula, inicialización, ecuaciones de movimiento, distribuciones de proximidad, topología de las poblaciones.

La inspiración biológica

- Bandadas de pájaros.
- Cada individuo vuela/navega el espacio ajustando su posición en base a su experiencia y a la información de los individuos de su entorno.

Algoritmos

Buscan emular el éxito de los vecinos y propio

- Regla básica: $\boldsymbol{x}_k(t+1) = \boldsymbol{x}_k(t) + \boldsymbol{v}_k(t+1)$
- $\boldsymbol{x}_k(t)$: posición de la partícula k, en el tiempo t (espacio $\mathbb{R}^N$: podría tener temperatura, velocidad del viento, etc)
- $\boldsymbol{v}_k(t)$: velocidad de la partícula k, en el tiempo t
- $\boldsymbol{y}_k$: mejor posición **personal** de la particula k (la mejor que visito desde $t = 0$ hasta la actualidad)
- $f(\boldsymbol{x})$: función de error a minimizar

Algoritmo 3: Enjambre del mejor global (gEP)

- Topología estrella: usa información de todo el enjambre, cada individuo está relacionado con todos.
- $\widehat{\boldsymbol{y}}$: mejor posición visitada por todo el enjambre.

1. Inicializar $x_{ki}(0) \leftsquigarrow U\left(\boldsymbol{x}_i^{min}, \boldsymbol{x}_i^{max}\right)$
2. **Repetir**
   2.1. **Para** cada partícula $k = 1,2,\dots,N$
   - *Si el error con la posición actual es menor que el mejor que tuvo, actualizo el mejor:*
     $$Si\ f\left(\boldsymbol{x}_k(t)\right) < f(\boldsymbol{y}_k) \rightarrow \boldsymbol{y}_k = \boldsymbol{x}_k(t)$$
   - *Si ese individuo es mejor que el mejor individuo global, actualizo el mejor:*
     $$Si\ f(\boldsymbol{y}_k) < f(\widehat{\boldsymbol{y}}) \rightarrow \widehat{\boldsymbol{y}} = \boldsymbol{y}_k$$

   2.2. **Para** cada partícula $k = 1,2,\dots,N$
   - $v_{ki}(t+1) = v_{ki}(t) + c_1 r_{1i}[y_{ki} - x_{ki}(t)] + \rightarrow Experiencia\ personal$
     $c_2 r_{2i}[\hat{y}_i - x_{ki}(t)] \rightarrow Experiencia\ social$

     Donde $r_{1i}, r_{2i} \leftsquigarrow U(0,1)$

     Actualizo la velocidad del individuo k en la dimensión i-esima
     c es la constante de aceleración y r es una constante aleatoria que determina que tanto sigue su conocimiento personal o el conocimiento global.
     r es la componente aleatoria que permite que los individuos no solamente sigan la orientación de acuerdo a su experiencia personal o experiencia social, sino que también explore otras regiones del espacio. Esto evita que caiga en un mínimo local.
   - $\boldsymbol{x}_k(t+1) = \boldsymbol{x}_k(t) + \boldsymbol{v}_k(t+1)$

   Hasta cumplir con la condición de finalización
3. **Devolver** la mejor particula encontrada

Algoritmo 4: Enjambre del mejor local (lEP)

Variante que en vez de tener en cuenta la información social global, tiene una información social más acotada.

- Topología anillo: usa información del entorno cercano a la partícula, cada individuo está relacionado con el que tiene al lado. Seleccionados de la siguiente manera:
- Selección: basada en los índices de las partículas (caso mas simple)
  $$\mathcal{E}_{\mathrm{k}} = \{\boldsymbol{y}_{k-n}, \dots, \boldsymbol{y}_{k-1}, \boldsymbol{y}_k, \boldsymbol{y}_{k+1}, \dots, \boldsymbol{y}_{k+n}\}$$
  $\mathcal{E}_k$ representa la vecindad de la partícula k de radio n
- $\hat{\mathbf{y}}_{\mathrm{k}} = \arg\min_{\forall \mathbf{y}_{\mathrm{u}} \in \mathcal{E}_{\mathrm{k}}} \{\mathrm{f}(\mathbf{y}_{\mathrm{u}})\}$ mejor posición visitada por el entorno de la partícula k

Ahora la información social, es decir la mejor posición no es de todo el enjambre, sino que del entorno de cada partícula.

1. Inicializar $\boldsymbol{x}_{ki}(0) \leftsquigarrow U\left(\boldsymbol{x}_i^{min}, \boldsymbol{x}_i^{max}\right)$
2. **Repetir**
   2.1. **Para** cada partícula $k = 1,2,\dots,N$
   - $Si\ f\left(\boldsymbol{x}_k(t)\right) < f(\boldsymbol{y}_k) \rightarrow \boldsymbol{y}_k = \boldsymbol{x}_k(t)$
   - $Si\ f(\boldsymbol{y}_k) < f(\widehat{\boldsymbol{y}}_k) \rightarrow \widehat{\boldsymbol{y}}_k = \boldsymbol{y}_k$

   2.2. **Para** cada partícula $k = 1,2,\dots,N$
   - $v_{ki}(t+1) = v_{ki}(t) + c_1 r_{1i}[y_{ki} - x_{ki}(t)] + c_2 r_{2i}[\hat{y}_{ki} - x_{ki}(t)]$

     donde $\mathrm{r_{1i}, r_{2i}} \leftsquigarrow \mathrm{U}(0,1)$, $\hat{\mathbf{y}}_{\mathrm{k}} = \arg\min_{\forall \mathbf{y}_{\mathrm{u}} \in \mathcal{E}_{\mathrm{k}}} \{\mathrm{f}(\mathbf{y}_{\mathrm{u}})\}$, y $\mathcal{E}_{\mathrm{k}} = \{\boldsymbol{y}_{k-n}, \boldsymbol{y}_{k-n+1}, \dots, \boldsymbol{y}_{k-1}, \boldsymbol{y}_k, \boldsymbol{y}_{k+1}, \dots, \boldsymbol{y}_{k+n}\}$
   - $\boldsymbol{x}_k(t+1) = \boldsymbol{x}_k(t) + \boldsymbol{v}_k(t+1)$

   Hasta cumplir con la condición de finalización
3. **Devolver** la mejor partícula encontrada

Lógica borrosa

Introducción a los sistemas basados en conocimientos. La borrosidad como multivalencia: incerteza versus aleatoriedad, función de membresía. Geometría de los conjuntos borrosos. Operadores borrosos. Caracterización de conjuntos borrosos. Entropía borrosa.

Introducción

La lógica borrosa surge para modelar cómo las personas piensan y toman decisiones cuando no hay certezas absolutas. En la vida real usamos conceptos que no están completamente definidas, no son verdadero o falsos, no son cosas absolutas, sino que contienen cierto grado de incertidumbre. (“hace frío”, “poné el aire en medio”), y la lógica borrosa busca capturar y manipular ese tipo de reglas lingüísticas.

La idea clave es diferenciar:

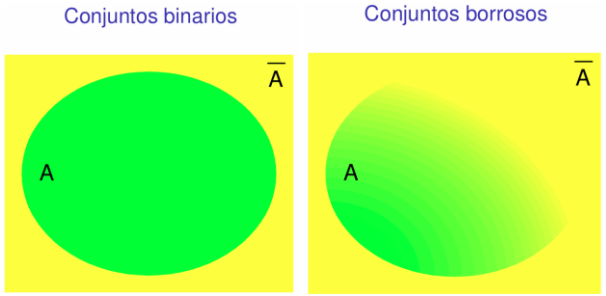
Incerteza (borrosidad)

- Se trata de cuán verdadero es un concepto para **un** objeto.
- No es blanco o negro, sino en grados.
- Ejemplo: “¿Qué tan conocida es una persona?”
- Se maneja con sistemas borrosos.

Aleatoriedad (probabilidad)

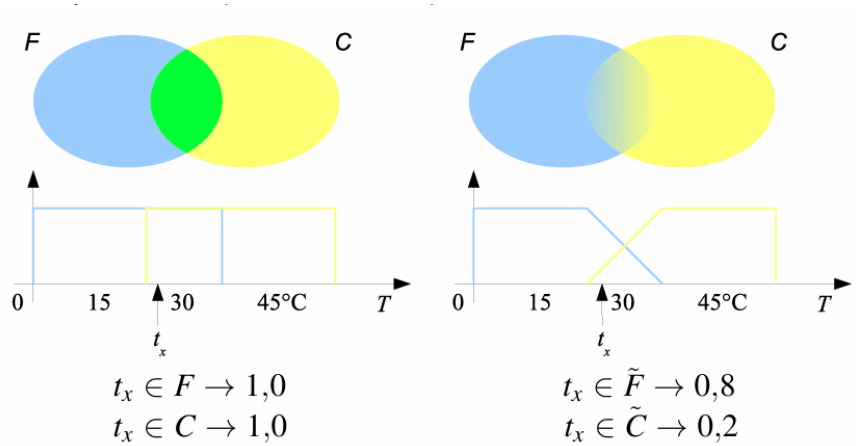
- Se trata de eventos que ocurren muchas veces y tienen cierta frecuencia.
- Se aplica a poblaciones o múltiples casos.
- Ejemplo: “probabilidad de que llueva mañana”
- Se maneja con estadística y probabilidad.

Conjuntos binarios vs conjuntos borrosos



Ejemplos

- **Temperaturas (caso continuo)**

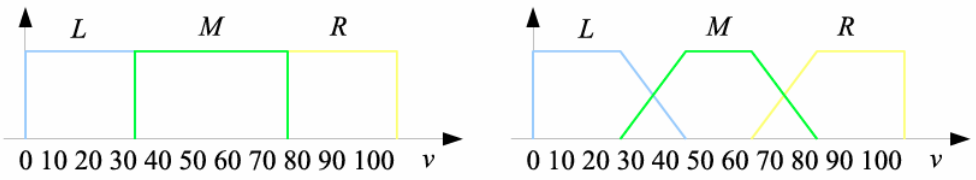


$t_x \in F \rightarrow 1{,}0$
$t_x \in C \rightarrow 1{,}0$

$t_x \in \tilde{F} \rightarrow 0{,}8$
$t_x \in \tilde{C} \rightarrow 0{,}2$

15 grados pertenece al conjunto frio, 45 grados pertenece al cálido, tx pertenece a la intersección de los dos.
En el caso borroso, pertenecen en la intersección en distintos grados, no es rectangular. Puede pertenecer un poco a un conjunto y un poco a otro conjunto.

- **Velocidades (caso discreto)**



$\mu_L = 1\ 1\ 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0$
$\mu_M = 0\ 0\ 0\ 0\ 1\ 1\ 1\ 1\ 0\ 0\ 0$
$\mu_R = 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 1\ 1$

$\mu_{\tilde{L}} = 1\ 1\ 1\ 0{,}6\ 0{,}3\ 0\ 0\ 0\ \ 0\ \ 0\ 0$
$\mu_{\tilde{M}} = 0\ 0\ 0\ 0{,}3\ 0{,}6\ 1\ 1\ 0{,}6\ 0{,}3\ 0\ 0$
$\mu_{\tilde{R}} = 0\ 0\ 0\ 0\ \ 0\ \ 0\ 0\ 0{,}3\ 0{,}6\ 1\ 1$

Representaciones de conjuntos borrosos

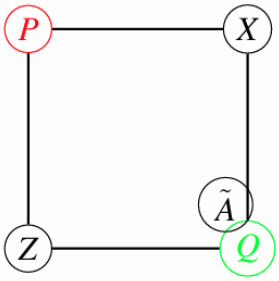
Simplificación a conjuntos de 2 elementos ($\mathbb{R}^2$):

Conjuntos binarios $P = (0;1)$
y $Q = (1;0)$
Conjutnos universo $X = (1;1)$
y vacío $\Phi = (0;0)$

Suponiendo que hay un eje cartesiano, y tengo dos elementos, en el conjunto P no está el primer elemento, pero si el segundo, en el conjunto Q es al revés.

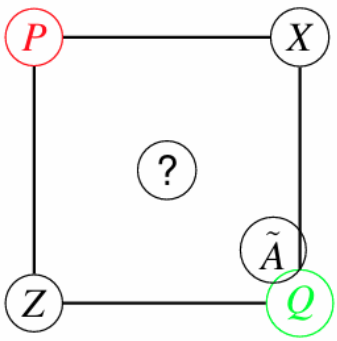
También se puede representar de la siguiente manera:

Vértices de un cuadrado



X= conjunto universal, Z=conjunto vacío.
En esta representación puedo introducir un nuevo conjunto A, que no es por completo del conjunto Q, sino que tiene algo del conjunto P. No es tan puro.

Los conjuntos de las esquinas son los conjuntos ideales, binarios o los conjuntos de los que se tiene certidumbre total sobre la pertenencia o no de los elementos.



Este conjunto en el medio del cual tenemos total incerteza. Es una mezcla del conjunto P y el conjunto Q.

Ponerlos en este esquema nos permite trabajar con esa incerteza y poder desarrollar algoritmos inteligentes a partir de objetos de los que tenemos incerteza

Funciones de membresía o pertenencia

- Conjuntos binarios: $\mu_A: x \rightarrow \{0,1\}$
- Conjuntos borrosos: $\mu_{\tilde{A}}: x \rightarrow [0,1]$

En el conjunto binario, el grado de pertenencia es 0 o 1, en el conjunto borroso el grado de pertenencia puede tomar cualquier número real entre 0 y 1.

Operaciones con conjuntos borrosos

Definición de subconjunto borroso

Sea E un conjunto enumerable y x un elemento de E. Un subconjunto borroso $\tilde{A}$ de E es un conjunto de pares ordenados:

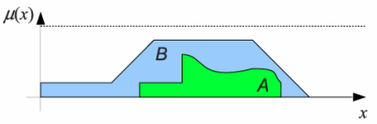
$$\tilde{A} = \{(x_i, \mu_A(x_i))\}; \quad x_i \in E$$

Donde $\mu_A(x_i)$ es el grado de membresía de x en $\tilde{A}$

Operaciones borrosas elementales

- **Inclusión:**

$$\widetilde{\boldsymbol{A}} \subset \widetilde{\boldsymbol{B}} \Leftrightarrow \mu_A(x) \leq \mu_B(x) \quad \forall x$$



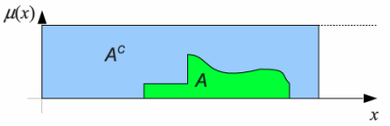
A esta incluido en B si y solo si la pertenencia de todos los elementos en A es menor o igual a la pertenencia de esos mismos elementos en B.

- **Igualdad:**

$$\widetilde{\boldsymbol{A}} = \widetilde{\boldsymbol{B}} \Leftrightarrow \mu_A(x) = \mu_B(x) \quad \forall x$$

- **Complemento:**

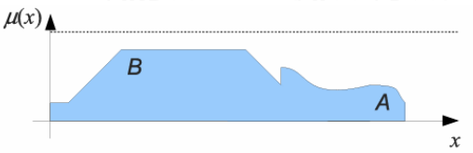
$$\widetilde{\boldsymbol{B}} = \widetilde{\boldsymbol{A}}^c \Leftrightarrow \mu_B(x) = 1 - \mu_A(x) \quad \forall x$$



Si un elemento pertenece 0.2 en A, va a pertenecer 0.8 en B.

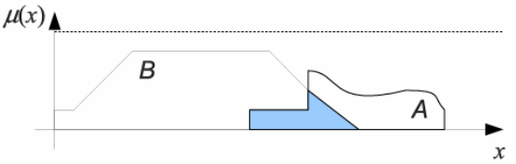
- **Union:**

$$\widetilde{\boldsymbol{A}} \cup \widetilde{\boldsymbol{B}} \Leftrightarrow \mu_{A \cup \boldsymbol{B}}(x) = \max\{\mu_A(x), \mu_B(x)\} \quad \forall x$$



- **Interseccion:**

$$\widetilde{\boldsymbol{A}} \cap \widetilde{\boldsymbol{B}} \Leftrightarrow \mu_{A \cap \boldsymbol{B}}(x) = \min\{\mu_A(x), \mu_B(x)\} \quad \forall x$$



Medidas de distancia entre conjuntos borrosos

Las relativas son para que no se vea afectado por la cantidad de elementos de los conjuntos.

- Distancia de Hamming:

$$d(\tilde{A}, \tilde{B}) = \sum_{i=1}^{n} |\mu_A(x_i) - \mu_B(x_i)|$$

- Distancia de Hamming relativa:

$$\delta(\tilde{A}, \tilde{B}) = \frac{d(\tilde{A}, \tilde{B})}{n}$$

- Distancia euclídea:

$$e(\tilde{A}, \tilde{B}) = \sqrt{\sum_{i=1}^{n} |\mu_A(x_i) - \mu_B(x_i)|^2}$$

- Distancia euclídea relativa:

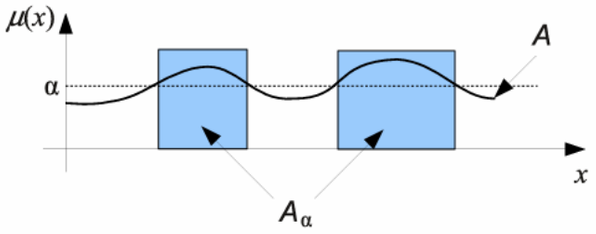
$$\epsilon(\tilde{A}, \tilde{B}) = \frac{e(\tilde{A}, \tilde{B})}{\sqrt{n}}$$

Caracterización de los conjuntos borrosos

Como medir o cuantificar los conjuntos borrosos. Un conjunto borroso puede ser:

- **Conjunto binario de nivel α:**

$$A_\alpha = \{x/\mu_A(x) \geq \alpha \quad \forall x \in A\}$$

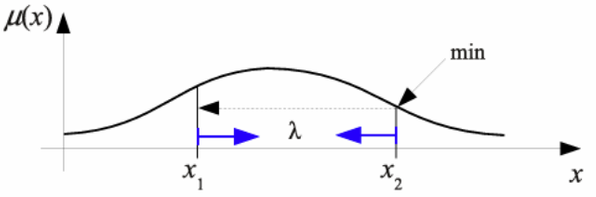


Es un conjunto binario A_α pero esta asociado a un conjunto borroso A.

- **Conjunto convexo:**

Es convexo si:

$$\mu_A(\lambda x_1 + (1-\lambda)x_2)) \geq \min\{\mu_A(x_1), \mu_A(x_2)\}$$
$$\forall \lambda \in [0,1], \qquad \forall x_1, x_2 \in \mathbb{R}$$



O sea que cualquier valor x que tome entre x_1 y x_2 debe ser mayor o igual a la membresía de los extremos. El lambda hace que me vaya más cerca de x_1 o de x_2 (interpolación lineal).

- **Conjunto normal:**

Es normal si:

$$\max\{\mu_A(x)\} = 1$$

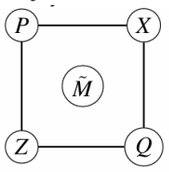


- **Tamaño de un conjunto borroso:**

$$|\tilde{A}| = \sum_{i=1}^{n} \mu_A(x_i)$$

Esta definición es una extensión del tamaño de conjuntos binarios, ya que en conjuntos binarios sumara 0 o 1, contando así la cantidad de elementos.

- **Conjunto borroso medio:**



Es el conjunto más borroso

Es el que todos sus elementos pertenecen con grado de membresía 1/2, Es decir que tengo incertidumbre total. Siempre se tiene la mitad de la certeza de si pertenece o no.

Es borroso medio $\tilde{M}$ si:

$$\tilde{M}/\mu_M(x) = \frac{1}{2} \; \forall x \in E$$

Propiedades:

$$\tilde{M} = \tilde{M} \cap \tilde{M}^c = \tilde{M} \cup \tilde{M}^c = \tilde{M}^c$$

Otras propiedades curiosas

- La unión de un conjunto borroso con su complemento NO da el conjunto universal:

$$\tilde{A} \cup \tilde{A}^c \neq X$$

- La intersección de un conjunto borroso con su complemento NO da el conjunto vacio:

$$\tilde{A} \cap \tilde{A}^c \neq \Phi$$

- **Ejemplo:**

$$A = (0.2,0.8), \quad A^c = (0.8,0.2)$$
$$\tilde{A} \cup \tilde{A}^c = (0.8,0.8), \quad \tilde{A} \cap \tilde{A}^c = (0.2,0.2)$$

Entropía borrosa S

Es como medir el grado de incertidumbre o borrosidad de un conjunto borroso:

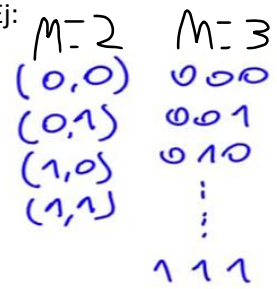
Es la distancia entre el conjunto que queremos medir y el extremo más cercano sobre la distancia entre el conjunto y el extremo más lejano.

$$S(\tilde{A}) = \frac{d(\tilde{A}, I^n_{min})}{d(\tilde{A}, I^n_{max})}$$

Con $I^n = \{0,1\}^n$

Los extremos son n-uplas, es decir combinaciones de 0s y 1s en una cantidad de n. n es la cantidad de elementos que tenemos en el conjunto.

Ej:



En n=2 tenemos el cuadrado, en n=3 tendríamos un cubo.

Entonces vamos a buscar la distancia a la esquina más cercana y la esquina más lejana:

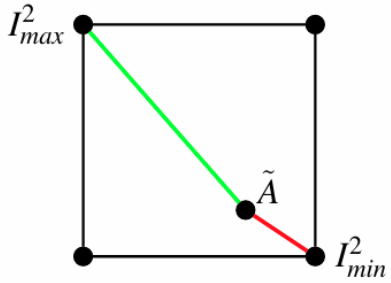
$$I^n_{min} = I^n_{j^*} \Leftrightarrow d(\tilde{A}, I^n_{j^*}) < d(\tilde{A}, I^n_j)\ \forall j \neq j^*$$

$$I^n_{max} = I^n_{j^*} \Leftrightarrow d(\tilde{A}, I^n_{j^*}) > d(\tilde{A}, I^n_j)\ \forall j \neq j^*$$

La esquina $I^n_{j^*}$ va a ser la más cercana/lejana sii la distancia entre el conjunto borroso y esa es quina es menor/mayor a todas las otras esquinas.

Ejemplo:

$A = (0{,}7; 0{,}2)$



$$d_{max} = |0{,}7 - 0{,}0| + |0{,}2 - 1{,}0| = 1{,}5$$
$$d_{min} = |0{,}7 - 1{,}0| + |0{,}2 - 0{,}0| = 0{,}5$$
$$S(\tilde{A}) = \frac{0{,}5}{1{,}5} = \frac{1}{3}$$

Es decir que tiene una entropía borrosa de 1/3 o es 1/3 borroso.

Curiosidades:

$$S(\tilde{M}) = 1{,}0$$
$$S(\tilde{I}^n) = 0{,}0$$

Del conjunto medio me da el mismo número para el mínimo que para el máximo, por lo que es el conjunto más borroso. El conjunto con mayor incertidumbre.

De un conjunto binario tengo certeza absoluta, la distancia más cercana dará 0, por lo que sin importar cual es el denominador, ya me da 0.

Memorias asociativas borrosas como mapeos. Codificación de reglas borrosas. Composición de reglas. Conjuntos de membresía continuos. Varios antecedentes por consecuente.

Memorias Asociativas Borrosas (FAM)

Es un sistema borroso que a partir de las operaciones con conjuntos borrosos pueda realizar ciertas tareas que queremos resolver. Estos sistemas se construyen mediante FAMs, estas intentan codificar las reglas borrosas de las que vamos a realizar algunas acciones.

Introducción

El objetivo es resolver un IF borroso: (regla simple local FAM)

$$\text{if } X = \widetilde{A} \text{ then } Y = \widetilde{B}$$

Si una entrada X pertenece al conjunto borroso A, entonces la salida es el conjunto borroso B.

Donde $\begin{Bmatrix}\widetilde{A} \in I^n \\ \widetilde{B} \in I^p\end{Bmatrix} f: [0,1]^n \to [0,1]^n$

Es decir, tanto A como B están dentro del hipercubo cuyos vértices eran los conjuntos binarios.
f hace un mapeo desde el primer espacio hasta el segundo. Esto lo hacen las reglas borrosas IF-THEN.

Diagrama en bloques:

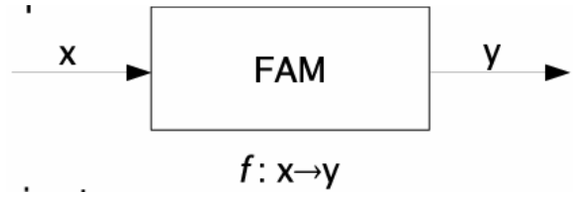
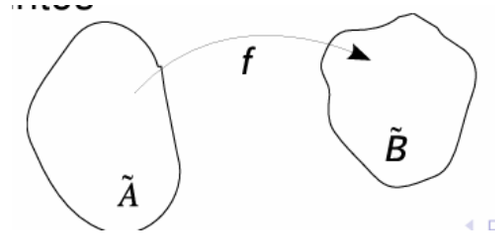
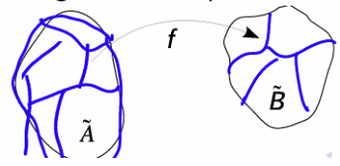


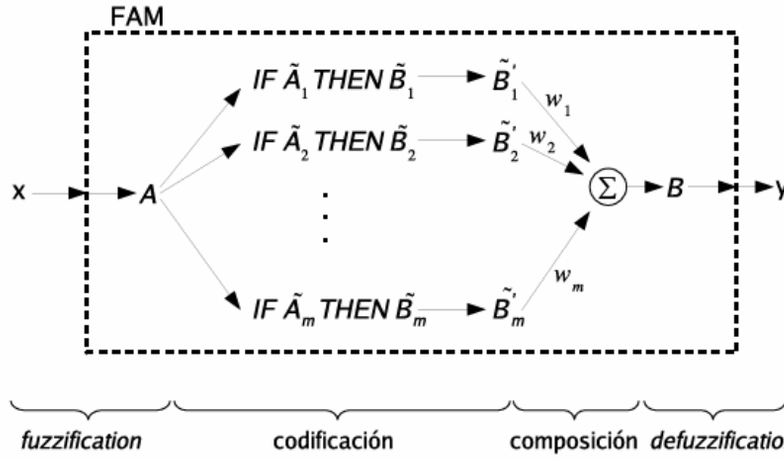
Diagrama de conjuntos:



Puede haber muchas reglas para mapear distintas regiones del espacio de entrada a distintas regiones del espacio de salida:



El FAM que realiza el mapeo de las entradas a las salidas mediante reglas IF-THEN esta compuesto de las siguientes etapas:



Fuzzification: convertir una entrada, un número real en un conjunto borroso.
Codificación: todas las reglas que vamos a aplicar y que definen como se va a comportar el sistema.
Composición: Todas las salidas de la codificación se combinan para encontrar un conjunto borroso de salida.
Defuzzification: convertir un conjunto borroso a una salida, un número para aproximar, clasificar, etc.

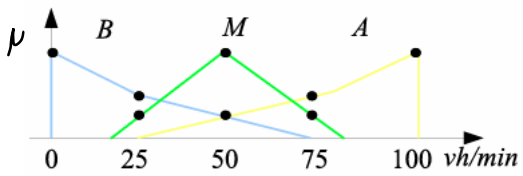
Fuzzyfication: un ejemplo sencillo

Control automático de un semáforo por nivel de trafico

Como entrada tiene estimación del tráfico.

Como salida queremos saber cuántos segundos debe durar el semáforo.

- **Conjuntos borrosos de entrada:**



  Trafico Bajo, Medio, Alto. Discretizamos en los puntitos.

- **Si por ejemplo recibimos como entrada $x = 75$ vehiculos por minuto, las activaciones seran:**
  - $\mu_{BAJO} = 0.00 \Longrightarrow$ IGNORAR reglas de tráfico BAJO: Grado de activación $a_1(x) = 0.00$
  - $\mu_{MEDIO} = 0.25 \Longrightarrow$ ACTIVAR reglas de tráfico MEDIO: Grado de activación $a_2(x) = 0.25$
  - $\mu_{ALTO} = 0.50 \Longrightarrow$ ACTIVAR reglas de tráfico ALTO: Grado de activación $a_3(x) = 0.50$

Codificación

Consiste en construir las matrices M que nos permiten convertir los antecedentes en consecuentes. Codificamos las reglas borrosas.

- **Discretización de los conjuntos de entrada:**

v/min	$\rightarrow$	0	25	50	75	100	
$\tilde{A}_1 = BAJO$	$= ($	1.00	0.50	0.25	0.00	0.00	$)$
$\tilde{A}_2 = MEDIO$	$= ($	0.00	0.25	1.00	0.25	0.00	$)$
$\tilde{A}_3 = ALTO$	$= ($	0.00	0.00	0.25	0.50	1.00	$)$

  Por columnas tengo cantidad de vehículos por minuto, por filas los conjuntos borrosos y por celda el grado de pertenencia de cada velocidad al conjunto.

- **Conjuntos borrosos de salida:**

  tiempo de espera $CORTO$, $NORMAL$ y $LARGO$

- **Discretización de los conjuntos de salida:**

seg.	$\rightarrow$	0	30	60	90	120	
$\tilde{B}_1 = CORTO$	$= ($	1.00	0.25	0.00	0.00	0.00	$)$
$\tilde{B}_2 = NORMAL$	$= ($	0.00	0.50	1.00	0.50	0.00	$)$
$\tilde{B}_3 = LARGO$	$= ($	0.00	0.00	0.00	0.25	1.00	$)$

Voy a codificar la siguiente regla borrosa: (ejemplo para una única regla)

Dicho de distintas maneras:

- `if` x es $ALTO$ `then` y es $LARGO$
- `if` $ALTO$ `then` $LARGO$
- $(ALTO; LARGO)$
- (antecedente, consecuente)

Para codificar la regla IF $\widetilde{\mathrm{A}}$ then $\widetilde{\mathrm{B}}$, lo puedo hacer mediante alguna de las siguientes maneras:

Codificación correlación-mínimo

Voy a obtener la matriz M que codifica la regla IF $\widetilde{\mathrm{A}}$ THEN $\widetilde{\mathrm{B}}$:

$$M = \widetilde{\mathrm{A}}^T \circ \widetilde{\mathrm{B}}$$

Esta matriz se obtiene de calcular el mínimo entre los dos. Ejemplo para una única regla (la de arriba):

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0{,}25 & 0{,}25 \\ 0 & 0 & 0 & 0{,}25 & 0{,}50 \\ 0 & 0 & 0 & 0{,}25 & 1{,}00 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0{,}25 \\ 0{,}50 \\ 1{,}00 \end{bmatrix} \circ [0 \;\; 0 \;\; 0 \;\; 0{,}25 \;\; 1{,}00]$$

Para verificar si se codifico bien, la siguiente operación debe cumplirse $\widetilde{\mathrm{A}} \circ M = \widetilde{\mathrm{B}}$ (“el máximo del mínimo”)

De todos los mínimos busco el máximo:

$$[0 \quad 0 \quad 0.25 \quad 0.5 \quad 1] \circ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.25 & 0.25 \\ 0 & 0 & 0 & 0.25 & 0.5 \\ 0 & 0 & 0 & 0.25 & 1 \end{bmatrix} = [0 \quad 0 \quad 0 \quad 0.25 \quad 1]$$

Codificación correlación-producto

$$M = \widetilde{\mathrm{A}}^T \widetilde{\mathrm{B}}$$

Esta matriz se obtiene de calcular el producto entre los dos. Ejemplo para una única regla (la de arriba):

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0{,}0625 & 0{,}25 \\ 0 & 0 & 0 & 0{,}125 & 0{,}50 \\ 0 & 0 & 0 & 0{,}25 & 1{,}00 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0{,}25 \\ 0{,}50 \\ 1{,}00 \end{bmatrix} [0 \;\; 0 \;\; 0 \;\; 0{,}25 \;\; 1{,}00]$$

Para verificar si se codifico bien, la siguiente operación debe cumplirse $\widetilde{\mathrm{A}}M = \widetilde{\mathrm{B}}$ (“el máximo del producto”)

Codificación: Consideraciones practicas

- Si la etapa de fuzzification es directa y solo admite una de las entradas discretas (la entrada pertenece a un conjunto y a ninguno de los otros), los conjuntos de entrada siempre tendrán la forma de un δ_i
- Siguiendo con el ejemplo de $x = 75$ vehículos/minuto, el conjunto de entrada sería un conjunto binario clásico:
$$A' = (0.00 \quad 0.00 \quad 0.00 \quad 1.00 \quad 0.00)$$
- Con este tipo de conjuntos binarios de entrada, el producto matricial para la regla k se simplifica como
$$A' \circ M_k = A' \circ \left(\tilde{A}_k^T \circ \widetilde{\mathrm{B}}_k\right) = \left(A' \circ \tilde{A}_k^T\right) \circ \widetilde{\mathrm{B}}_k = a_{ki} \widetilde{\mathrm{B}}_k$$
A por A transpuesta me queda el elemento i-esimo de A, por lo que directamente se pueden multiplicar todos los elementos de B por esa cantidad y se va a hacer 0 en los elementos que no corresponda. Da una salida bastante directa.

- Cuando el conjunto de entrada es un conjunto borroso realmente, ya sea por:
  - Provenir de un real proceso de fuzzification
  - Ser el resultado de un sistema borroso en una etapa previa
- El conjunto borroso de entrada podría ser:
$$A' = (0.00 \quad 0.00 \quad 0.25 \quad 0.50 \quad 0.25)$$
- Con lo que vuelve a tener sentido el producto matricial para cada regla $A' \circ M_k$

Composición de reglas

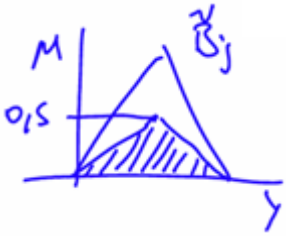
Como voy a tener muchas matrices M codificando, necesito obtener un resultado único, un único conjunto borroso de salida, eso se realiza en esta etapa. Composición de reglas o composición de salidas.

Hay varias alternativas:

- **Superposición de reglas codificadas en matrices:**
  Consiste en armar una matriz M con los máximos de todas las matrices de las reglas k en cada posición.
  $$M = \max_{1 \le k \le m} M_k$$
  Es eficiente pero no conviene porque a medida que incremento la cantidad de reglas, esta matriz tiende a quedar completa con 1s.
- **Modelo aditivo estándar (SAM):**
  $$\widetilde{\mathrm{B}} = \sum_{j=1}^{m} w_j \widetilde{\mathrm{B}}'_j$$
  En lugar de componer las reglas se componen las salidas de cada una de las reglas.
  El $\widetilde{\mathrm{B}}'_j$ es el A multiplicado con correlación mínima o producto por el M correspondiente.
  El peso es para determinar las importancias de las reglas, si hay alguna más critica que otra. O podrían calcularse de manera automática en función de los datos.
- **SAM con factor de activación:**
  $$\widetilde{\mathrm{B}} = \sum_{j=1}^{m} w_j a_j(x) \widetilde{\mathrm{B}}'_j$$
  Es un modelo aditivo estándar con función de activación a_j, esto es con qué grado se activa cada regla.
  El producto de $a_j(x)\widetilde{\mathrm{B}}'_j$ se puede definir como:
  - **Mínimo**: es el equivalente a truncar el conjunto, por ej. para un factor de activación $a(x) = 0.5$

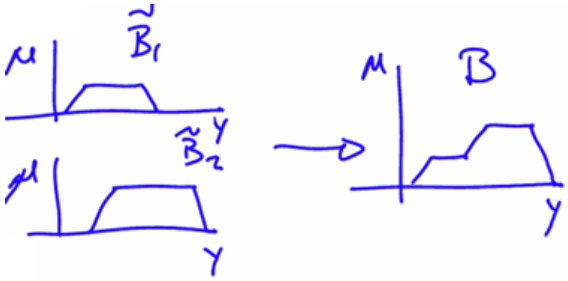
    

  - **Producto:** es el equivalente a escalar el triángulo, por ej. para un factor de activación $a(x) = 0.5$

La sumatoria en SAM y SAM con factor de activacion (es decir la combinacion de conjuntos) tambien se puede definir de varias maneras:

- **Maximo:** Busca en cada punto el maximo entre los dos

- **Suma:** hace la suma de los grados de pertenencia y truncar en 1 en caso de que lo supere.
- **Centroides (compuesta con defuzzification)**

Defuzzification

Convertir el conjunto borroso de salida a un número. Por ejemplo, la cantidad de segundos que tiene que estar en verde.

Repasando: el proceso completo es

- $x \xrightarrow{fuzz} \tilde{A} \rightarrow \underbrace{REGLAS}_{if\ \dots then} \rightarrow \underbrace{SAM}_{\sum w_j B_j'} \rightarrow \tilde{B} \overset{??}{\rightarrow} y$
- **Fuzzification** (sigue ejemplo control de un semáforo):
  Supongo que entro una determinada entrada x y obtengo los siguientes niveles de activación:
  $$x \xrightarrow{fuzz} \begin{Bmatrix} a_1(x) = 0.00 \\ a_2(x) = 0.25 \\ a_3(x) = 0.50 \end{Bmatrix}$$
- **Aplicación de las reglas:**
  - If BAJO then CORTO $\rightarrow \tilde{B}_1 = \tilde{A}_1 \circ M_1$, no activada ($a_1(x) = 0.00$)
  - If MEDIO then NORMAL $\rightarrow \tilde{B}_2 = \tilde{A}_2 \circ M_2$, activada con $a_2(x) = 0.25$
  - If ALTO then LARGO $\rightarrow \tilde{B}_3 = \tilde{A}_3 \circ M_3$, activada con $a_3(x) = 0.50$
- **Composición de las reglas activadas** (simple escala-suma):
  - $\widetilde{\mathrm{B}} = \sum_{j=1}^{m} w_j a_j(x) \widetilde{\mathrm{B}}_j'$
  - $\widetilde{\mathrm{B}} = 0.0B_1 + 0.25B_2 + 0.5B_3$
    (supongo que todos tienen el mismo peso $w_j = 1$)
  - $\widetilde{\mathrm{B}} = 0.25[0 \quad 0.5 \quad 1.0 \quad 0.5 \quad 0] + 0.5[0 \quad 0 \quad 0 \quad 0.25 \quad 1.0]$
  - $\widetilde{\mathrm{B}} = [0 \quad 0.125 \quad 0.25 \quad 0.25 \quad 0.5]$
- **Obtencion de y, la salida no borrosa**
  En este caso es los segundos de semáforo en verde. Puede ser de dos maneras:
  - **Maximum-defuzzification**
  - **Centroid-defuzzyfication**

Maximum-defuzzyfication

$$y = \arg\max_{\mathrm{y_j}} \left(\mu_B(y_j)\right)$$

El y_j serian los segundos del semáforo: 0, 30, 60, 90, 120

Busca el y_j con máximo nivel de membresía

Centroid-defuzzyfication

Pesa cada una de las posibles velocidades y_j por el grado de membresía que tiene en el conjunto $\mu_B(y_j)$. Dividido para normalizar.

$$y = \frac{\sum_j y_j \mu_B(y_j)}{\sum_j \mu_B(y_j)}$$

Es más equilibrada.

FAM: Extensiones útiles

- **Conjuntos de pertenencia continuos:** Nosotros habíamos simplificado para conjuntos discretos.
- **Representación de dos antecedentes por consecuente**: Por ejemplo, además del tráfico, si llueve como entrada.
- **Composición de dos antecedentes**

Conjuntos de pertenencia continuos

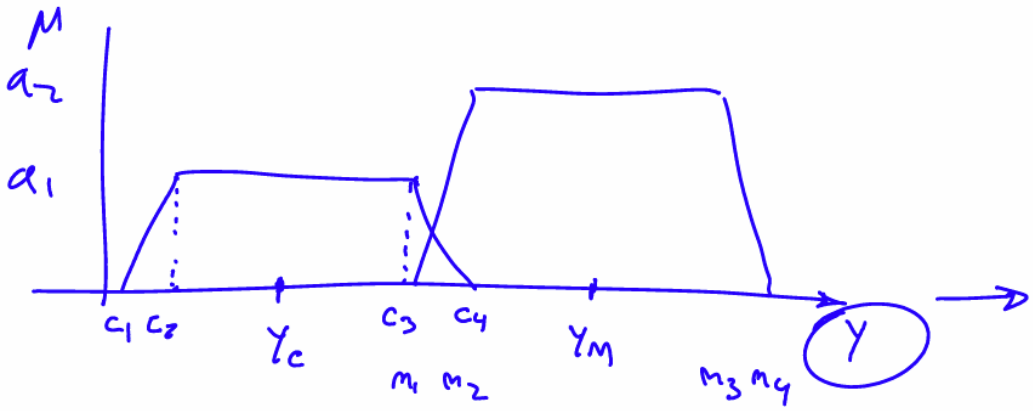
- Los conjuntos son funciones continuas
- No existen las matrices M
- Método simplificado para guardar los trapecios
- Evaluación de reglas (una a una…)
- Composición de los conjuntos de cada regla y deffuzzification

Composición de los conjuntos de cada regla y defuzzyfication para los *centroides trapezoidales*

Supongo que tengo dos conjuntos con niveles de activación a_1 y a_2.

Composición de los conjuntos de cada regla y defuzzyfication para los *centroides trapezoidales*

- Representación gráfica



Y_c y y_m son los centroides

- Cálculo: $y = \frac{\sum_j y_j B_j}{\sum_j B_j} = \frac{y_c B_c + y_m B_m}{B_c + B_m}$

Multiplico la posición del centroide del conjunto c por el área del conjunto c + la posición del centroide del conjunto m por el área del conjunto m.

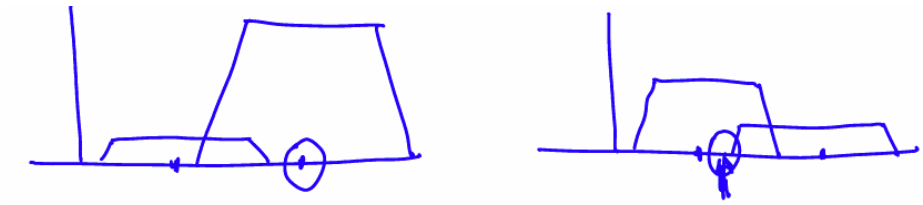
- Suponiendo $c_2 - c_1 = c_4 - c_3$ y $m_2 - m_1 = m_4 - m_3$:

Es decir, supongo que las distancias de los dos trapecios son iguales.

$$y = \frac{y_c a_1 [(c_3 - c_2) + (c_2 - c_1)] + y_m a_2 [(m_3 - m_2) + (m_2 - m_1)]}{B_c + B_m}$$

Es base por altura

Multiplica por los centroides, esos centroides están pesados por el área del trapecio:

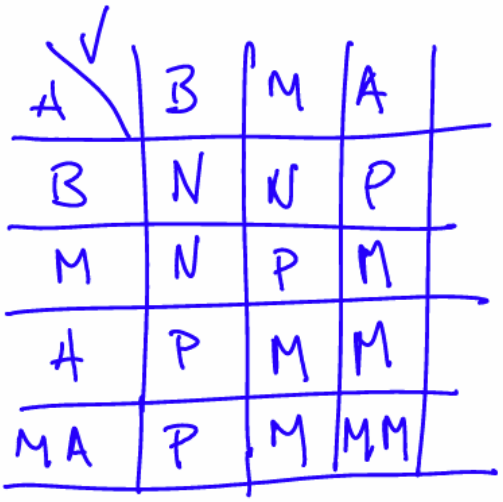


Simplificando queda asi:

$$y = \frac{y_c a_1 (c_3 - c_1) + y_m a_2 (m_3 - m_1)}{a_1 (c_3 - c_1) + a_2 (m_3 - m_1)}$$

Representación de dos antecedentes por consecuente

Para esto se utiliza una tabla de doble entrada, por ejemplo, si queremos saber si presionar el freno en base a la velocidad y aceleración:



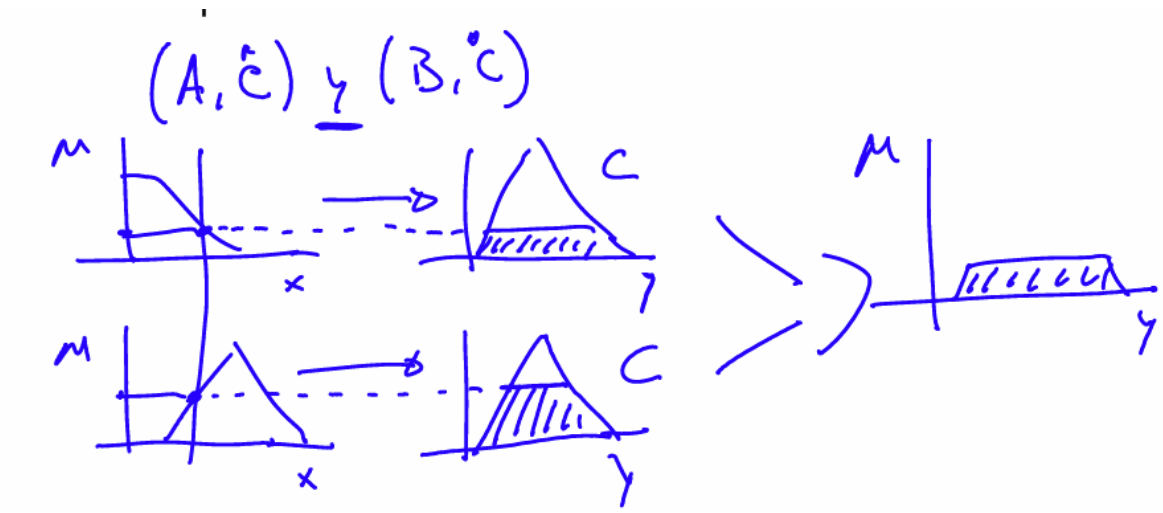
A \ V	B	M	A
B	N	N	P
M	N	P	M
A	P	M	M
MA	P	M	MM

12 reglas que dependen de dos antecedentes. Los consecuentes pueden ser apretar el freno Nada, Poco, Mucho, Muy Mucho

Si hubiera dos consecuentes se podrían armar dos tablas. Por ej, uno para el freno y otro para el acelerador.

Composición de dos antecedentes

Como combinar dos antecedentes.
Supongo que tengo dos antecedentes (A y B) cada un con su consecuente C.



La entrada activa en cierta cantidad un conjunto y en cierta cantidad del otro, tenemos asociado para los dos un mismo conjunto de salida, el grado de activación va a truncar el conjunto, y como se combina con un AND, se toma el mínimo.